



Cryptographie de l'identité

Paul Germouty

► To cite this version:

Paul Germouty. Cryptographie de l'identité. Cryptography and Security [cs.CR]. Université de Limoges, 2018. English. NNT : 2018LIMO0041 . tel-01920497

HAL Id: tel-01920497

<https://theses.hal.science/tel-01920497v1>

Submitted on 13 Nov 2018

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

UNIVERSITÉ DE LIMOGES
Ecole Doctorale SISMI (610)
XLim

Thesis to Obtain the title of:
DOCTEUR DE L'UNIVERSITÉ DE LIMOGES in Computer Science
Presentation and Defence by:

PAUL GERMOUTY

September 13, 2018

Identity-based Cryptography

Directed by: **Olivier Blazy** and **Duong Hieu Phan**

Reviewers:

Sébastien Canard (*Orange*)

Damien Vergnaud (*Sorbonne Université and Institut Universitaire de France*)

Examinators:

Céline Chevalier (*Université Paris II - Panthéon Assas*)

Philippe Gaborit (*Université de Limoges*)

Acknowledgements

I would never be doing this PhD without Olivier, it was not always easy mostly because nothing prepare you to start a PhD. Everything is new and you can easily get lost in what you have to do. But Olivier has always been kind and comprehensive no matter how hard were the times, he is a good person¹. I hope we will be able to work a bit together in the future. I also would like to thank Hieu for the time he took to help me figure out how to express my ideas and the work we have done together. Let's not forget Philippe which played many roles even before the start of this PhD. He puts me in relation with Olivier, helped me figure out what I wanted to do in terms of research and directed my PhD for my first year. He also explained to me why I should not worried about a reject about a paper. I want to thank Céline for the work we have done together and the help she gave me for my different presentations.

I also thanks the reviewers of my thesis Sébastien Canard and Damien Vergnaud for their interesting comments that helped me in improving this document.

Realizing a PhD is hard. Not only because a work of research is hard but also because it is new. Before it, the studies are quite linear, we learn things that people already discovered, that we never truly doubt. Beginning a work of research is about doubting. Doubting about the papers you read but mostly doubting about yourself. This is very important, the more you doubt the more you criticize your work and the more you improve it. At some point you have to stop doubt your ideas and have faith in them and in you. This is also important because having faith in what you do help to convince other researchers that what you do is good and that it can be published and used.

This crucial point between the doubt and the faith is hard to place. It is even harder when this is your first years of research and you don't know how much you are worth. The problem is that I do not trust myself that much. This is why a lot of people were necessary to my success and I will never thanks them enough for it: my Wife, my parents, my sister, my friends Xavier, Zoé, Mickaël, Mathieu, Maël, Adrien, MG, Joëlle, Tam, Florent. Also sport helped me to end this hard last year so I would like to thanks every people I met in the climbing club Climb Up. Please forgive me if I forgot you in my acknowledgement.

¹This is for me one of the best compliment

Contents

1	Introduction	4
1.1	History of Cryptography	4
1.1.1	Antic Cryptography	4
1.1.2	Cryptography in the Middle Ages	5
1.1.3	Modern Era	6
1.2	High Level Definitions	8
1.3	This Work	9
2	Definitions And Preliminaries	12
2.1	Pairing Based Cryptography	12
2.1.1	Diffie-Hellman Problems	12
2.1.2	Security Assumption: Pairing Groups and Matrix Diffie-Hellman Assumption.	13
2.1.3	Security Models	15
2.2	Parameters and Efficiency	16
2.3	Basis Of Cryptography: Encryption	16
2.4	Digital Signatures	19
2.4.1	Definition and Security	19
2.5	Identity-based Encryption	20
2.5.1	Definition and Security	20
2.5.2	IBE's State of The Art	22
2.5.3	Instantiation From [BKP14]	24
2.5.4	Signature from Identity-based Encryption	25
2.5.5	Interesting Features and Properties	25
2.6	Non Interactive Zero Knowledge (NIZK) Argument	28
2.7	Commitments and Smooth Projective Hash Functions	29
2.7.1	Definitions and Security	29

2.7.2	Examples	32
2.8	Oblivious Transfer	34
2.8.1	Definition and Security	35
2.8.2	Example	37
3	New Featured-Identity-based Encryption and Generalization	39
3.1	Downgradable Identity-based Encryption	39
3.1.1	Downgradable Identity-based Encryption	39
3.1.2	Generalization of Existing Id-based Primitives	41
3.1.3	Instantiation and Proof of security	45
3.2	Blind IBE and Fragmented IBE	52
3.2.1	Constructing a Blind Fragmented IBKEM from an IBKEM	52
3.2.2	Pairing-Based Instantiation	55
4	Applications of Identity-based Encryption	58
4.1	Identity Based Designated Verifier Signature	58
4.1.1	Definition and Security	59
4.1.2	A New Construction	61
4.1.3	Moving To The Identity-based Context	64
4.2	Attribute-based Encryption from Downgradable IBE	68
4.2.1	Definitions and Security	68
4.2.2	Transformation from DIBE to ABE	69
4.2.3	Efficiency comparison for ABE	71
4.3	From IBE To Oblivious Transfer	72
4.3.1	Oblivious Transfer Formalism and Main Issues	72
4.3.2	High Level Idea of the Construction:	73
4.3.3	Generic Construction of Adaptive OT	74
5	Oblivious Transfer Generalization and New Approach	80
5.1	Oblivious Language-based Envelope	80
5.1.1	Oblivious Signature-Based Envelope	80
5.1.2	Definition of an Oblivious Language-Based Envelope . . .	82
5.1.3	Security Properties and Ideal Functionality of OLBE . . .	83
5.1.4	Generic UC-Secure Instantiation of OLBE with Adaptive Security	85
5.1.5	Oblivious Primitives Obtained by the Framework	90

5.2	Very Efficient Oblivious Transfer	91
5.2.1	Password Authenticated Key Exchange	91
5.2.2	Generic Construction of a UC-Secure OT Scheme	93
5.2.3	A Warm Up	100
5.2.4	Very Efficient Oblivious Transfer from QA-NIZK	105
5.2.5	Applying the Framework to Obtain a UC-Secure OT Scheme	106
6	Conclusion	109

Chapter 1

Introduction

1.1 History of Cryptography

In this section we will see that the willingness of hiding messages is very old. For ages the systems used to hide a message needed a common knowledge between the two parties that are exchanging messages. We will also see that there exists a kind of cryptography which does not need a common knowledge or a common key to encrypt a message. This kind of cryptography is very young compared to the former one. We call the former: symmetric cryptography or secret-key cryptography, and the latter: asymmetric cryptography or public-key cryptography. For this part we will focus only on encryption techniques e.g. the ways to transform a text such that only few people can understand it. Then we will try to describe the nowadays problems to understand our special needs in cryptography.

1.1.1 Antic Cryptography

The Scytale: one of the most ancient example of cryptography is the use of the tool called the Scytale. It was used between the 10th and 7th century BC in Greece. It is a simple stick with a precise diameter. To encrypt a message the sender will wrap a piece of tissue on the stick and write the message on it (see Figure 1.1). To decrypt

the receiver will use his own scytale, wrap the tissue on it and simply read the message. The security of this scheme lies in the diameter of the scytale. The diameter of the scytale can be



FIGURE 1.1: SCYTALE TECHNIQUE
(CREDITS: WIKIPEDIA)

though as a key of encryption and decryption. The problem with this technique is that there is not enough diameters possible to align the letters. Thus an eavesdropper knowing the method of encryption can try all the possible alignments of letter, and find the one giving meaning to the text. It allows to decrypt this message but also all the other encryptions since the size of the scytale never changes.

The Caesar's Cipher: A very known example is the one call the Caesar's Cipher. The idea is to shift the letters of the alphabet by a precise number n (that will be the encryption/decryption key). It is described in Figure 1.2. To decrypt a message the receiver will do the same operation but shifting in the reverse order. In a sense this technique has the same problem than the previous one: it is possible to test different number for n , since there are only 26 letters in the alphabet. This means that if the method of encryption is known, the scheme is no longer secure. We can say that the size of universe of possible keys is too small. This is not the only problem, for example a letter will always be encrypted in the same letter.

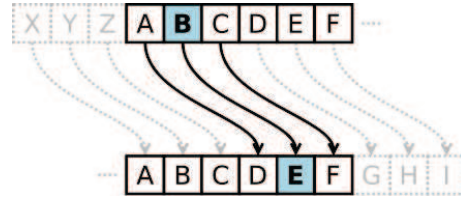


FIGURE 1.2: CAESAR CIPHER
(CREDITS: WIKIPEDIA)

1.1.2 Cryptography in the Middle Ages

Kerckhoffs principles: At the end of the 19th century, Auguste Kerckhoffs wrote 6 principles that¹ any cipher system should satisfy. They will guide the future creators to keep in mind what is important to achieve for an encryption scheme.

- the system must be practically, if not mathematically, indecipherable
- it should not require secrecy, and it should not be a problem if it falls into enemy hands
- it must be possible to communicate and remember the key without using written notes, and correspondents must be able to change or modify it at will
- it must be applicable to telegraph communications
- it must be portable, and should not require several persons to handle or operate
- lastly, given the circumstances in which it is to be used, the system must be easy to use and should not be stressful to use or require its users to know and comply with a long list of rules.

¹in his opinion

The two last principles are less relevant in our world² and the third is not respected. The most important is the second one that we could reformulate as: "the security should lie in the knowledge of the key, not in the method of encryption". This principle excludes some kind of schemes like the 2 firsts that we saw because of the small size of the possible "key" universe. Indeed the fact that the size of potential keys set is small allows to guess the key and then the security is not anymore relied on the knowledge of the key. However the next example that does satisfy this second principle³:

Vigenere Cipher: This encryption scheme has been created at the end of the 16th century. It is based on the Caesar Cipher with a whole word as key: to encrypt the sender will copy the key word under the text has many times needed to "cover" the whole text. Each letter of the key word represent a number: its position in the alphabet. Each letter of the text will be shifted in the alphabet by the number represented by the letter under it. For example the text *PLAINTEXT* encrypt with the key word *KEY* (e.g. the numbers 11, 5, 25) will be *ZPYS-RROBR*. The first letter has been shift by 11, the second one by 5 then 25 then again 11 etc... There is a table helping the encryption in Figure 1.3. The decryption is made by shifting in the other way in the alphabet. This scheme seems as secure as n simple Caesar Cipher, n being the size of the key word. Because the adversary can cut the text in n pieces and then solve n different Caesar cipher. However n is secret so it has to be guessed first⁴. In this encryption scheme it is hard to guess the key because the key can be any word. In the case where the size of the word is equal to the size of the text, the scheme is perfectly secure. It is called the Vernam Cipher.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

FIGURE 1.3: VIGENERE CIPHER
(CREDITS: WIKIPEDIA)

1.1.3 Modern Era

The Enigma Machine: During World War 2 the German army and their allies used a machine called Enigma. This complex machine was easy to use because the sender only had to press the button of the letter he wants to encrypt and the machine responds by enlightening the outputted letter. This machine creates an electrical circuit mapping 2 different letters. The decryption phase is done the same way as the encryption. This machine was very secure because the electrical circuit between the letters change each time a button is pressed.

²it still apply in lightweight cryptography for example

³Note that the Vigenere Cipher is anterior to the principles of Kerckhoffs, Kerckhoffs might have seen what in this cipher was great and learn from it the second principle

⁴there exist simpler method to break this scheme

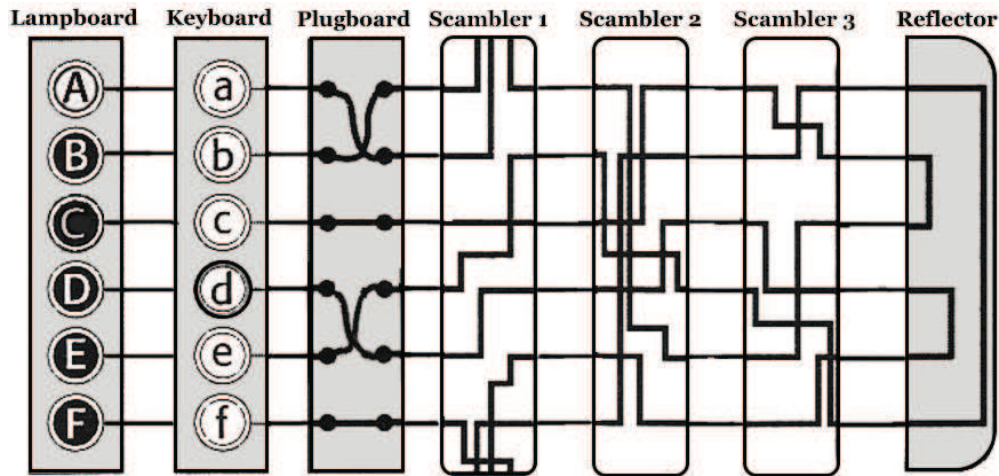


FIGURE 1.4: THE ENIGMA MACHINE

The key of encryption is the initial position of the scrambler and choose of the plugboard (see Figure 1.4). Without going into too much details, this machine has been broken due to: first the fact that a letter can not be encrypted in itself decreasing hugely the complexity of any attacks, second the knowledge of a lot of plaintext/ciphertext couple (mostly weather records).

The few examples we have seen are all using the same key for encryption and decryption. This forces the two party to share something before starting exchanging. This is called symmetric cryptography. Finding systems where no secret data are shared before the first exchange was a more challenging problem and find a solution with the Diffie-Hellman cryptosystem in 1976 and the famous RSA system in 1977 [RSA78]⁵. This systems belongs to the public key cryptography.

In this Thesis we look at schemes that go a step further than a simple public key cryptography: they do not need public key to encrypt a message to someone we only need some general public informations and the identity of the receiver. They are called identity-based encryption schemes which are a part of the more general identity-based cryptography.

Today: an era of connections

The last 30 years have seen the rise of Internet. This brings us to consider the digital security as a major problem of this decade and the next ones. This is even more essential with the exponential growth of the Internet of things which forces us to consider a huge number of possible devices constantly connected to Internet. For example, the use of these devices brought DDoS (Distributed Denial of Service) attacks to a whole new level that was not anticipated.

⁵Clifford Cocks created the same kind of system few years before but could not share his work because it was classified by the British Government.

Many aspects of security are desirable. We will list some of them with examples to highlight the central role of the digital security in this new world. We want to ensure the security of the private information of the users. For example we do not want to share our bank card number that we could have enter in an application or an online market account. This example is obvious and so the security of this kind of sensitive data are generally securely handle. There is more concern to have about data that are defined less sensitive. For example the identity of the customer, his name, his address, the name of his close family member or customer related information that could have been harvest like the political opinion, the sexuality ... Those are information that may not seem sensitive. In fact this kind of information can be used in different way that could harm the user through harassment, discrimination ...

We want to ensure the security of the devices/algorithms used on the wild connected world. Indeed an encrypted message should remain secret for every non receiver party. This is this aspect of cryptography that is considered central for the population. Another example which the population does not seem to care a lot about is the possibility of a device to be corrupted. Imagine a device of the Internet of things that could be remotely controlled. This is not a big deal if someone hack a pillow to see how good was the sleep of the owner but it seems more a problem when the device is a car or a medical device. Those are dramatic examples but we can have different interest in taking the control or damaging devices. For example, if the home-thermostat are not secure enough then a gas company could slightly change the temperature measured by the device allowing this company to make bigger profits without anyone noticing.

These examples show how important it is to secure this new world and I hope it somehow responds to the "I have nothing to hide" sentence that we hear too often. So we want to secure this world but as in the real life there is always a trade off between security and freedom. In the context of digital security freedom will be efficiency/speed: the more we secure a simple message we send the more we need time or power of computation to achieve it.

In this Thesis we are going to investigate a very secure aspect of the public key cryptography and thus our solutions will have a little overcost⁶ compared to less secure solutions. But we stress that our solutions are practical and can be used with an overcost in term of computations and communications very acceptable (with regard to the level of security achieved). Also we will talk a lot about Identity-based cryptography which will be described in the following. This kind of cryptography solves a lot of problems when there is a lot of users in the system which is the case in our time.

1.2 High Level Definitions

Identity-based Encryption The concept of encryption is understood by most

⁶which can be in terms of computation or communication

of people. An identity-based encryption (IBE) is an encryption scheme where there is no need to know a public key to encrypt for a user. Only the identity of the recipient and some general public information (called a master public key) need to be known by the sender. For example an identity can be a name or a mail address (because of its uniqueness). This particular kind of encryption has many advantages: users do not need to store a public key for each user they might discuss with. If the keys can be hold by a trusted authority in the context of the PKI architecture, in the identity-based context the users do not need to trust this kind of authority any more. The strongest advantage and maybe the most underrated is the fact that IBE schemes allows to compute ciphertexts for a string of bits. We will develop that in this Thesis.

The disadvantages of this kind of encryptions is the difficulty to create them. It took several years between the idea of IBE by Shamir [Sha84] and the realization of this idea [BF01]. It needed a new and heavy tool called pairing to be achieved and at this day there is very few other efficient way to create IBE.

Note that there exists many kind of Identity-based cryptography like Identity-based Signature that we will be used in this Thesis.

Oblivious Transfer (OT) In our research about Identity-based Encryption we will encounter and then study protocols called Oblivious Transfer. An Oblivious Transfer is an interactive way to exchange files in a database with a very high degree of secrecy. In fact a user will be able to get a file from a database without this database knowing which file has been requested. On top of that the database does not want the user to have access to a file he does not query. In the context of the security of protocols we need to be very careful. If the protocol is secure when used as it should be used it can be insecure when used dishonestly. The UC model [Can01] is the security model that ensure that the security remains the same in both context. Highly secure Oblivious Transfer will be proved secure in the UC model.

1.3 This Work

What was the idea behind this Thesis? Most of the things existing in the world are designed or used for a special purpose. However sometimes people voluntary or involuntary find a new use to this objects. This is not a new thing, for example the flint was used by the first kind of Homo 1.6 millions years ago to cut meat or leather. But approximatively 1.4 millions year after they found out that this same flint could be used to create fire. Once they learned that this new application to the flint was possible, the deep knowledge about the use of a flint helped them to quickly master the creation of fire. This example highlights the fact that using known concepts or tools to create new ones is sometimes a good starting point. Indeed it is easier to work in a familiar environment. Moreover the idea of using anything in a strange way and make it work and sometimes efficient is itself very funny.

This was in fact the true idea behind this Thesis: "How far can we go with" and we focused on the case of "How far can we go with Identity-based Encryption". We got a bit lost in the way when studying Oblivious Transfer itself but it was also an interesting part of the journey. Obviously this Thesis does not set the limits of the use of IBE but knowing where we can go is knowing that the limits are further. Roughly speaking: How far can we go? Well the answer is beyond what we have done here.

What do we achieve in this Thesis?

Downgradable Identity-based Encryption (DIBE) We manage to generalize a whole class of different identity-based encryption schemes. We called this new scheme Downgradable Identity-based Encryption. This is an Identity-based Encryption that enables key delegation with some conditions: a user with the identity id can recover a user secret key of any identity $\tilde{\text{id}}$ where if $\tilde{\text{id}}_i = 1$ then $\text{id}_i = 1$, we note $\tilde{\text{id}} \preceq \text{id}$. This condition gives some kind of power to bits equal to 1 compared to bits equal to 0. We find out that this special type of hierarchy is present in Attribute-based scheme. Indeed in an Attribute-based Encryption a user is able to decrypt a message if he verifies a policy made of attributes. We can draw a parallel between the role of an attribute and the role of a 1 in an identity, if each attribute is represented by a bit in an identity string then to encrypt for the conjunction of several attribute, we only need to encrypt for the identity made of 0's and having 1's at the position of these attributes. This is our starting point for the Section 4.2. We also detail the transformations from DIBE to Hierarchical IBE, wicked IBE and wildcarded IBE, and an instantiation of DIBE derived from the Hierarchical Identity-based Encryption from [BKP14] in Section 3.1. This work is still in the process of submission.

Identity-based Designated Verifier Signature We use Hierarchical IBE to construct a special type of signature called Identity-based Designated Verifier Signature. In fact we first create a secure and efficient Designated Verifier Signature scheme using implicit decommitment, namely Smooth Projective Hash Functions, to verify a signature without being able to reveal the signer. Then we describe a way to transform it into an Identity-based scheme. This work has been published in the article [BCGJ17] and is detailed in Section 4.1

Oblivious Transfer from Blind IBE First we find a way to securely and efficiently transform any IBE into a blind IBE described in Section 3.2, in this context a user can ask for any secret key without the authority knowing which identity is the target of this request. This transformation is done using Smooth Projective Hash Functions allowing user to commit a request that will be implicitly decommit to answer the request. This technique is even more efficient with affine IBE. Using this new efficient primitive we are able to instantiate an adaptive UC-secure Oblivious Transfer: first the database is encrypted with an IBE, each line is encrypted for the identity equal to the bit string corresponding to the number of the line we are looking at. Then it is sent to the user and

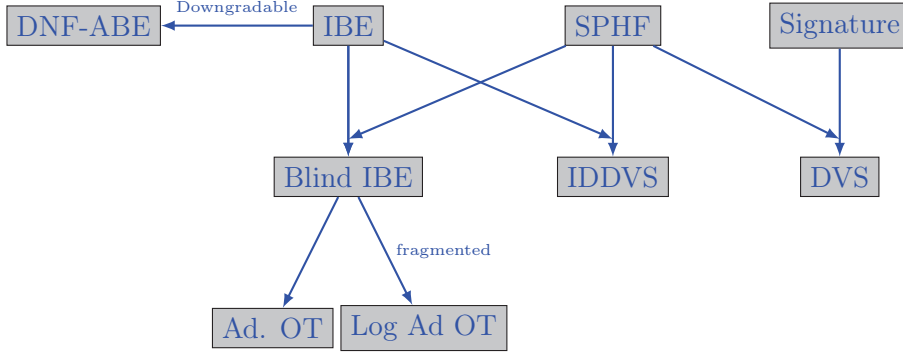


FIGURE 1.5: RELATIONS BETWEEN PRIMITIVES IN THIS THESIS

then the user asks for user secret key as for the Blind IBE scheme. There are some tricks to achieve the UC-security and decrease communication costs, it is described in section 4.3. This work has been published in the article [BCG16].

Oblivious Transfer Generalization We are able to generalize different schemes such as Oblivious Transfer and Oblivious Signature-based Envelope into the concept of Oblivious Language-based Envelope and even give a general UC-secure instantiation. This new protocol allows a user to recover one or several informations if it owns a word in one or several languages. Depending on the type of language this scheme can instantiate different protocols. Our instantiation also makes use of Smooth Projective Hash Functions. This work has been published in [BCG16].

Very Efficient Oblivious Transfer from Password Authenticated Key Exchange During the study of Oblivious Transfer we find out that it was possible to instantiate it using Password Authenticated Key Exchange. We apply and optimize this transformation to multiple PAKE and the one from [JR15] gave the most efficient UC-secure Oblivious Transfer. This work is described in Section 5.2.3 and has been published in [BCG17].

Outline In the second chapter we will first describe the different primitives/tools that we will need for our work. Each time we will define the notion, define the security requirement and give examples. The next chapter is about the work on the Identity-based Primitives as a stand alone. After that we will use this work and apply it. The last chapter is about Oblivious Transfer: generalizations on one side and efficient instantiation from PAKE on the other side.

We summarize all the link between primitives that we found in this thesis in Figure 1.5.

Chapter 2

Definitions And Preliminaries

2.1 Pairing Based Cryptography

2.1.1 Diffie-Hellman Problems

In provable cryptography, the security relies on supposedly hard assumptions. In other words we say that a scheme is secure if the underlying hard assumption holds. There are many candidates of hard assumptions, a notable one is the hardness of the discrete logarithm (DL) problem: let $\mathbb{G}$ be a group of order q and g, h two elements of this group. The discrete logarithm of h in basis g is the element $a \in \mathbb{Z}_q$ such that $g^a = h$ ¹. The discrete logarithm problem is to recover a knowing g and h .

Another notable assumption is the Computational Diffie Hellman assumption (CDH) [DH76]. Given a group $\mathbb{G}$ and 3 elements of this group: (g, g^a, g^b) . It is hard to compute g^{ab} . The decisional version of this assumption states that it is hard to distinguish g^{ab} from a random element. An adversary solving the computational problem can trivially solve the decisional one. More formally:

Definition 1 (Decisional Diffie-Hellman Assumptions DDH). *We say that the Decisional Diffie-Hellman Assumption holds relative to $(\mathbb{G}, g)$ a group and its generators. if for all Probabilistic Polynomial Time (PPT) adversaries $\mathcal{D}$,*

$$\mathbf{Adv}_{\mathcal{D}_k, \mathbb{G}\text{Gen}}(\mathcal{D}) := |\Pr[\mathcal{D}(\mathcal{G}, g^a, g^b, g^{ab}) = 1] - \Pr[\mathcal{D}(\mathcal{G}, g^a, g^b, h) = 1]| = \text{negl}(\lambda),$$

where the probability is taken over the group generation algorithm, $(a, b) \xleftarrow{\$} \mathbb{Z}_p^2$, $h \xleftarrow{\$} \mathbb{G}$.

Solving the discrete logarithm problem allows to solve the Diffie-Hellman (DH) assumptions thus DH assumptions is weaker than the DL one. In other words a scheme secure under the DL assumption is also secure under the DH assumption. In this thesis we will use a lot of variants of the decisional Diffie-Hellman assumption such as the matrix Diffie-Hellman or the bilinear DH (BDH).

¹if it exists, it is sure if g is a generator of $\mathbb{G}$

² $\mathbb{Z}_p$ is the group $\mathbb{Z}/p\mathbb{Z}$ with p a prime number

Before defining these variants we first recall the notion of pairings. Pairings are one of the most powerful tool used in modern cryptography. Roughly speaking, it allows to compute the Diffie Hellman tuple but with fourth element in another group.

Definition 2 (Pairing). *A pairing e is a bilinear map from $\mathbb{G}_1 \times \mathbb{G}_2$ to $\mathbb{G}_T$ (where $\mathbb{G}_i, i \in \{1, 2, T\}$ are groups) and must be:*

- *Bilinear: $e(g_1^a, g_2^b) = g_T^{ab}$ with $g_T = e(g_1, g_2)$ and for all $(g_1, g_2) \in \mathbb{G}_1 \times \mathbb{G}_2$ and $a, b \in \mathbb{Z}$*
- *Non degenerate: the image of e should not be reduce to $\{1_{\mathbb{G}_T}\}$. If $\mathbb{G}_1$ and $\mathbb{G}_2$ are prime order group and g_1, g_2 are generators then it means that: $e(g_1, g_2)$ is a generator of $\mathbb{G}_T$.*
- *Computable: it should exist an polynomial time algorithm able to compute the pairing for any $(g_1, g_2) \in \mathbb{G}_1 \times \mathbb{G}_2$.*

2.1.2 Security Assumption: Pairing Groups and Matrix Diffie-Hellman Assumption.

For simplicity when we will use different groups, we will use implicit representation of group elements as introduced in [EHK⁺13]. For $s \in \{1, 2, T\}$ and $a \in \mathbb{Z}_p$ define $[a]_s = g_s^a \in \mathbb{G}_s$ as the *implicit representation* of a in $\mathbb{G}_s$. Note that from $[a]_s \in \mathbb{G}_s$ it is generally hard to compute the value a (this correspond to solving the discrete logarithm problem in $\mathbb{G}_s$). Moreover, from $[b]_T \in \mathbb{G}_T$ it is hard to compute the value $[b]_1 \in \mathbb{G}_1$ and $[b]_2 \in \mathbb{G}_2$ (pairing inversion problem). Obviously, given $[a]_s \in \mathbb{G}_s$ and a scalar $x \in \mathbb{Z}_p$, one can efficiently compute $[ax]_s \in \mathbb{G}_s$. Let **GGen** be a probabilistic polynomial time (PPT) algorithm that on input 1^κ returns a description $\mathcal{G} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$ of asymmetric pairing groups where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are cyclic groups of order p for a κ -bit prime p , g_1 and g_2 are generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively, and e a pairing. Define $g_T := e(g_1, g_2)$, which is a generator in $\mathbb{G}_T$.

Definition 3 (Bilinear Diffie-Hellman problem (BDH)). *The decisional bilinear Diffie-Hellman problem is to distinguish two distributions:*

$([a], [b]_1, [c]_2, [abc]_T)$ and $([a]_1, [b]_1, [c]_2, h)$ where $\mathcal{G} \leftarrow \mathbf{GGen}(1^\kappa)$ and $h \xleftarrow{\$} \mathbb{G}_T$.

More generally, for a matrix $\mathbf{A} = (a_{ij}) \in \mathbb{Z}_p^{n \times m}$ we define $[\mathbf{A}]_s$ as the implicit representation of $\mathbf{A}$ in $\mathbb{G}_s$:

$$[\mathbf{A}]_s := \begin{pmatrix} g_s^{a_{11}} & \dots & g_s^{a_{1m}} \\ \vdots & \ddots & \vdots \\ g_s^{a_{n1}} & \dots & g_s^{a_{nm}} \end{pmatrix} \in \mathbb{G}_s^{n \times m}$$

For $a, b \in \mathbb{Z}_p^k$, $e([a]_1, [b]_2) := [a^\top b]_T \in \mathbb{G}_T$. We define the matrix Diffie-Hellman (MDDH) assumption [EHK⁺13].

Definition 4 (Matrix Distribution). Let $k \in \mathbb{N}$. We call $\mathcal{D}_k$ a matrix distribution an algorithm that outputs matrices in $\mathbb{Z}_p^{(k+1) \times k}$ of full rank k in polynomial time.

Without loss of generality, we assume the first k rows of $A \xleftarrow{\$} \mathcal{D}_k^3$ form an invertible matrix, we denote this matrix $\overline{A}$, while the last line is denoted $\underline{A}$. The $\mathcal{D}_k$ -Matrix Diffie-Hellman problem is to distinguish the two distributions $([A], [Aw])$ and $([A], [u])$ where $A \xleftarrow{\$} \mathcal{D}_k$, $w \xleftarrow{\$} \mathbb{Z}_p^k$ and $u \xleftarrow{\$} \mathbb{Z}_p^{k+1}$. Formally:

Definition 5 ($\mathcal{D}_k$ -Matrix Diffie-Hellman Assumption $\mathcal{D}_k$ -MDDH). Let $\mathcal{D}_k$ be a matrix distribution and $s \in \{1, 2, T\}$. We say that the $\mathcal{D}_k$ -Matrix Diffie-Hellman ($\mathcal{D}_k$ -MDDH) Assumption holds relative to $\mathbf{GGen}$ in group $\mathbb{G}_s$ if for all PPT adversaries $\mathcal{D}$,

$$\mathbf{Adv}_{\mathcal{D}_k, \mathbf{GGen}}(\mathcal{D}) := |\Pr[\mathcal{D}(\mathcal{G}, [\mathbf{A}]_s, [\mathbf{A}bw]_s) = 1] - \Pr[\mathcal{D}(\mathcal{G}, [\mathbf{A}]_s, [bu]_s) = 1]| = \text{negl}(\lambda),$$

where the probability is taken over $\mathcal{G} \xleftarrow{\$} \mathbf{GGen}(1^\lambda)$, $\mathbf{A} \xleftarrow{\$} \mathcal{D}_k$, $bw \xleftarrow{\$} \mathbb{Z}_p^k$, $bu \xleftarrow{\$} \mathbb{Z}_p^{k+1}$.

For each $k \geq 1$, [EHK⁺13] specifies distributions $\mathcal{L}_k, \mathcal{U}_k, \dots$ such that the corresponding $\mathcal{D}_k$ -MDDH assumption is the k -Linear assumption, the k -uniform and others. All assumptions are generically secure in bilinear groups and form a hierarchy of increasingly weaker assumptions. For example the distributions are for $k = 2$, where $a_1, \dots, a_6 \xleftarrow{\$} \mathbb{Z}_p$.

$$\mathcal{L}_2 : \mathbf{A} = \begin{pmatrix} a_1 & 0 \\ 0 & a_2 \\ 1 & 1 \end{pmatrix} \quad \mathcal{U}_2 : \mathbf{A} = \begin{pmatrix} a_1 & a_2 \\ a_3 & a_4 \\ a_5 & a_6 \end{pmatrix}.$$

It was also shown in [EHK⁺13] that $\mathcal{U}_k$ -MDDH is implied by all other $\mathcal{D}_k$ -MDDH assumptions.

Lemma 1 (Random self reducibility [EHK⁺13]). For any matrix distribution $\mathcal{D}_k$, $\mathcal{D}_k$ -MDDH is random self-reducible. In particular, for any $m \geq 1$ and for all PPT adversaries $\mathcal{D}$ and $\mathcal{D}'$,

$$\mathbf{Adv}_{\mathcal{D}_k, \mathbf{GGen}}(\mathcal{D}) + \frac{1}{q-1} \geq \mathbf{Adv}_{\mathcal{D}_k, \mathbf{GGen}}^m(\mathcal{D}')$$

where $\mathbf{Adv}_{\mathcal{D}_k, \mathbf{GGen}}^m(\mathcal{D}') := \Pr[\mathcal{D}'(\mathcal{G}, [\mathbf{A}], [\mathbf{A}\mathbf{W}]) \Rightarrow 1] - \Pr[\mathcal{D}'(\mathcal{G}, [\mathbf{A}], [\mathbf{U}]) \Rightarrow 1]$, with $\mathcal{G} \leftarrow \mathbf{GGen}(1^\lambda)$, $\mathbf{A} \xleftarrow{\$} \mathcal{D}_k$, $\mathbf{W} \xleftarrow{\$} \mathbb{Z}_p^{k \times m}$, $\mathbf{U} \xleftarrow{\$} \mathbb{Z}_p^{(k+1) \times m}$.

Remark: It should be noted that $\mathcal{L}_1, \mathcal{L}_2$ are respectively named semi-external Diffie Hellman⁴ (SXDH) and Decision Linear (DLin) [EHK⁺13] assumptions. $\mathcal{L}_1$ is generated using a scalar as matrix.

³the notation $\xleftarrow{\$}$ means that the left element is chosen randomly in the set at the right of the arrow

⁴in other terms this assumption state that DH holds in both groups $\mathbb{G}_1$ and $\mathbb{G}_2$

2.1.3 Security Models

When rigorously defining security of protocols we have different ways to link the security and the hardness assumption. We will here describe the most common ways.

Random Oracle Model In the random oracle model (ROM), we assume that a public accessible algorithm (often a hash function) can be modelled as a random oracle. It means that any adversary cannot distinguish the output of the algorithm from a distribution uniformly chosen at random element of the output domain. In [CCG⁺94] they explained that there exists no true random oracle. Moreover when instantiated some signature or encryption scheme which were secure in the ROM, it leads to truly insecure scheme. The ROM can give a reasonable level of security. However the high level research tend to avoid it.

Standard Model In opposition to random oracle model, the desirable model of security is the standard model. In the standard model the security is only based on the complexity assumptions. It means that the adversary is only limited by the time and computational power available.

Universally Composable (UC) Framework. The goal of the UC framework [Can01] is to ensure that UC-secure protocols will continue to behave in the ideal way even if executed in a concurrent way in arbitrary environments. It is a simulation-based model, relying on the indistinguishability between the real world and the ideal world. In the ideal world, the security is provided by an ideal functionality $\mathcal{F}$, capturing all the properties required for the protocol and all the means of the adversary. In order to prove that a protocol Π emulates $\mathcal{F}$, one has to construct, for any polynomial time adversary $\mathcal{A}$ (which controls the communication between the players), a simulator $\mathcal{S}$ such that no polynomial time environment $\mathcal{Z}$ can distinguish between the real world (with the real players interacting with themselves and $\mathcal{A}$ and executing the protocol π) and the ideal world (with dummy players interacting with $\mathcal{S}$ and $\mathcal{F}$) with a significant advantage. The adversary can be either *adaptive*, *i.e.* allowed to corrupt users whenever it likes to, or *static*, *i.e.* required to choose which users to corrupt prior to the execution of the session sid of the protocol. After corrupting a player, $\mathcal{A}$ has complete access to the internal state and private values of the player, takes its entire control, and plays on its behalf.

Simple UC Framework. Canetti, Cohen and Lindell formalized a simpler variant in [CCL15], that we use here. This simplifies the description of the functionalities for the following reasons (in a nutshell): All channels are automatically assumed to be authenticated; There is no need for *public delayed outputs* (waiting for the adversary before delivering a message to a party), neither for an explicit description of the corruptions.

Advantage A common notion when discussing about security games and adversaries playing these games is the advantage. This is a simple notion that we

describe in the following definition.

Definition 6. *The advantage of an adversary $\mathcal{A}$ in winning the experiment Exp is the probability that $\mathcal{A}$ wins the experiment minus $1/2$.*

2.2 Parameters and Efficiency

When there is an efficiency comparison between different schemes/protocols we often compare them in terms of communication cost. This communication cost is expressed in number of group elements.

We have to be careful with this approach, indeed the structure/properties of the groups involved change a group element's size of storage. The first thing to take into account is the cardinal of the group, there is a ratio superior to 12 between the size of an element in a prime order group and an element in a composite order group (according to the National Institute of Standards and Technology [Gui13]).

Computational time is also a very important parameter to describe the efficiency. Once again, the type of group used imply different time performance: for the same 126 bit security, a pairing computation takes 254 more time in a composite order group than in a prime order group and exponentiation is more than 300 time slower. This is quite problematic because pairing computations already take a lot of time. All these information can be found in [EMJ17].

The size of group elements depends on the security we want but also on the security reduction from the hard assumption we have. Thus the more direct is the reduction the less group elements' size increases.

2.3 Basis Of Cryptography: Encryption

We will formally define what is an encryption⁵ scheme and what kind of security requirement it must follow.

Definition: an encryption scheme $\mathcal{E}$ is described through four PPT algorithms (Setup, KeyGen, Encrypt, Decrypt):

- **Setup**($1^{\mathfrak{K}}$), where $\mathfrak{K}$ is the security parameter, generates the global parameters **param** of the scheme;
- **KeyGen**(**param**) outputs a pair of keys, a (public) encryption key **ek** and a (private) decryption key **dk**;
- **Encrypt**(**ek**, M ; ρ) outputs a ciphertext $\mathcal{C}$, on a plaintext M , under the encryption key **ek**, using the randomness ρ ;
- **Decrypt**(**dk**, $\mathcal{C}$) outputs M , encrypted in the ciphertext $\mathcal{C}$ or $\perp$.

⁵Here we obviously define a public key encryption scheme

Procedure Initialize: $\text{param} \leftarrow \text{Setup}(1^{\mathcal{K}})$ $(\text{ek}, \text{dk}) \leftarrow \text{KeyGen}(\text{param})$ Return ek Procedure Decrypt($\mathcal{C}$): Return $\text{Decrypt}(\text{dk}, \mathcal{C})$ $\mathcal{CT} \leftarrow \mathcal{C}$	Procedure Encrypt(ek, M_b): $(\mathcal{C}^*) \leftarrow \text{Encrypt}(\text{ek}, M_b)$ Return $\mathcal{C}^*$ Procedure Finalize(β): If $\mathcal{C}^* \in \mathcal{CT}$ return 0 Else return $b == \beta$
---	--

FIGURE 2.1: IND-CCA-SECURITY GAME

Security: such encryption scheme is required to have the following security properties:

- *Correctness:* For every pair of keys (ek, dk) generated by KeyGen , every messages M , and every random ρ , we should have $\text{Decrypt}(\text{dk}, \text{Encrypt}(\text{ek}, M; \rho)) = M$.
- *Indistinguishability under Adaptive Chosen Ciphertext Attack* IND-CCA [NY90, RS92]:

We say that the encryption scheme is IND-CCA secure if the advantage⁶ of an adversary in winning IND-CCA-security game Figure 2.1 is negligible. Description of the game in Figure 2.1: a challenger $\mathcal{C}$ will create a game for the adversary $\mathcal{A}$.

- **Procedure Initialize:** $\mathcal{C}$ will honestly compute the parameters and the pair of keys and return ek to $\mathcal{A}$.
- **Procedure Decrypt($\mathcal{C}$):** $\mathcal{A}$ can request at any time decryption of ciphertexts, the ciphertext decrypted are stored (if the adversary has not access to the procedure Decrypt the security is called IND-CPA (Indistinguishability under Chosen Plaintext Ciphertext Attack)).
- **Procedure Encrypt(ek, M_b):** $\mathcal{A}$ will submit two messages M_0 and M_1 , in game $b \in \{0, 1\}$ the challenger encrypt the message M_b and return the ciphertext to $\mathcal{A}$.
- **Procedure Finalize(β):** $\mathcal{A}$ submit a bit β and win if it never decrypt the ciphertext $\mathcal{C}^*$ and if M_β is encrypted in $\mathcal{C}^*$.

Examples: One of the most simple example is the ElGamal encryption [ElG84] which is IND-CPA secure under DDH assumption. It is described in Figure 2.2.

The linear encryption described in Figure 2.3 has been introduced by Boneh Boyen and Shacham in [BBS04] is IND-CPA-secure under DLin.

In the following of the thesis we will need another encryption called the Cramer-Shoup encryption scheme [CS98] which is an IND-CCA version of the ElGamal Encryption. We present it here as a labeled public-key encryption scheme in

⁶see definition 6

- **Setup**($1^{\mathfrak{K}}$): Generates a group $\mathbb{G}$ of prime order p , with a generator g , $\text{param} = (\mathbb{G}, p, g)$.
- **KeyGen**(param): Picks $h \xleftarrow{\$} \mathbb{G}$, $\text{dk} = x \xleftarrow{\$} \mathbb{Z}_p$, and sets $\text{ek} = g^x$.
- **Encrypt**($\text{ek}, M \in \mathbb{G}; r$): Outputs $\mathcal{C} = (M \cdot \text{ek}^r, g^r)$
- **Decrypt**(dk, C): Outputs $\mathcal{C}_1 / (\mathcal{C}_2^{\text{dk}})$

FIGURE 2.2: ELGAMAL ENCRYPTION

- **Setup**($1^{\mathfrak{K}}$): generates a group $\mathbb{G}$ of prime order p , with a generator g , $\text{param} = (\mathbb{G}, p, g)$.
- **KeyGen**(param): $\text{dk} = (y_1, y_3) \xleftarrow{\$} \mathbb{Z}_p^2$,
 $\text{ek} = (\text{ek}_1, \text{ek}_2, \text{ek}_3) := (g^{y_1}, g, g^{y_3})$
- **Encrypt**(ek, M): $(r_1, r_2) \xleftarrow{\$} \mathbb{Z}_p^2$,
 $\mathcal{C} := (\text{ek}_1^{r_1}, \text{ek}_2^{r_2}, \text{ek}_3^{r_1+r_2} \cdot M)$
- **Decrypt**($\text{dk}, \mathcal{C}$): outputs $\mathcal{C}_3 / (\mathcal{C}_1^{y_3/y_1} \mathcal{C}_2^{y_3})$

FIGURE 2.3: LINEAR ENCRYPTION

Figure 2.4, the classical version is done with the label space set to $\emptyset^7$. The security of the scheme is proven under the DDH assumption and the hash function used is a Universal One-Way Hash Function. A generalization of this encryption to the k – MDDH assumption can be found in figure 2.5.

⁷A labeled public-key encryption scheme is a set of public-key encryption schemes. The choose of the label make the encryption scheme unique. Two labels gives two different encryption schemes

- **Setup**($1^{\mathfrak{K}}$): generates a group $\mathbb{G}$ of order p , $\text{param} = (\mathbb{G}, p)$.
- **KeyGen**(param): generates $(g_1, g_2) \xleftarrow{\$} \mathbb{G}^2$, $\text{dk} = (x_1, x_2, y_1, y_2, z) \xleftarrow{\$} \mathbb{Z}_p^5$, and sets, $c = g_1^{x_1} g_2^{x_2}$, $d = g_1^{y_1} g_2^{y_2}$, and $h = g_1^z$. Chooses a Universal One-Way hash function $\mathfrak{H}_K$ in a hash family $\mathcal{H}$ (see def 14) $\text{ek} = (g_1, g_2, c, d, h, \mathfrak{H}_K)$.
- **Encrypt**($\ell, \text{ek}, M \in \mathbb{G}; r$): Outputs $\mathcal{C} = (\ell, \mathbf{u} = (g_1^r, g_2^r), e = M \cdot h^r, v = (cd^\xi)^r)$, where v is computed afterwards with $\xi = \mathfrak{H}_K(\ell, \mathbf{u}, e)$.
- **Decrypt**($\ell, \text{dk}, \mathcal{C}$): Computes $\xi = \mathfrak{H}_K(\ell, \mathbf{u}, e)$ and checks whether $u_1^{x_1+\xi y_1} \cdot u_2^{x_2+\xi y_2} \stackrel{?}{=} v$. If True computes $M = e / (u_1^z)$ and outputs M . Otherwise, outputs $\perp$.

FIGURE 2.4: CRAMER SHOUP ENCRYPTION

- **Setup**($1^{\mathfrak{K}}$): generates groups $\mathbb{G}_1$ of order p with generator g_1 , **param** = $(\mathbb{G}, p)$.
- **KeyGen**($\mathfrak{K}$): Picks $E \xleftarrow{\$} \mathcal{D}_k, u, v \xleftarrow{\$} \mathbb{Z}_p^{k+1}$, sets **ek** = $([E]_1, [u^\top E]_1, [v^\top E]_1)$, **dk** = $(\underline{E} \cdot \overline{E^{-1}}, u, v)$ to be the public encryption key, where $\mathfrak{H}_K$ is a random collision-resistant hash function from $\mathcal{H}$ ^a.
- **Encrypt**(**ek**, $M \in \mathbb{G}_1, \ell; \mu$) If $M = g_1^m$, $\mathcal{C} = (e = \text{ek}_1 \mu + \begin{bmatrix} 0 \\ m \end{bmatrix}_1, w = [(\text{ek}_2 + \theta \text{pk}_3) \mu]_1)$. where $\theta = \mathfrak{H}_K(\ell, e)$.
- **Decrypt**($\ell, \text{dk}, \mathcal{C}$) If $[w]_1 \stackrel{?}{=} [(u^\top + \theta v^\top) \bar{e}]_1$, then outputs $M = \text{sk} \bar{e} + \underline{e}$. Otherwise output $\perp$.

^aLike Cramer-Shoup one could rely on an universal one-way hash function family instead see def 14

FIGURE 2.5: CRAMER SHOUP ENCRYPTION UNDER $k - \text{MDDH}$

2.4 Digital Signatures

2.4.1 Definition and Security

A digital signature scheme $\mathcal{S}$ [DH76, GMR88] allows a signer to produce a verifiable proof that proves the authenticity of a message. It is described through four algorithms:

Definition 7 (Digital Signature Scheme). $\sigma = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$:

- **Setup**($1^{\mathfrak{K}}$) where $\mathfrak{K}$ is the security parameter, generates the global parameters **param** of the scheme, for example the message space;
- **KeyGen**(**param**), outputs a pair of (**sk**, **vk**), where **sk** is the (secret) signing key, and **vk** is the (public) verification key;
- **Sign**(**sk**, $M; \mu$), outputs a signature σ , on a message M , under the signing key **sk**, and some randomness μ ;
- **Verify**(**vk**, M, σ) checks the validity of the signature σ with respect to the message M and the verification key **vk**. And so outputs a bit.

In the following we will expect at least two properties for signatures:

- **Correctness**: For every pair (**vk**, **sk**) generated by **KeyGen**, for every message M , and for all randomness μ , we have $\text{Verify}(\text{vk}, M, \text{Sign}(\text{sk}, M; \mu)) = 1$.

- *Existential Unforgeability under Chosen Message Attacks [GMR88] (EUF – CMA).* We say that the signature scheme is EUF – CMA secure if the advantage of an adversary in winning $\text{Exp}_{S,A}^{\text{euf}}$ is negligible. The oracle OSign sign a message and store any message queried in $\mathcal{SM}$.

$\text{Exp}_{S,A}^{\text{euf}}(\mathcal{R})$

1. $\text{param} \leftarrow \text{Setup}(1^{\mathcal{R}})$
2. $(\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(\text{param})$
3. $(m^*, \sigma^*) \leftarrow \mathcal{A}(\text{vk}, \text{OSign}(\text{sk}, \cdot))$
4. $b \leftarrow \text{Verify}(\text{vk}, m^*, \sigma^*)$
5. If $m^* \in \mathcal{SM}$ or $b \neq 1$ output 0
6. Else output 1

The probability of success against this game is denoted by

$$\text{Succ}_{S,A}^{\text{euf}}(\mathcal{R}) = \Pr[\text{Exp}_{S,A}^{\text{euf}}(\mathcal{R}) = 1], \quad \text{Succ}_S^{\text{euf}}(\mathcal{R}, t) = \max_{A \leq t} \text{Succ}_{S,A}^{\text{euf}}(\mathcal{R}).$$

There is an interesting primitive close to signature but in a symmetrical cryptography setup, this is the message authentication code (MAC)⁸.

Definition 8 (Message Authentication Code). *A MAC is described through three algorithms ($\text{Gen}_{\text{MAC}}, \text{Tag}, \text{Ver}$):*

- $\text{Gen}_{\text{MAC}}(\text{param})$: *Returns sk_{MAC} .*
- $\text{Tag}(\text{sk}_{\text{MAC}}, m)$: *Returns a tag τ on a message m from the message space $\mathcal{M}$.*
- $\text{Ver}(\text{sk}_{\text{MAC}}, m, \tau)$: *Returns a verification bit b .*

The MAC is correct if for any correctly computed tag τ the verification bit outputted by Ver is equal to 1. For the security we define is unforgeability under Chosen Message Attacks as for signature schemes.

2.5 Identity-based Encryption

The most studied Identity-based primitive is by far the Identity-based Encryption (IBE). In this part we will describe formally what is an IBE and what kind of security is required for this type of scheme. Further we will see how the building of such schemes has changed through time, then we will see a very recent example which will be a basis of our work. Finally we will talk about some features that will be very important to use IBE in unexpected ways.

2.5.1 Definition and Security

Identity-based encryption was first introduced by Shamir in [Sha84] who was expecting an encryption scheme where no public key will be needed for sending a message to a precise user, defined by his identity. Thus any user wanting to send a private message to a user only needs this user's identity and a master public key. It took 17 years for the cryptographic community to find a way to

⁸We will use them in the proof of security of the DIBKEM in section 3.1

Procedure Initialize: $(\text{mpk}, \text{msk}) \xleftarrow{\\$} \text{Setup}(\mathfrak{K})$ Return mpk Procedure USKGen(id): $\mathcal{Q}_{\text{ID}} \leftarrow \mathcal{Q}_{\text{ID}} \cup \{\text{id}\}$ Return $\text{usk}[\text{id}] \xleftarrow{\\$} \text{USKGen}(\text{msk}, \text{id})$	Procedure Enc(id*): // one query $(K^*, C^*) \xleftarrow{\\$} \text{Enc}(\text{mpk}, \text{id}^*)$ $K^* \xleftarrow{\\$} \mathcal{K}; C^* \xleftarrow{\\$} \text{CS}$ Return (K^*, C^*) Procedure Finalize(β): Return $(\text{id}^* \notin \mathcal{Q}_{\text{ID}}) \wedge \beta$
---	---

FIGURE 2.6: PR-ID-CPA-SECURITY: SECURITY GAMES $\text{PR-ID-CPA}_{\text{real}}$ AND $\text{PR-ID-CPA}_{\text{rand}}$ (BOXED).

realize this idea. The first instantiation was proposed in [BF01] by Boneh and Franklin.

In this thesis we will consider the slightly different concept of identity-based key encapsulation mechanism (IBKEM). In an IBKEM the sender does not choose a message to encrypt. The encryption algorithm will output a key and a ciphertext. The recipient will recover the key by decrypting the ciphertext.

Formally an IBKEM scheme IBKEM consists of four algorithms $\text{IBKEM} = (\text{Setup}, \text{USKGen}, \text{Enc}, \text{Dec})$. Every IBKEM can be transformed into an ID-based encryption scheme IBE using a (one-time secure) symmetric cipher.

Definition 9 (Identity-based Key Encapsulation Scheme). *An identity-based key encapsulation scheme IBKEM consists of four PPT algorithms $\text{IBKEM} = (\text{Setup}, \text{USKGen}, \text{Enc}, \text{Dec})$ with the following properties.*

- $\text{Setup}(1^{\mathfrak{K}})$: from the security parameter $\mathfrak{K}$, returns the (master) public/secret key (mpk, msk) . We assume that mpk implicitly defines an identity space ID , a key space $\mathcal{K}$, and ciphertext space CS .
- $\text{USKGen}(\text{msk}, \text{id})$: returns the user secret-key $\text{usk}[\text{id}]$ for identity $\text{id} \in \text{ID}$.
- $\text{Enc}(\text{mpk}, \text{id})$: returns a symmetric key $K \in \mathcal{K}$ together with a ciphertext $C \in \text{CS}$ with respect to identity id .
- $\text{Dec}(\text{usk}[\text{id}], \text{id}, C)$: returns the decapsulated key $K \in \mathcal{K}$ or the reject symbol $\perp$.

For perfect correctness we require that for all $\mathfrak{K} \in \mathbb{N}$, all pairs (mpk, msk) generated by $\text{Setup}(\mathfrak{K})$, all identities $\text{id} \in \text{ID}$, all $\text{usk}[\text{id}]$ generated by $\text{USKGen}(\text{msk}, \text{id})$ and all (K, C) output by $\text{Enc}(\text{mpk}, \text{id})$: $\Pr[\text{Dec}(\text{usk}[\text{id}], \text{id}, C) = K] = 1$.

The security requirements for an IBKEM we consider here are indistinguishability and anonymity against chosen plaintext and identity attacks we define pseudorandom ciphertexts against chosen plaintext and identity attacks (PR-ID-CPA) which means that challenge key and ciphertext are both pseudorandom. We define PR-ID-CPA -security of IBKEM formally via the games given in Figure 2.6.

Definition 10 (PR-ID-CPA Security). *An ID-based key encapsulation scheme IBKEM is PR-ID-CPA-secure if for all PPT $\mathcal{A}$, the following advantage is negligible:*

$$\text{Adv}_{\text{IBKEM}}^{\text{pr-id-cpa}}(\mathcal{A}) := |\Pr[\text{PR-ID-CPA}_{\text{real}}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{PR-ID-CPA}_{\text{rand}}^{\mathcal{A}} \Rightarrow 1]|.$$

There are many different definitions of security for IBE each one corresponding to different power given to the adversary. Of course the more power we give to the adversary in the security definition the more secure is our scheme.

An interesting definition of security, is the selective-ID security definition. The only difference with the one from Figure 2.6 is that the challenge identity has to be output by the adversary before the procedure $\text{USKGen}(\text{id})$. This is a weaker security definition. It may look like a detail but in fact there are a lot of problems when moving from selective-ID security to full security. For example in [BB04] the authors state that a selectively secure IBE is fully secure as soon as we can have a collision resistant hash function⁹ from the identity space to bits string of size $d \in \mathbb{Z}$. But the reduction is not tight and introduces a factor loss 2^d . As well as in the case of the scheme [BBG05] using the techniques of Waters [Wat05], the authors have to add a collision resistant hash function and they lose an exponential factor in the security parameters (here 2^{nd} where n is the hierarchy depth).

Remark 1. *For simplicity when talking about IBKEM and IBE we will often only refer to IBE. This is not such a problem since it is easy to have one from the other:*

- *IBE to IBKEM: choose $K \xleftarrow{\$} \mathcal{K}$ and set the output of $\text{Enc}_{\text{IBKEM}}(\text{mpk}, \text{id})$ to $(C, K) = (\text{Enc}_{\text{IBE}}(\text{mpk}, \text{id}, K), K)$.*
- *IBKEM to IBE: set the output of $\text{Enc}_{\text{IBE}}(\text{mpk}, \text{id}, M)$ to $(M \oplus K, C)$ where $(C, K) = \text{Enc}_{\text{IBKEM}}(\text{mpk}, \text{id})$. Also set the output of Dec_{IBE} to $(M \oplus K) \oplus \text{Dec}_{\text{IBKEM}}(\text{usk}[\text{id}], \text{id}, C)$.*

The other algorithms stay the same.

2.5.2 IBE's State of The Art

In this section we will describe some IBE schemes and see what has changed during these years of research on IBE. We will describe two historical schemes and talk about the main other ones without describing them completely. As said before the first IBE scheme was the one from Boneh and Franklin in [BF01]. Their scheme is described in Figure 2.7

This IBE is secure in the random oracle model, both hash functions are modelled as random oracles. As you can see this work is based on pairings. It has to be

⁹see definition 14

<u>Setup($\mathfrak{K}$)</u> $(p, \mathbb{G}, \mathbb{G}_T, e, g) \xleftarrow{\$} \text{GGen}_s(1^\mathfrak{K})$ Choose H (resp. H_T) two hash functions from $\{0, 1\}^*$ to $\mathbb{G}$ (resp. $\mathcal{M}$) $s \xleftarrow{\$} \mathbb{Z}_p^*, h \leftarrow g^s$ Return $(\text{mpk} = (\mathbb{G}, \mathbb{G}_T, H, H_T, e, h),$ $\text{msk} = s)$	<u>Enc($\text{mpk}, \text{id}, M \in \mathcal{M}$)</u> Return $C = (g^r, M \oplus H_T(e(H(\text{id}), h)^r))$
<u>USKGen(msk, id)</u> Return $\text{usk}[\text{id}] = H(\text{id})^s$	<u>Dec($\text{usk}[\text{id}], \text{id}, C$)</u> Set C as (C_1, C_2) Return $M = C_2 \oplus H_T(e(\text{usk}[\text{id}], C_1))$

FIGURE 2.7: BONEH FRANKLIN IBE

noticed that in the same time Cocks¹⁰ in [Coc01] created an IBE based on quadratic residue.

The first IBE scheme secure in the standard model was the one from Boneh and Boyen [BB04]. It is secure under the bilinear Diffie-Hellman problem. We describe it in Figure 2.8.

<u>Setup($\mathfrak{K}$)</u> $(p, \mathbb{G}, \mathbb{G}_T, e, g_1) \xleftarrow{\$} \text{GGen}_s(1^\mathfrak{K})$ $g_2 \xleftarrow{\$} \mathbb{G}, U = \{u_{i,j}\}_{i \in \{1,n\}, j \in \{0,1\}} \xleftarrow{\$} \mathbb{G}^{2n}$ Choose $s \xleftarrow{\$} \mathbb{Z}_p^*, h \leftarrow g_1^s$ and k a hash function key in the appropriate key space Return $(\text{mpk} = (\mathbb{G}, \mathbb{G}_T, e, U, k, g_1, h, g_2), \text{msk} =$ $g_2^s)$	<u>Enc($\text{mpk}, \text{id}, M \in \mathcal{M}$)</u> Compute $a = (a_1, \dots, a_n) = \mathfrak{H}_K(\text{id}) \in$ $\{0, 1\}^n$ Choose $r \xleftarrow{\$} \mathbb{Z}_p$ Return $C = (e(h, g_2)^r \cdot M, g^r, u_{1,a_1}^r, \dots, u_{n,a_n}^r)$
<u>USKGen(msk, id)</u> Compute $a = (a_1, \dots, a_n) = \mathfrak{H}_K(\text{id}) \in$ $\{0, 1\}^n$ Choose $(t_1, \dots, t_n) \in \mathbb{Z}_p^n$ Return $\text{usk}[\text{id}] = (g_2^s \cdot \prod_{i=1}^n u_{i,a_i}^{t_i}, g_1^{t_1}, \dots, g_1^{t_n})$	<u>Dec($\text{usk}[\text{id}], \text{id}, C$)</u> Set C as $(A, B, C_1, \dots, C_n)$ And $\text{usk}[\text{id}]$ as $(\text{usk}_0, \dots, \text{usk}_n)$ Return $M = A \cdot \frac{\prod_{j=1}^n e(C_j, \text{usk}_j)}{e(B, \text{usk}_0)}$

FIGURE 2.8: BONEH BOYEN IBE

This IBE is secure in the standard model and under a classical assumption (BDH). However this scheme is unpractical because of the size of the ciphertext: it is logarithmic in the number of identities. The first practical scheme by

¹⁰Once again Clifford Cocks could not share his work because it was classified by the British government

Waters in [Wat05] brings a new construction which is efficient and fully secure in the standard model under a classical assumption. The only drawback was the size of the public key which was linear in the security parameter. In [Wat09] Waters fixes this problem with new techniques in the proof called Dual system encryption. More than bringing a new scheme of IBE the dual system encryption will allow a lot of new works in IBE and Attribute-based Encryption.

2.5.3 Instantiation From [BKP14]

Through this thesis we needed an efficient IBE scheme with particular properties (in particular without hash functions). The natural choice was to use the one from [BKP14]. It has many advantages such as the tightness security¹¹ to a classical assumption and the fact that the group order is prime. This scheme is following the steps of the one from [Wat09] and use Dual system encryption techniques to prove its security. Note that it is actually an IBKEM scheme.

<u>Gen($\mathcal{R}$):</u> $A \xleftarrow{\\$} \mathbb{Z}_p^{2 \times 1}$ For $i = 1, \dots, 2\ell$: $Y_i \xleftarrow{\\$} \mathbb{Z}_p^2; Z_i = Y_i^\top \cdot A \in \mathbb{Z}_p$ $y'_0 \xleftarrow{\\$} \mathbb{Z}_p^2; z'_0 = y'_0{}^\top \cdot A \in \mathbb{Z}_p$ $\text{mpk} := (\mathcal{G}, [A]_1, ([Z_i]_1)_{0 \leq i \leq \ell}, [z'_0]_1)$ $\text{msk} := (B, (Y_i)_{0 \leq i \leq \ell}, y'_0)$ Return (mpk, msk) <u>USKGen($\text{msk}, \text{id} \in \text{ID}$):</u> $t \xleftarrow{\\$} \mathbb{Z}_p$ $v = \sum_{i=1}^{\ell} \text{id}_i Y_i t + y'_0 \in \mathbb{Z}_p^2$ For $i \in \{0, \dots, \ell - 1\}$: $\text{usk}[\text{id}] := ([t]_2, [v]_2) \in \mathbb{G}_2 \times \mathbb{G}_2^2$ Return ($\text{usk}[\text{id}]$)	<u>Enc(mpk, id):</u> $r \xleftarrow{\\$} \mathbb{Z}_p; c_0 = Ar \in \mathbb{Z}_p^2$ $c_1 = (\sum_{i=1}^{2\ell} f_i(\text{id}_1) Z_i) \cdot r \in \mathbb{Z}_p$ $K = z'_0 \cdot r \in \mathbb{Z}_p$ Return $\text{K} = [K]_T$ and $\text{C} = ([c_0]_1, [c_1]_1)$ <u>Dec($\text{usk}[\text{id}], \text{id}, \text{C}$):</u> Parse $\text{usk}[\text{id}] = ([t]_2, [v]_2)$ Parse $\text{C} = ([c_0]_1, [c_1]_1)$ $\text{K} = e([c_0]_1, [v]_2) \cdot e([c_1]_1, [t]_2)^{-1}$ Return $\text{K} \in \mathbb{G}_T$
---	---

FIGURE 2.9: TIGHT IBKEM UNDER SXDH FROM [BKP14]

Today there is still no practical IBE without pairing. There are some attempts, for example an IBE using garbled circuit [DG17], learning with error problem [GPV08, CHKP10, ABB10] or code-based cryptography [GHPT17] but nothing as practical as pairings. Still it could be very interesting to construct efficient IBE without pairing because pairing computations take time and need an important amount of power of computing. For example in light weight cryptography the use of pairing computations is avoided.

¹¹This means that the reduction lost is minimal, to achieve the same level of security we will be able to have shorter parameters

- $\text{Wat.KeyGen}(\text{param})$: $(\text{pk}_1, \text{sk}_1) = (g^z, h^z)$, $z \xleftarrow{\$} \mathbb{Z}_p$
- $\text{Wat.Sign}(\text{sk}_1, \mathcal{M})$: chooses $s \in \mathbb{Z}_p$ and computes $\sigma := (\text{sk}_1 \mathcal{F}(\mathcal{M})^s, g^s)$. Please note that σ is composed of two parts (σ_1, σ_2) .
- $\text{Wat.Verify}(\text{pk}_1, \mathcal{M}, \sigma)$: checks $e(g, \sigma_1) = e(\sigma_2, \mathcal{F}(\mathcal{M})) \cdot e(\text{pk}_1, h)$, $\mathcal{F}(\mathcal{M}) := u_0 \prod_{i \in [1, k]} u_i^{\mathcal{M}_i}$, with $\mathcal{M} = (\mathcal{M}_1, \dots, \mathcal{M}_k)$ and $\mathcal{M}_i \in \{0, 1\}$

FIGURE 2.10: WATERS SIGNATURE

2.5.4 Signature from Identity-based Encryption

As described in [BF01] it is possible to produce a signature scheme from an IBE one. They proved that the security of the signature relies on the security of the IBE scheme involved. The idea is quite simple: the signer will play the role of the authority of the IBE. Thus the public key of the signature scheme will be all public parameters of the IBE. To sign a message M the signer will produce a private key for the identity: M . The verification is easily done by testing whether or not this element is the private key of M (by encrypting a dummy message and decrypting it).

An interesting example of this kind of signature is the one from Waters in [Wat05] coming from the IBE of the same paper. It is secure under the decisional bilinear Diffie-Hellman problem. For the Waters function $\mathcal{F}$, we assume the existence of independent group generators u_i that define the function $\mathcal{F}(\mathcal{M}) := u_0 \prod_{i \in [1, k]} u_i^{\mathcal{M}_i}$, with $\mathcal{M} = (\mathcal{M}_1, \dots, \mathcal{M}_k)$ and $\mathcal{M}_i \in \{0, 1\}$.

This signature verification makes use of pairing computations. Thus this scheme is not very efficient, but since the verification is made with pairing computations it can be used along other schemes using pairings. This will allow implicit verifications (see Oblivious Signature-based Envelope Section 5.1 or Figure 2.13).

2.5.5 Interesting Features and Properties

Hierarchical Identity-based Encryption: Identities can have a special form, this allow to create some kind of hierarchy between them. That is enough to justify the construction of hierarchical IBE (HIBE) which were introduced in [GS02]. We often represent HIBE with a binary tree, each user represents a node in this tree. In these primitives a user possesses a secret key and is able to construct/delegate a new key for every node/identities under him. Once again HIBE can be used as an actual system of encryption with hierarchy or sometimes it can be used in different ways (see Section 3). We recall a formal definition of HIBE in the term of HIBKEM.

Definition 11. An *HIBKEM* HIBKEM is described through five algorithm (Gen , USKGen , USKDel , Enc , Dec).

- $\text{Gen}(\mathfrak{K})$ returns the (master) public/secret key and delegation key $(\text{pk}, \text{sk}, \text{dk})$.

- $\text{USKGen}(\text{sk}, \text{id})$ returns a secret key $\text{usk}[\text{id}]$ and a delegation value $\text{udk}[\text{id}]$.
- $\text{USKDel}(\text{dk}, \text{usk}[\text{id}], \text{udk}[\text{id}], \text{id}, \text{id}')$ returns a user secret key $\text{usk}[\text{id}|\text{id}']$.
- $\text{Enc}(\text{pk}, \text{id})$ returns a symmetric key K together with a ciphertext C .
- $\text{Dec}(\text{usk}[\text{id}], \text{id}, C)$ returns the decapsulated key K or the reject symbol $\perp$.

In our HIBKEM definition we make the delegation key dk and the user delegation key $\text{udk}[\text{id}]$ explicit to make our constructions more readable. It will help us for redefining the PR-CMA security we are expecting from an HIBKEM. Indeed to prevent trivial win from the adversary in this game we will not allow a win from the adversary if he requested a user secret key which is a prefix of the challenge identity.

The HIBE from [BKP14] will be our favourite instantiation to use HIBE as a tool. Indeed it is a practical scheme with tight security reduction under a classical assumption namely k -MDDH. We describe this scheme in Figure 2.11. Of course these scheme comes from the IBE from the same paper.

Blind IBE A blind IBE is an IBE where the user secret key can be queried without the authority knowing which identity is related to the key. This might seems awkward to construct such primitive and hard to find applications to this, but it has been created in a precise purpose that we will explain in section 4.3.

We continue to follow the KEM formalism by adapting the definition of a Blind IBE scheme given in [GH07] to this setting.

Definition 12 (Blind Identity-Based Key Encapsulation Scheme). *A Blind Identity-Based Key Encapsulation scheme BlindIBKEM consists of four PPT algorithms*

(Setup, BlindUSKGen, Enc, Dec) with the following properties:

- Setup, Enc and Dec are defined as for a traditional IBKEM scheme.
- $\text{BlindUSKGen}(\langle (\mathcal{S}, \text{msk})(U, \text{id}, \ell; \rho) \rangle)$ is an interactive protocol, in which an honest user U with identity $\text{id} \in \text{ID}$ obtains the corresponding user secret key $\text{usk}[\text{id}]$ from the master authority $\mathcal{S}$ or outputs an error message, while $\mathcal{S}$'s output is nothing or an error message (ℓ is a label and ρ the randomness).

Defining the security of a BlindIBKEM requires two additional properties, stated as follows (see [GH07, pages 6 and 7] for the formal security games):

1. **Leak-free Secret Key Generation** (called Leak-free Extract for Blind IBE security in the original paper): A potentially malicious user cannot learn anything by executing the BlindUSKGen protocol with an honest authority which he could not have learned by executing the USKGen protocol with an honest authority. Moreover, as in USKGen, the user must know the identity for which he is extracting a key.

<u>Setup(1^k)</u> Return $\text{param} \leftarrow \text{GGen}(1^k)$ <u>Setup(param):</u> $A \xleftarrow{\\$} \mathcal{D}_k, B = \bar{A}$ For $i = 0, \dots, \ell$: $z_i \xleftarrow{\\$} \mathbb{Z}_p^{k+1 \times n}; Z_i = z_i^\top \cdot A \in \mathbb{Z}_p^{n \times k}$ $z' \xleftarrow{\\$} \mathbb{Z}_p^{k+1}; Z' = z'^\top \cdot A \in \mathbb{Z}_p^{1 \times k}$ $\text{pk} := (\mathcal{G}, [A]_1, ([Z_i]_1)_{0 \leq i \leq \ell}, [Z']_1)$ $\text{sk} := ((z_i)_{0 \leq i \leq \ell}, z')$ Return (pk, sk) <u>USKGen(sk, id $\in \text{ID}$)^a:</u> $t \xleftarrow{\\$} \mathbb{Z}_q^n;$ $v = \sum_{i=0}^{l(\text{id})} f_i(\text{id}) z_i t + z' \in \mathbb{Z}_q^{k+1}$ $S \xleftarrow{\\$} \mathbb{Z}_p^{n \times \mu}; T = B \cdot S \in \mathbb{Z}_p^{n \times \mu}$ $V = \sum_{i=0}^{l(\text{id})} f_i(\text{id}) Z_i T \in \mathbb{Z}_p^{(k+1) \times \mu}$ For $i = 0, \dots, \ell$: $e_i = Z_i t \in \mathbb{Z}_p^{k+1}; E_i = Z_i T \in \mathbb{Z}_p^{k+1 \times \mu}$ $\text{usk}[\text{id}] := ([t]_2, [v]_2) \in \mathbb{G}_2^n \times \mathbb{G}_2^{k+1}$ $\text{udk}[\text{id}] := ([T]_2, [V]_2, ([e_i]_2, [E_i]_2)_{i, \text{id}[i]=1})$ $\in \mathbb{G}_2^{n \times \mu} \times \mathbb{G}_2^{(k+1) \times \mu} \times (\mathbb{G}_2^{k+1} \times \mathbb{G}_2^{(k+1) \times \mu})^{\text{Ham}(\text{id})}$ Return $(\text{usk}[\text{id}], \text{udk}[\text{id}])$ <u>Enc(pk, id):</u> $r \xleftarrow{\\$} \mathbb{Z}_p^k$ $c_0 = Ar \in \mathbb{Z}_p^{k+1}$ $c_1 = (\sum_{i=0}^{l(\text{id})} f_i(\text{id}) Z_i) \cdot r \in \mathbb{Z}_p^n$ $K = z'_0 \cdot r \in \mathbb{Z}_p$ Return $\text{sk} = [K]_T$ and $C = ([c_0]_1, [c_1]_1)$	<u>USKDel(usk[id], $\tilde{\text{id}}$):</u> Note ℓ the size of id, and $\tilde{\ell}$ the size of $\tilde{\text{id}}$ // Delegating the key: $\hat{v} = v + \sum_{i=\ell+1}^{\tilde{\ell}} f_i(\text{id}') e_i \in \mathbb{Z}_p^k$ $\hat{V} = V + \sum_{i=\ell+1}^{\tilde{\ell}} f_i(\text{id}') E_i \in \mathbb{Z}_p^{k \times \mu}$ // Rerandomization of $(\hat{v}, \hat{V})$: $s' \xleftarrow{\\$} \mathbb{Z}_p^\mu; S' \xleftarrow{\\$} \mathbb{Z}_p^{\mu \times \mu}$ $t' = t + Ts' \in \mathbb{Z}_p^n;$ $T' = \hat{T} \cdot S' \in \mathbb{Z}_p^{n \times \mu}$ $v' = \hat{v} + \hat{V} \cdot s' \in \mathbb{Z}_p^k;$ $V' = \hat{V} \cdot S' \in \mathbb{Z}_p^{k \times \mu}$ // Rerandomization of e_i: For $i, \tilde{\text{id}}[i] = 1$: $e'_i = e_i + E_i s' \in \mathbb{Z}_p^{k+1};$ $E'_i = E_i \cdot S' \in \mathbb{Z}_p^{(k+1) \times \mu}$ $\text{usk}[\text{id}'] := ([t']_2, [v']_2)$ $\text{udk}[\text{id}'] := ([T']_2, [V']_2, ([e'_i]_2, [E'_i]_2)_{i, \tilde{\text{id}}[i]=1})$ Return $(\text{usk}[\text{id}'], \text{udk}[\text{id}'])$ <u>Dec(usk[id], id, C):</u> Parse $\text{usk}[\text{id}] = ([t]_2, [v]_2)$ Parse $C = ([c_0]_1, [c_1]_1)$ $\text{sk} = e([c_0]_1, [v]_2) \cdot e([c_1]_1, [t]_2)^{-1}$ Return $\text{sk} \in \mathbb{G}_T$
---	---

^a μ is the difference between the maximal size for and identity and the size of the identity id

FIGURE 2.11: HIBE BASED ON k – MDDH

2. **Selective-failure Blindness:** A potentially malicious authority cannot learn anything about the user's choice of identity during the BlindUSKGen protocol; Moreover, the authority cannot cause the BlindUSKGen protocol to fail in a manner dependent on the user's choice.

For our applications, we only need a weakened property for blindness:

3. **Weak Blindness:** A potentially malicious authority cannot learn anything about the user's choice of identity during the BlindUSKGen protocol.

2.6 Non Interactive Zero Knowledge (NIZK) Argument

A Zero Knowledge Argument introduced in [GMR89], is a set of algorithms involving two parties: a prover and a verifier. These algorithms allow the prover to prove the veracity of a statement without leaking any information. We want three properties on this set of algorithms:

- The verifier should be convinced of the statement if it is actually true
- The verifier should not be able to prove a false statement
- A listener should not learn anything from communication listening.

We call these three properties respectively **Completeness**, **Soundness** and **Zero Knowledge**. Let's formally define a NIZK Argument and describe the properties. For simplicity we describe a statement as a word in a (set of) language(s). For example to prove that a user possesses a solution from an equation, the language will be the set of all solutions of this precise equation.

Definition 13 (Non-Interactive Zero Knowledge Argument). *Assuming a randomness distribution $\mathcal{D}_{\text{param}}$ (for a public set of parameters param) and a set of languages $\{\mathcal{L}_\rho\}_\rho$ parameterized by a randomness $\rho \leftarrow \mathcal{D}_{\text{param}}$ and associated with a relation $\mathcal{R}_\rho$ (meaning that a word x belongs to $\mathcal{L}_\rho$ if and only if there exists a witness w such that $\mathcal{R}_\rho(x, w)$ holds), a NIZK argument Π for this set of languages $\{\mathcal{L}_\rho\}_\rho$ consists of five probabilistic polynomial time (PPT) algorithms $\Pi = (\text{Gen}_{\text{param}}, \text{Gen}_{\text{crs}}, \text{Prove}, \text{Sim}, \text{Ver})$ defined as follows:*

- $\text{Gen}_{\text{param}}(\mathfrak{K})$ returns the public parameters param .
- $\text{Gen}_{\text{crs}}(\text{param}, \rho)$ returns a common reference string crs and a trapdoor tk . We assume that crs contains the parameters param , a description of the language $\mathcal{L}_\rho$.
- Prove takes as input crs , a word x of the language $\mathcal{L}_\rho$ and a witness w corresponding to this word. If $(x, w) \notin \mathcal{R}_\rho$, it outputs failure. Otherwise, it outputs a proof π .
- Ver takes as input crs , a word x and a proof π . It outputs a bit b (either 1 if the proof is correct, or 0 otherwise).
- Sim takes as input crs , a trapdoor tk and a word x . It outputs a proof π for x (not necessarily in $\mathcal{L}_\rho$).

These algorithms must satisfy the following properties

Perfect Completeness: *for all adversary $\mathcal{A}$, all security parameter $\mathfrak{K}$, all public parameters $\text{param} \leftarrow \text{Gen}_{\text{param}}(\mathfrak{K})$, all randomness $\rho \leftarrow \mathcal{D}_{\text{param}}$ and all $(x, w) \in \mathcal{R}_\rho$, the following holds:*

$$\Pr[(\text{crs}, \text{tk}) \leftarrow \text{Gen}_{\text{crs}}(\text{param}, \rho); \pi \leftarrow \text{Prove}(\text{crs}, x, w) : \text{Ver}(\text{crs}, x, \pi) = 1] = 1$$

Perfect Soundness: For a given security parameter $\mathfrak{K}$, a scheme achieves soundness if for all adversary $\mathcal{A}$, we have

$$\Pr[\text{param} \leftarrow \text{Gen}_{\text{param}}(\mathfrak{K}); \rho \leftarrow \mathcal{D}_{\text{param}}; (\text{crs}, \text{tk}) \leftarrow \text{Gen}_{\text{crs}}(\text{param}, \rho); \\ (x, \pi) \leftarrow \mathcal{A}(\text{param}, \text{crs}, \rho) : x \notin \mathcal{L}_\rho \wedge \text{Ver}(\text{crs}, x, \pi) = 1] = 0$$

Perfect Zero-Knowledge: for all $\mathfrak{K}$, all $\text{param} \leftarrow \text{Gen}_{\text{param}}(\mathfrak{K})$, all $\rho \leftarrow \mathcal{D}_{\text{param}}$, all $(\text{crs}, \text{tk}) \leftarrow \text{Gen}_{\text{crs}}(\text{param}, \rho)$ and all $(x, w) \in \mathcal{R}_\rho$, the distributions $\text{Prove}(\text{crs}, x, w)$ and $\text{Sim}(\text{crs}, \text{tk}, x)$ are the same.

We say that Zero-Knowledge Proofs are Non-interactive (NIZK) if there is no interaction between the prover and the verifier. Note that efficient NIZK proof for pairing equations were found only in 2008 by Groth and Sahai [GS08]. Today their scheme has been optimized and improved in different work like [BFI⁺10] but this is still the only efficient way to deal with this problem.

2.7 Commitments and Smooth Projective Hash Functions

In this section we will define, give security definition and example of Commitment and Smooth Projective Hash Functions. We will use these two primitives together to achieve different kind of protocols like Oblivious Transfer or Oblivious Language-based Envelop. It has already been used in many cases like Oblivious Signature-based Envelop [BPV12] or Password Authenticated Key Exchange [ABB⁺13].

Definition 14 (One Way Hash Function Family). *A One Way Hash Function Family is a set $\mathcal{H} = \{H_k\}_{k \in K}$ of functions from a space A to a space B that verifies:*

- *The probability that an adversary find $k \in K, (a, a') \in A^2$ such that $a \neq a'$ and $H_k(a) = H_k(a')$ is negligible. This is called the collision resistance property.*
- *$\forall k \in K$ H_k is efficiently computable.*

2.7.1 Definitions and Security

Commitments allow a user to commit to a value without revealing it, but without the possibility to later change his mind. It is composed of these algorithms:

- $\text{SetupCom}(1^\mathfrak{K})$ generates the system parameters param , according to the security parameter $\mathfrak{K}$.
- $\text{KeyGen}(\text{param})$ generates a commitment key ck .
- $\text{Commit}(\text{ck}, m; r)$ produces a commitment c and an opening data δ on the input message $m \in \mathcal{M}$ using the random coins $r \xleftarrow{\$} \mathcal{R}$.

- $\text{VerCom}(\text{ck}, \delta, c, m; r)$ outputs 1 if c is a commitment of m with the randomness r for the commitment key ck along with opening data δ .

Such a commitment scheme should be both **hiding**, which says that the commit phase does not leak any information about m , and **binding**, which says that the decommit phase should not be able to open to two different messages.

Additional features are also sometimes required, such as non-malleability, extractability, and/or equivocability. We may also include a label ℓ^{12} , which is an additional public information that has to be the same in both the commit and the decommit phases.

A commitment scheme is said **equivocable** if it has a second setup $\text{SetupComT}(1^{\mathbb{R}})$ that additionally outputs a trapdoor τ , and two algorithms

- $\text{SimCom}^{\ell}(\tau)$ that takes as input the trapdoor τ and a label ℓ and outputs a pair (C, eqk) , where C is a commitment and eqk an equivocation key;
- $\text{OpenCom}^{\ell}(\text{eqk}, C, x)$ that takes as input a commitment C , a label ℓ , a message x , an equivocation key eqk , and outputs an opening data δ for C and ℓ on m .

such that the following properties are satisfied:

- *trapdoor correctness* (all simulated commitments can be opened on any message)
- *setup indistinguishability* (one cannot distinguish the parameters param generated by SetupCom from the one generated by SimCom)
- *simulation indistinguishability* (one cannot distinguish a real commitment (generated by Com) from a fake commitment (generated by SCom) even with oracle access to fake commitments), denoting by SCom the algorithm that takes as input the trapdoor τ , a label ℓ and a message x and which outputs $(C, \delta) \xleftarrow{\$} \text{SCom}^{\ell}(\tau, x)$, computed as $(C, \text{eqk}) \xleftarrow{\$} \text{SimCom}^{\ell}(\tau)$ and $\delta \leftarrow \text{OpenCom}^{\ell}(\text{eqk}, C, x)$.

A commitment scheme $\mathcal{C}$ is said **extractable** if it has a second setup $\text{SetupComT}(1^{\mathbb{R}})$ that additionally outputs a trapdoor τ , and a new algorithm

- $\text{ExtCom}^{\ell}(\tau, C)$ which takes as input the trapdoor τ , a commitment C , and a label ℓ , and outputs the committed message x , or $\perp$ if the commitment is invalid.

such that the following properties are satisfied:

¹²The label is here to avoid resend of the exact same commitment, thus the label will contain session's id

- *trapdoor correctness* (all commitments honestly generated can be correctly extracted: for all ℓ, x , if $(C, \delta) \xleftarrow{\$} \text{Com}^\ell(x)$ then $\text{ExtCom}^\ell(C, \tau) = x$)
- *setup indistinguishability* (as above)
- *binding extractability* (one cannot fool the extractor, *i.e.*, produce a commitment and a valid opening data to an input x while the commitment does not extract to x).

Smooth projective hash functions (SPHF) were introduced by Cramer and Shoup in [CS02] for constructing encryption schemes. A projective hashing family is a family of hash functions that can be evaluated in two ways: using the (secret) hashing key, one can compute the function on every point in its domain, whereas using the (public) *projected* key one can only compute the function on a special subset of its domain. Such a family is deemed *smooth* if the value of the hash function on any point outside the special subset is independent of the projected key. The notion of SPHF has already found applications in various contexts in cryptography (*e.g.* [GL03, Kal05, ACP09]). A Smooth Projective Hash Function over a language $\mathcal{L} \subset X$, onto a set $\mathcal{G}$, is defined by five algorithms (Setup, HashKG, ProjKG, Hash, ProjHash):

- Setup($1^\mathfrak{K}$) where $\mathfrak{K}$ is the security parameter, generates the global parameters param of the scheme, and the description of an $\mathcal{NP}$ language $\mathcal{L}$ where there exists witness to prove that a word belong to a language;
- HashKG($\mathcal{L}, \text{param}$), outputs a hashing key hk for the language $\mathcal{L}$;
- ProjKG($\text{hk}, (\mathcal{L}, \text{param}), W$), derives the projection key hp from the hashing key hk and the word W .
- Hash($\text{hk}, (\mathcal{L}, \text{param}), W$), outputs a hash value $v \in \mathcal{G}$, using the hashing key hk and the word W .
- ProjHash($\text{hp}, (\mathcal{L}, \text{param}), W, w$), outputs the hash value $v' \in \mathcal{G}$, using the projection key hp and the witness w that the word $W \in \mathcal{L}$.

In the following, we assume $\mathcal{L}$ is a hard-partitioned subset of X , *i.e.* it is computationally hard to distinguish a random element in $\mathcal{L}$ from a random element in $X \setminus \mathcal{L}$. An SPHF should satisfy the following properties:

- *Correctness*: Let $W \in \mathcal{L}$ and w a witness of this membership. Then, for all param , all hashing keys hk and associated projection keys hp we have $\text{Hash}(\text{hk}, (\mathcal{L}, \text{param}), W) = \text{ProjHash}(\text{hp}, (\mathcal{L}, \text{param}), W, w)$.
- *Smoothness*: For all $W \in X \setminus \mathcal{L}$ the following distributions are statistically indistinguishable:

$$\begin{aligned} \Delta_0 &= \left\{ (\mathcal{L}, \text{param}, W, \text{hp}, v) \left| \begin{array}{l} \text{param} = \text{Setup}(1^\mathfrak{K}), \text{hk} = \text{HashKG}(\mathcal{L}, \text{param}), \\ \text{hp} = \text{ProjKG}(\text{hk}, (\mathcal{L}, \text{param}), W), \\ v = \text{Hash}(\text{hk}, (\mathcal{L}, \text{param}), W) \end{array} \right. \right\} \\ \Delta_1 &= \left\{ (\mathcal{L}, \text{param}, W, \text{hp}, v) \left| \begin{array}{l} \text{param} = \text{Setup}(1^\mathfrak{K}), \text{hk} = \text{HashKG}(\mathcal{L}, \text{param}), \\ \text{hp} = \text{ProjKG}(\text{hk}, (\mathcal{L}, \text{param}), W), v \xleftarrow{\$} \mathcal{G} \end{array} \right. \right\}. \end{aligned}$$

This is formalized by: $\text{Adv}_{\text{SPHF}}^{\text{smooth}}(\mathfrak{K}) = \sum_{V \in \mathcal{G}} |\Pr_{\Delta_1}[v = V] - \Pr_{\Delta_0}[v = V]|$ is negligible.

- *Pseudo-Randomness*: If $W \in \mathfrak{L}$, then without a witness of membership the two previous distributions should remain computationally indistinguishable. For any PPT adversary $\mathcal{A}$, this advantage is negligible:

$$\text{Adv}_{\text{SPHF}, \mathcal{A}}^{\text{pr}}(\mathfrak{K}) = |\Pr_{\Delta_1}[\mathcal{A}(\mathfrak{L}, \text{param}, W, \text{hp}, v) = 1] - \Pr_{\Delta_0}[\mathcal{A}(\mathfrak{L}, \text{param}, W, \text{hp}, v) = 1]|$$

2.7.2 Examples

Examples of Commitment: As well described in [ABB⁺13] we have interesting examples of commitment schemes:

An encryption scheme $\mathcal{E}$ can be used as a commitment in the following way:

- $\text{SetupCom}(1^{\mathfrak{K}}) \leftarrow \text{Setup}_{\mathcal{E}}(1^{\mathfrak{K}})$
- $\text{KeyGen}(\text{param}) \leftarrow \text{KeyGen}_{\mathcal{E}}(\text{param}): \text{ck} \leftarrow \text{pk}$
- $\text{Commit}(\text{ck}, m) \leftarrow \text{Encrypt}_{\mathcal{E}}(\text{ek}, m; r) : (c, \delta) = (c_{\mathcal{E}}, r)$
- $\text{VerCom}(\text{ck}, \delta, c) \leftarrow (c == \text{Encrypt}_{\mathcal{E}}(\text{ek}, m; r))$

This commitment is perfectly binding since the correctness of the encryption scheme ensure that the message encrypted is m . The scheme is also computationally hiding because of the *ind-cpa* security of the encryption scheme. Note that from this type of commitment it is easy to have an extractable commitment by adding as extraction algorithm the decryption algorithm of $\mathcal{E}$ the trapdoor will be the decryption key.

The commitment from [AFG⁺10]:

- $\text{SetupCom}(1^{\mathfrak{K}}): \text{param} \leftarrow \text{GGen}(1^{\mathfrak{K}})$
- $\text{KeyGen}(\text{param}):$ Outputs an independent generator T .
- $\text{Commit}(T, m; r) : (c, \delta) = (g_2^r T^m, g_1^r) \in \mathbb{G}_T$
- $\text{VerCom}(\text{ck}, \delta, c) : \text{Checks } e(g_1, c/T^m) = e(\delta, g_2) \in \mathbb{G}_T$

It is perfectly hiding: indeed from one commitment every message could have been in it: if $T = g^t$ and $c = g_2^a$ then a valid δ for m is g_1^{a-tm} . The scheme is computationally binding under DDH in $\mathbb{G}_2$. This lead to an equivocal commitment scheme when leaking the discrete logarithm of T as equivocation key.

Chameleon hash functions

Chameleon hash functions can instantiate a perfectly hiding commitment. A chameleon hash function is a hash function where there is an additional algorithm using a trapdoor key, which, given a message a randomness and a second message, can find a second randomness such that the hash of the first message with the first randomness is equal to the hash of the second message with the second randomness. Moreover without the trapdoor key the function should be collision resistant. If a commitment is a hash value since it could be from any message, the commitment is perfectly hiding under the correctness of the chameleon hash function. The commitment is computationally binding under the collision resistance of the hash function. CDH-based Chameleon Hash [BC15] is a nice example of such scheme and we will use it later.

Definition 15 (Chameleon Hash Function). *A chameleon hash function is made of five algorithm (KeyGen, VKeyGen, CH, Coll, Valid)*

- $\text{KeyGen}(\mathfrak{K})$ returns a hashing key ck and a trapdoor key tk .
- $\text{VKeyGen}(\text{ck})$ generates the verification keys vtk and add an element to ck
- $\text{CH}(\text{ck}, m; r)$ returns a hash value of the message m using the randomness r .
- $\text{Coll}(m, r, m', \text{tk})$ returns a random r' such that the couples (m, r) and (m', r') have the same hash value.
- $\text{Valid}(\text{ck}, m, a, d, \text{vtk})$ returns 1 if the hash is valid.

$\text{KeyGen}(\mathfrak{K})$: Outputs $\text{ck} = (g, h)$ and $\text{tk} = \alpha$, where $g^\alpha = h$	$\text{CH}(\text{ck}, m; r)$: Picks $r \xleftarrow{\$} \mathbb{Z}_p$ outputs $a = h^r g^m$, sets $d = f^r$
$\text{VKeyGen}(\text{ck})$: Appends f to ck and $\text{vtk} = \log_g(f)$	$\text{Coll}(m, r, m', \text{tk})$: outputs $r' = r + (m - m')/\alpha$
	$\text{Valid}(\text{ck}, m, a, d, \text{vtk})$: Checks $a = h^m \cdot d^{1/\text{vtk}}$

FIGURE 2.12: CDH-BASED CHAMELEON HASH [BC15]

This is a CDH variant of the Pedersen chameleon hash [Ped92] (Figure 2.12).

Example of SPHF: Let's describe a simple example:

Let's say param contains $(g, h) \in \mathcal{G}$

- Language: $\mathcal{L} = \{(g_1, h_1) | \exists \alpha \in \mathbb{Z}_p, g_1 = g^\alpha \wedge h_1 = h^\alpha\}$, the witness: $w = \alpha$.

- $\text{HashKG}(\mathcal{L}) \leftarrow \text{hk} = \text{sk}_2 = (x_1, x_2)$
- $\text{ProjKG}(\text{hk}; \mathcal{L}) \leftarrow \text{hp} = \text{pk}_2 = Y_1^{x_1} g^{x_2}$
- $\text{Hash}(\text{sk}_2; \mathcal{L}, \mathcal{C}) \leftarrow e(C_1, \nu)^{x_1} \cdot (e(C_3, g) / (e(h, \text{pk}_1) \cdot e(\nu \mathcal{F}(\mathcal{M}), C_2)))^{x_2}$
- $\text{ProjHash}(\text{pk}_2, \mathcal{L}, \mathcal{C}, r) \leftarrow e(\text{pk}_2^r, \nu)$

FIGURE 2.13: SPHF OVER LINEAR ENCRYPTIONS OF VALID WATERS SIGNATURE

- $\text{HashKG}(\mathcal{L}_M) : \text{hk} \xleftarrow{\$} \mathbb{Z}_p^{k+2},$
- $\text{ProjKG}(\text{hk}, (\mathcal{L}_M, \ell, [\mathcal{C}]_2)) :$ Setting $\text{ht} = \begin{pmatrix} \text{pk}_1 \\ \text{pk}_2 + \theta \text{pk}_3 \end{pmatrix}$, we have $[\text{hp}]_2 = [\text{hk}^\top \text{ht}]_2$, where $\theta = H(\ell, e)$,
- $\text{Hash}(\text{hk}, (\mathcal{L}_M, \ell, [\mathcal{C}]_2)) : [H]_2 \leftarrow [\text{hk}^\top (\mathcal{C} - (0 \mid M)^\top)]_2,$
- $\text{ProjHash}(\text{hp}, (\mathcal{L}_M, \ell, [\mathcal{C}]_2), \mu) : [H']_2 \leftarrow [\text{hp} \mu]_2$

FIGURE 2.14: SPHF OVER k – MDDH CRAMER SHOUP ENCRYPTION

- Hashing key (private): $\text{hk} = (\lambda, \mu) \xleftarrow{\$} \mathbb{Z}_p^2$
- Projection key (public): $\text{hp} = g^\lambda h^\mu$
- Hash: $H = g_1^\lambda h_1^\mu$
- Projection: $H' = \text{hp}^\alpha$

At this point we can make a link between some primitives already mentioned in Figures 2.3 and 2.10. We will describe an SPHF on the language: linear encryptions of valid waters signature which has already been used in the literature [BPV12].

We can build a SPHF over the k – MDDH Cramer Shoup encryption by following [EHK⁺13, BBC⁺13b] described in figure 2.14 ($\text{ek} = (\text{ek}_1, \text{ek}_2, \text{ek}_3)$ coming from the encryption scheme).

2.8 Oblivious Transfer

An Oblivious Transfer is a protocol involving a sender and a receiver that we will sometimes call database and user respectively. It allows a user to have access to a special line in a database that he is requesting. This has to be done under two conditions: first the user should not have any information about the other lines, second the database should not know which line has been requested. Let's describe this formally:

2.8.1 Definition and Security

Definition and Security Model for Oblivious Transfer We will now rigorously describe our OT protocol formalism using languages. It will thus fit what as been said and the formalism of SPHF that we will need.

In this protocol a server $\mathcal{S}$ possesses a database of n lines $(m_1, \dots, m_n) \in (\{0, 1\}^{\mathfrak{K}})^n$. And a user U will be able to recover m_k as soon as he requested it. We will say that he is able to recover a line as long as he owns a word $W_k \in \mathfrak{L}_k$ ¹³. As we consider simulation-based security (in the UC framework), we allow a simulated setup **SetupT** to be run instead of the classical setup **Setup** in order to allow the simulator to possess some trapdoors. Those two setup algorithms should be indistinguishable.

Definition 16 (Oblivious Transfer). *An OT scheme can be formalized in five algorithms (**Setup**, **KeyGen**, **DBGen**, **Samp**, **Verify**), along with an interactive protocol $\text{Protocol}\langle \mathcal{S}, U \rangle$:*

- **Setup**($1^{\mathfrak{K}}$), where $\mathfrak{K}$ is the security parameter, generates the global parameters **param**, among which the number n ;
- or **SetupT**($1^{\mathfrak{K}}$), where $\mathfrak{K}$ is the security parameter, additionally allows the existence¹⁴ of a trapdoor **tk** for the collection of languages $(\mathfrak{L}_1, \dots, \mathfrak{L}_n)$.
- **KeyGen**(**param**, $\mathfrak{K}$) generates, for all $i \in \{1, \dots, n\}$, description of the language $\mathfrak{L}_i$ (as well as the language key $\text{sk}_{\mathfrak{L}_i}$ if need be). If the parameters **param** were defined by **SetupT**, this implicitly also defines the common trapdoor **tk** for the collection of languages $(\mathfrak{L}_1, \dots, \mathfrak{L}_n)$.
- **Samp**(**param**, $(\text{sk}_{\mathfrak{L}_i})_{i \in \{1, \dots, n\}}$ ¹⁵) generates a word $W_i \in \mathfrak{L}_i$;
- **Verify** _{i} ($W_i, \mathfrak{L}_i$) checks whether W_i is a valid word in the language $\mathfrak{L}_i$. It outputs 1 if the word is valid, 0 otherwise;
- **Protocol** $\langle \mathcal{S}((\mathfrak{L}_1, \dots, \mathfrak{L}_n), (m_1, \dots, m_n)), U((\mathfrak{L}_1, \dots, \mathfrak{L}_n), W_i) \rangle$, which is executed between the server $\mathcal{S}$ with the private database $(m_1, \dots, m_n)$ and corresponding languages $(\mathfrak{L}_1, \dots, \mathfrak{L}_n)$, and the user U with the same languages and the word W_i , proceeds as follows. If the algorithm **Verify** _{i} ($W_i, \mathfrak{L}_i$) returns 1, then U receives m_k , otherwise it does not. In any case, $\mathcal{S}$ does not learn anything.

We define the security of Oblivious Transfer in the UC model because we will use only UC-secure Oblivious Transfer.

The ideal functionality of an Oblivious Transfer (OT) protocol was given in [Can01, CKWZ13, ABB⁺13], and an adaptive version in [GH08]. We describe it in figure 2.15.

¹³The languages $(\mathfrak{L}_1, \dots, \mathfrak{L}_n)$ will be assumed to be a trapdoor collection of languages, publicly verifiable and self-randomizable, these notion are formally defined in the next section, where we introduce what is an Oblivious-based Language Envelop

¹⁴The specific trapdoor will depend on the languages and be computed in the **KeyGen** algorithm.

¹⁵Optional

The functionality $\mathcal{F}_{(1,n)\text{-OT}}$ is parametrized by a security parameter κ . It interacts with an adversary $\mathcal{S}$ and a set of parties $\mathfrak{P}_1, \dots, \mathfrak{P}_N$ via the following queries:

- **Upon receiving an input** (**Send**, **sid**, **ssid**, $\mathfrak{P}_i$, $\mathfrak{P}_j$, $(m_1, \dots, m_n)$) **from** $\mathfrak{P}_i$, with $m_k \in \{0, 1\}^\kappa$: record the tuple $(\text{sid}, \text{ssid}, \mathfrak{P}_i, \mathfrak{P}_j, (m_1, \dots, m_n))$ and reveal (**Send**, **sid**, **ssid**, $\mathfrak{P}_i$, $\mathfrak{P}_j$) to $\mathcal{S}$. Ignore further **Send**-message with the same **ssid** from $\mathfrak{P}_i$.
- **Upon receiving an input** (**Receive**, **sid**, **ssid**, $\mathfrak{P}_i$, $\mathfrak{P}_j$, s) **from** $\mathfrak{P}_j$, with $s \in \{1, \dots, n\}$: record the tuple $(\text{sid}, \text{ssid}, \mathfrak{P}_i, \mathfrak{P}_j, s)$, and reveal (**Receive**, **sid**, **ssid**, $\mathfrak{P}_i$, $\mathfrak{P}_j$) to $\mathcal{S}$. Ignore further **Receive**-message with the same **ssid** from $\mathfrak{P}_j$.
- **Upon receiving a message** (**Sent**, **sid**, **ssid**, $\mathfrak{P}_i$, $\mathfrak{P}_j$) **from the adversary** $\mathcal{S}$: ignore the message if $(\text{sid}, \text{ssid}, \mathfrak{P}_i, \mathfrak{P}_j, (m_1, \dots, m_n))$ or $(\text{sid}, \text{ssid}, \mathfrak{P}_i, \mathfrak{P}_j, s)$ is not recorded; otherwise send (**Sent**, **sid**, **ssid**, $\mathfrak{P}_i$, $\mathfrak{P}_j$) to $\mathfrak{P}_i$ and ignore further **Sent**-message with the same **ssid** from the adversary.
- **Upon receiving a message** (**Received**, **sid**, **ssid**, $\mathfrak{P}_i$, $\mathfrak{P}_j$) **from the adversary** $\mathcal{S}$: ignore the message if $(\text{sid}, \text{ssid}, \mathfrak{P}_i, \mathfrak{P}_j, (m_1, \dots, m_n))$ or $(\text{sid}, \text{ssid}, \mathfrak{P}_i, \mathfrak{P}_j, s)$ is not recorded; otherwise send (**Received**, **sid**, **ssid**, $\mathfrak{P}_i$, $\mathfrak{P}_j, m_s$) to $\mathfrak{P}_j$ and ignore further **Received**-message with the same **ssid** from the adversary.

FIGURE 2.15: IDEAL FUNCTIONALITY FOR 1-OUT-OF- n OBLIVIOUS TRANSFER $\mathcal{F}_{(1,n)\text{-OT}}$

We will also need the ideal functionality of an adaptive Oblivious Transfer (OT) protocol. It was given in [Can01, CKWZ13, ABB⁺13], and an adaptive version in [GH08]. We here combine them and rewrite it in simple UC and using our language formalism (this enables us to get rid of **Sent** and **Received** queries from the adversary since the delayed outputs are automatically considered in this simpler framework: We implicitly let the adversary determine if it wants to acknowledge the fact that a message was indeed sent). The first step for the sender (**Send** query) consists in telling the functionality he is willing to take part in the protocol, giving as input his intended receiver and the messages he is willing to send (up to $n_{\max}$ messages). For the receiver, the first step (**Receive** query) consists in giving the functionality the name of the player he intends to receive the messages from, as well as his words. If the word does belong to the language, the receiver recovers the sent message, otherwise, he only gets a special symbol $\perp$. The resulting functionality $\mathcal{F}_{\text{OT}}^\mathcal{E}$ is given in Figure 2.16. Recall that there is no need to give an explicit description of the corruptions in the simple version of UC [CCL15].

The functionality $\mathcal{F}_{\text{OT}}^{\mathcal{L}}$ is parametrized by a security parameter κ and a set of languages $(\mathcal{L}_1, \dots, \mathcal{L}_n)$ along with the corresponding public verification algorithms $(\text{Verify}_1, \dots, \text{Verify}_n)$. It interacts with an adversary $\mathcal{S}$ and a set of parties $\mathcal{P}_1, \dots, \mathcal{P}_N$ via the following queries:

- **Upon receiving from party $\mathcal{P}_i$ an input $(\text{NewDataBase}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, (m_1, \dots, m_n))$** , with $m_k \in \{0, 1\}^{\kappa}$ for all k : record the tuple $(\text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, (m_1, \dots, m_n))$ and reveal $(\text{Send}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j)$ to the adversary $\mathcal{S}$. Ignore further **NewDataBase**-message with the same **ssid** from $\mathcal{P}_i$.
- **Upon receiving an input $(\text{Receive}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, W_k)$ from party $\mathcal{P}_j$** : ignore the message if $(\text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, (m_1, \dots, m_n))$ is not recorded. Otherwise, reveal $(\text{Receive}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j)$ to the adversary $\mathcal{S}$ and send the message $(\text{Received}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, m'_k)$ to $\mathcal{P}_j$ where $m'_k = m_k$ if $\text{Verify}_k(W_k, \mathcal{L}_k)$ returns 1, and $m'_k = \perp$ otherwise.

(Non-Adaptive case: Ignore further **Receive**-message with the same **ssid** from $\mathcal{P}_j$.)

FIGURE 2.16: IDEAL FUNCTIONALITY FOR (ADAPTIVE) OBLIVIOUS TRANSFER $\mathcal{F}_{\text{OT}}^{\mathcal{L}}$

2.8.2 Example

The example from [ABB⁺13] is interesting in our context since it makes use of commitments and Smooth Projective Hash Functions. We will talk again about this scheme in Section 5.2.3. The key idea of this scheme is to create a language for each line of the database: for a line m_i we will create the language of all commitments of the number i . Thus the sender will mask each line m_j with the hashed value of the commitment for the language $\{\text{com}(j)\}$. But the commitment lie only in one language: it means that the receiver can only recover the hashed value for the language he committed to. Moreover the sender does not know which line has been committed as wished. In the example we use a commitment, a **ind-cpa**-secure encryption and a Pseudo-Random Generator F with input size equal to the plaintext size and output size equal to the size of the lines in the database.

CRS generation:

$\rho \xleftarrow{\$} \text{SetupCom}(1^{\mathfrak{R}}), \text{param}_{\text{cpa}} \xleftarrow{\$} \text{Setup}_{\text{cpa}}(1^{\mathfrak{R}}).$

Pre-flow :

1. $\mathcal{S}$ generates a key pair $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}_{\text{cpa}}(\text{param}_{\text{cpa}})$ for $\mathcal{E}$, stores sk and completely erases the random coins used by $\text{KeyGen}_{\text{cpa}}$.
2. $\mathcal{S}$ publishes pk .

Index query on s :

1. $\mathcal{R}$ chooses a random value J , computes $R \leftarrow F(J)$ and encrypts J under pk : $c \xleftarrow{\$} \text{Encrypt}_{\text{cpa}}(\text{pk}, J)$
2. $\mathcal{R}$ computes $(C, \delta_s) \xleftarrow{\$} \text{Com}^\ell(s)$ with $\ell = (\text{sid}, \text{ssid}, \mathcal{R}, \mathcal{S})$
3. $\mathcal{R}$ stores δ_s, R , completely erases the random coins used and sends C and c to $\mathcal{S}$

Database answer $(m_1, \dots, m_n)$:

1. $\mathcal{S}$ decrypts $J \leftarrow \text{Decrypt}_{\text{cpa}}(\text{sk}, c)$ and gets $R \leftarrow F(J)$
2. For each line $k = 1, \dots, n$,
 - $\mathcal{S}$ computes $\text{hk}_k \xleftarrow{\$} \text{HashKG}(\mathfrak{L}_k)$, $\text{hp}_k \leftarrow \text{ProjKG}(\text{hk}_k, \mathfrak{L}_k, (\ell, C))$,
 $K_k \leftarrow \text{Hash}(\text{hk}_k, \mathfrak{L}_k, (\ell, C))$,
 - $\mathcal{S}$ computes $Q_k \leftarrow R \oplus K_k \oplus m_k$
3. $\mathcal{S}$ erases everything except $(\text{hp}_k, Q_k)_{k=1, \dots, n}$ and sends it to $\mathcal{R}$.

Data recovery:

Upon receiving $(\text{hp}_k, Q_k)_{k=1, \dots, n}$, $\mathcal{R}$ computes $K_s \leftarrow \text{ProjHash}(\text{hp}_s, \mathfrak{L}_s, (\ell, C), \delta)$ and gets $m_s \leftarrow R \oplus K_s \oplus Q_s$.
 Then $\mathcal{R}$ erases everything except m_s .

FIGURE 2.17: UC-SECURE 1-OUT-OF- n OT FROM AN SPHF-FRIENDLY COMMITMENT

Chapter 3

New Featured-Identity-based Encryption and Generalization

First, we present a generalization of IBE named Downgradable IBE that will help us to build a generic transformation from IBE to Attribute-based Encryption in the next chapter. We also describe the transformation to come from this new primitive to other already known Identity-based primitives. Then we present a generic transformation from IBE to Blind IBE. This primitive allows a user to ask for a user secret key without the authority knowing which user has been requested.

3.1 Downgradable Identity-based Encryption

The purpose of this section is not to just add another variant of Identity-based encryption. What we aim here is to create an IBE scheme with the minimal additional properties allowing it to be transformed into Attribute-based Encryption¹. We found out that this variant of IBE that we named Downgradable IBE can be seen as a generalization of many existing IBE's variant. First, we define this new primitive and then we describe the transformations from it to other Identity-based primitives.

3.1.1 Downgradable Identity-based Encryption

In this section we introduce the notion of Downgradable Identity-Based Encryption. For simplicity we are going to express it in term of Key Encapsulation.

Definition 17 (Downgradable Identity-based Key Encapsulation Scheme). A *Downgradable identity-based key encapsulation (DIBKEM) scheme* DIBKEM consists of five PPT algorithms $\text{DIBKEM} = (\text{Setup}, \text{USKGen}, \text{Enc}, \text{Dec}, \text{USKDown})$ with the following properties.

¹We describe the transformation in Section 4.2.

Procedure Initialize: $(pk, sk) \xleftarrow{\\$} \text{Setup}(\mathfrak{K})$ Return pk Procedure USKGen(id): $\mathcal{Q}_{ID} = \mathcal{Q}_{ID} \cup \{id\}$ Return $usk[id] \xleftarrow{\\$} \text{USKGen}(sk, id)$	Procedure Enc(id*): //one query $(K^*, C^*) \xleftarrow{\\$} \text{Enc}(pk, id^*)$ $K^* \xleftarrow{\\$} \mathcal{K}; C^* \xleftarrow{\\$} \text{CS}$ Return (K^*, C^*) Procedure Finalize(β): Return $(\neg(id^* \preceq \mathcal{Q}_{ID})) \wedge \beta$
--	--

FIGURE 3.1: SECURITY GAMES $\text{PR-ID-CPA}_{\text{real}}$ AND $\text{PR-ID-CPA}_{\text{rand}}$ (USING (K^*, C^*) BOXED) FOR DEFINING PR-ID-CPA-SECURITY FOR DIBKEM.

- The probabilistic key generation algorithm $\text{Setup}(\mathfrak{K})$ returns the (master) public/secret key (pk, sk) . We assume that pk implicitly defines a message space $\mathcal{M}$, an identity space ID , a key space $\mathcal{K}$, and ciphertext space CS .
- The probabilistic user secret key generation algorithm $\text{USKGen}(sk, id)$ returns the user secret-key $usk[id]$ for identity $id \in ID$.
- The probabilistic encapsulation algorithm $\text{Enc}(pk, id)$ returns the symmetric key $K \in \mathcal{K}$ together with a ciphertext $C \in CS$ with respect to identity id .
- The deterministic decapsulation algorithm $\text{Dec}(usk[id], id, C)$ returns the decapsulated key $K \in \mathcal{K}$ or the reject symbol $\perp$.
- The probabilistic user secret key downgrade algorithm $\text{USKDown}(usk[id], \tilde{id})$ returns the user secret-key $usk[\tilde{id}]$ as long as $\tilde{id} \preceq id^2$.

For perfect correctness we require that for all $\mathfrak{K} \in \mathbb{N}$, all pairs (pk, sk) generated by $\text{Setup}(\mathfrak{K})$, all identities $id \in ID$, all $usk[id]$ generated by $\text{USKGen}(sk, id)$ and all (sk, C) output by $\text{Enc}(pk, id)$:

$$\Pr[\text{Dec}(usk[id], id, C) = sk] = 1.$$

Moreover, we also require the distribution of $usk[\tilde{id}]$ from $\text{USKDown}(usk[id], \tilde{id})$ to be identical to the one from $\text{USKGen}(sk, \tilde{id})$.

The security requirements we consider here are indistinguishability and anonymity against chosen plaintext and identity attacks. Instead of defining both security notions separately, we define pseudorandom ciphertexts against chosen plaintext and identity attacks (PR-ID-CPA) which means that challenge key and ciphertext are both pseudorandom. We define PR-ID-CPA-security of DIBKEM formally via the games given in Figure 3.1.

Definition 18 (PR-ID-CPA Security). A downgradable identity-based key encapsulation scheme DIBKEM is PR-ID-CPA-secure if for all PPT $\mathcal{A}$,

$\text{Adv}_{\text{DIBKEM}}^{\text{pr-id-cpa}}(\mathcal{A}) := |\Pr[\text{PR-ID-CPA}_{\text{real}}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{PR-ID-CPA}_{\text{rand}}^{\mathcal{A}} \Rightarrow 1]|$ is negligible.

² $\tilde{id} \preceq id$ means that for all i , if $\tilde{id}_i = 1$ then $id_i = 1$

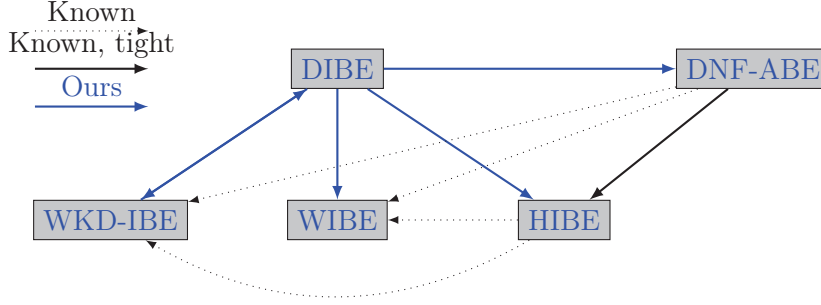


FIGURE 3.2: RELATIONS BETWEEN PRIMITIVES

We stress the importance of the condition: $(\neg(\text{id}^* \preceq \mathcal{Q}_{\text{ID}}))$. This is here to guarantee that the adversary did not query an identity that can be downgraded to the challenge one, as this would allow for a trivial attack.

Motivations: The point of creating Downgradable IBE was to construct a primitive the closer from IBE that could be transformed into Attribute-based Encryption. Attribute-Based Encryption (ABE), introduced by Sahai and Waters [SW05], is a generalization of both identity-based encryption and broadcast encryption. It gives a flexible way to define the target group of people who can receive the message: the target set can be defined in a more structural way via access policies on the user's attributes. While broadcast encryption can be obtained from WIBE, as far as we know, this work is the only efficient generic construction of ABE from a variant of IBE. The transformation from DIBE to ABE have for access policies as boolean formulas on the user's attributes in the DNF.

3.1.2 Generalization of Existing Id-based Primitives

As explained previously we will give transformations from DIBE to different variant of IBE. In the figure 3.2 we describe the new transformation we will achieve along with the existing ones.

From DIBE to WIBE

Wildcard Identity-Based Encryption is a concept introduced in [ACD⁺06]. The idea is to be able to encrypt messages for several identities by fixing some identity bits and letting others free (symbolized by the $*$). Thus only people with identity matching the one used to encrypt can decrypt. We say that id matches id' if $\forall i \text{id}_i = \text{id}'_i$ or $\text{id}'_i = *$ ³.

Definition 19 (Wildcard Identity-based Key Encapsulation Scheme). *A Wildcard identity-based key encapsulation scheme WIBKEM consists of five PPT algorithms $\text{WIBKEM} = (\text{Setup}, \text{USKGen}, \text{Enc}, \text{Dec})$ with the following properties.*

³ id_i are bits or group of bits called pattern

- The probabilistic key generation algorithm $\text{Setup}(\mathfrak{K})$ returns the (master) public/secret key (pk, sk) . We assume that pk implicitly defines a message space $\mathcal{M}$, an identity space $\text{ID} = \{0, 1\}^n$, a key space $\mathcal{K}$, and ciphertext space CS .
- The probabilistic user secret key generation algorithm $\text{USKGen}(\text{sk}, \text{id})$ returns the user secret-key $\text{usk}[\text{id}]$ for identity $\text{id} \in \text{ID}$.
- The probabilistic encapsulation algorithm $\text{Enc}(\text{pk}, \text{id})$ returns the symmetric key $K \in \mathcal{K}$ together with a ciphertext $\text{C} \in \text{CS}$ with respect to an identity $\text{id} \in \text{ID} = \{0, 1, *\}^n$, this means that $\forall i, \text{id}_i \in \{0, 1, *\}$.
- The deterministic decapsulation algorithm $\text{Dec}(\text{usk}[\text{id}], \hat{\text{id}}, \text{C})$ returns the decapsulated key $K \in \mathcal{K}$ or the reject symbol $\perp$.

For perfect correctness we require that for all $\mathfrak{K} \in \mathbb{N}$, all pairs (pk, sk) generated by $\text{Setup}(\mathfrak{K})$, all identities $\text{id} \in \text{ID}$, all $\text{usk}[\text{id}]$ generated by $\text{USKGen}(\text{sk}, \text{id})$ and all (sk, C) output by $\text{Enc}(\text{pk}, \hat{\text{id}})$ for $\hat{\text{id}} \in \hat{\text{ID}}$ such that $\hat{\text{id}} \preceq_* \text{id}$:

$$\Pr[\text{Dec}(\text{usk}[\text{id}], \text{id}, \text{C}) = \text{sk}] = 1.$$

We are now given a $\text{DIBKEM}(\text{Setup}, \text{USKGen}, \text{Enc}, \text{Dec}, \text{USKDown})$, let us show how to build the corresponding WIBKEM .

As with all the following constructions, the heart of the transformation will be to use a DIBKEM for identity of size 2ℓ to handle identities of size ℓ .

Let's consider an identity wid of size ℓ , we define $\text{id} = \phi(\text{wid})$ as follows:

$$\text{id}[2i, 2i + 1] = \begin{cases} 01 & \text{if } \text{wid}[i] = 0 \\ 10 & \text{if } \text{wid}[i] = 1 \\ 00 & \text{otherwise.} \end{cases}$$

Now we can define :

- $\text{WIBE.Setup}(\mathfrak{K}) : \text{Setup}(\mathfrak{K})$, except that instead of defining ID as strings of size 2ℓ , we suppose the public key define WID of enriched identities of size ℓ .
- $\text{WIBE.USKGen}(\text{sk}, \text{id}) = \text{USKGen}(\text{sk}, \phi(\text{id}))$.
- $\text{WIBE.Enc}(\text{pk}, \text{id}) = \text{Enc}(\text{pk}, \phi(\text{id}))$.
- $\text{WIBE.Dec}(\text{usk}[\text{id}], \hat{\text{id}}, \text{C})$ checks if $\hat{\text{id}} \preceq \text{id}$, then computes $\text{usk}[\phi(\hat{\text{id}})] = \text{USKDown}(\text{usk}[\phi(\text{id})])$. Returns $\text{Dec}(\text{usk}[\phi(\hat{\text{id}})], \hat{\text{id}}, \text{C})$ or rejects with $\perp$.

The WIBE constructed inherits his security from the DIBE . Due to the construction an adversary against the security of the WIBE will give an adversary against the security of the DIBE .

From DIBE to HIBE

Hierarchical Identity-Based Encryption is a concept introduced in [GS02]. The idea of this primitive is to introduce a hierarchy in the user secret key. A user can create a secret key from his one for any identity with prefix his own identity.

Instead of defining ID as strings of size 2ℓ needed for the transformation, we suppose the public key define HID of enriched identities of size ℓ .

We recall syntax and security of a hierarchical identity-based key encapsulation mechanism (HIBKEM).

Definition 20 (Hierarchical Identity-Based Key Encapsulation Mechanism). *A hierarchical identity-based key encapsulation mechanism HIBKEM consists of five PPT algorithms $\text{HIBKEM} = (\text{Setup}, \text{USKDel}, \text{USKGen}, \text{Enc}, \text{Dec})$ with the following properties.*

- The probabilistic key generation algorithm $\text{Setup}(\mathfrak{K})$ returns the (master) public/secret key and delegation key (pk, sk) . We assume that pk implicitly defines a message space $\mathcal{M}$ and hierarchical identity space $\text{ID} = \{0, 1\}^{\leq n}$.
- The probabilistic user secret key generation algorithm $\text{USKGen}(\text{sk}, \text{id})$ returns a secret key $\text{usk}[\text{id}]$ for hierarchical identity $\text{id} \in \text{ID}$.
- The probabilistic key delegation algorithm $\text{USKDel}(\text{usk}[\text{id}], \text{id} \in \{0, 1\}^p, \text{id}_{p+1} \in \{0, 1\}^{\leq n})$ returns a user secret key $\text{usk}[\text{id}|\text{id}_{p+1}]$ for the hierarchical identity $\text{id}' = \text{id} | \text{id}_{p+1} \in \{0, 1\}^{p+1}$. We require $1 \leq |\text{id}| \leq m - 1$.
- The probabilistic encapsulation algorithm $\text{Enc}(\text{pk}, \text{id})$ returns a symmetric key $K \in \mathcal{K}$ together with a ciphertext C with respect to the hierarchical identity $\text{id} \in \text{ID}$.
- The deterministic decapsulation algorithm $\text{Dec}(\text{usk}[\text{id}], \text{id}, C)$ returns a decapsulated key $K \in \mathcal{K}$ or $\perp$.

For correctness we require that for all $\mathfrak{K} \in \mathbb{N}$, all pairs (pk, sk) generated by $\text{Setup}(\mathfrak{K})$, all $\text{id} \in \text{ID}$, all $\text{usk}[\text{id}]$ generated by $\text{USKGen}(\text{sk}, \text{id})$ and all (sk, c) generated by $\text{Enc}(\text{pk}, \text{id})$:

$$\Pr[\text{Dec}(\text{usk}[\text{id}], \text{id}, C) = \text{sk}] = 1.$$

Moreover, we also require the distribution of $\text{usk}[\text{id}|\text{id}_{p+1}]$ from $\text{USKDel}(\text{usk}[\text{id}], \text{usk}[\text{id}], \text{id}, \text{id}_{p+1})$ to be identical to the one from $\text{USKGen}(\text{sk}, \text{id}|\text{id}_{p+1})$.

This time, we are going to map the identity space to a bigger set, with joker identity that can be downgraded to both 0 or 1.

Let's consider an identity hid of size ℓ , we define $\text{id} = \phi(\text{hid})$ as follows:

$$\text{id}[2i, 2i + 1] = \begin{cases} 01 & \text{if } \text{hid}[i] = 0 \\ 10 & \text{if } \text{hid}[i] = 1 \\ 11 & \text{otherwise}(\text{hid}[i] = \perp). \end{cases}$$

Now we can define :

- $\text{HIB.Setup}(\mathfrak{K}) : \text{Setup}(\mathfrak{K})$, except instead of defining ID define HID of enriched identities of size ℓ .

- $\text{HIB.USKGen}(\text{sk}, \text{id}) = \text{USKGen}(\text{sk}, \phi(\text{id}))$.⁴
- $\text{HIB.USKDel}(\text{usk}[\text{id}], \text{id} \in \{0, 1\}^p, \text{id}_{p+1} \in \{0, 1\}) = \text{USKDown}(\text{usk}[\phi(\text{id})], \phi(\text{id} \parallel \text{id}_{p+1}))$.
By construction we have $\phi(\text{id} \parallel \text{id}_{p+1}) \preceq \phi(\text{id})$.
- $\text{HIB.Enc}(\text{pk}, \text{id}) = \text{Enc}(\text{pk}, \phi(\text{id}))$.
- $\text{HIB.Dec}(\text{usk}[\text{id}], \text{id}, \text{C})$ returns $\text{Dec}(\text{usk}[\phi(\text{id})], \phi(\text{id}), \text{C})$ or the reject symbol $\perp$.

The HIBE constructed inherits his security from the DIBE. Due to the construction an adversary against the security of the HIBE will give an adversary against the security of the DIBE.

From DIBE to Wicked IBE The paper [AKN07] presents a variant of Identity-based Encryption called Wicked IBE (WKD-IBE). A wicked IBE or wildcard key derivation IBE is a generalization of the concept of limited delegation concept by Boneh-Boyen-Goh [BBG05].

This scheme allows secret key associated with a pattern $P = (P_1, \dots, P_l) \in \{\{0, 1\}^* \cup \{*\}\}^l$ to be delegated for a pattern $P' = (P'_1, \dots, P'_{l'})$ that matches P . We say that P' match P if $\forall i \leq l' P'_i = P_i$ or $P_i = *$ and $\forall l' + 1 \leq i \leq l P_i = *$.

Definition 21 (Wicked Identity-Based Key Encapsulation Mechanism). *A wicked identity-based key encapsulation mechanism WKDIB.IBKEM consists of five PPT algorithms WKDIB.HIBKEM = (WKDIB.Setup, WKDIB.USKDel, WKDIB.USKGen, WKDIB.Enc, WKDIB.Dec) with the following properties.*

- The probabilistic key generation algorithm $\text{WKDIB.Setup}(\mathfrak{K})$ returns the (master) public/secret key and delegation key (pk, sk) . We assume that pk implicitly defines a message space $\mathcal{M}$ and hierarchical identity space $\text{ID} = \{0, 1\}^{\leq n}$.
- The probabilistic user secret key generation algorithm $\text{WKDIB.USKGen}(\text{sk}, \text{id})$ returns a secret key $\text{usk}[\text{id}]$ for hierarchical identity $\text{id} \in \text{ID}$.
- The probabilistic key delegation algorithm $\text{WKDIB.USKDel}(\text{usk}[\text{id}], \text{id} \in \{0, 1, *\}^n, \text{id}' \in \{0, 1\}^n)$ returns a user secret key $\text{usk}[\text{id}']$ for the identity id' matching id ⁵.
- The probabilistic encapsulation algorithm $\text{WKDIB.Enc}(\text{pk}, \text{id})$ returns a symmetric key $K \in \mathcal{K}$ together with a ciphertext C with respect to the hierarchical identity $\text{id} \in \text{ID}$.
- The deterministic decapsulation algorithm $\text{WKDIB.Dec}(\text{usk}[\text{id}], \text{id}, \text{C})$ returns a decapsulated key $\text{sk} \in \mathcal{K}$ or $\perp$.

⁴It should be noted that in case of an HIBKEM, some identities are never to be queried to the downgradable IBKEM: those with 00 at $2i, 2i + 1$, or those with 11 at $2i, 2i + 1$ and then a 0 (this would correspond to *punctured* identities).

⁵same definition of matching than with WIBE

For correctness we require that for all $\mathfrak{K} \in \mathbb{N}$, all pairs (pk, sk) generated by $WKDIB.Setup(\mathfrak{K})$, all $id \in ID$, all $usk[id]$ generated by $WKDIB.USKGen(sk, id)$ and all (sk, c) generated by $WKDIB.Enc(pk, id)$:

$$\Pr[WKDIB.Dec(usk[id], id, C) = sk] = 1.$$

Moreover, we also require the distribution of $usk[id']$ from $USKDel(usk[id], udk[id], id, id_{p+1})$ to be identical to the one from $USKGen(sk, id|id_{p+1})$.

Here again, we are going to map the identity space to a bigger set.

Let us consider an identity id of size ℓ , we define $id = \phi(wkdid)$ as follows:

$$id[2i, 2i + 1] = \begin{cases} 01 & \text{if } wkdid[i] = 0 \\ 10 & \text{if } wkdid[i] = 1 \\ 11 & \text{if } wkdid[i] = * \end{cases}$$

Now we can define :

- $WKDIB.Setup(\mathfrak{K})$: $Setup(\mathfrak{K})$, except instead of defining ID as strings of size 2ℓ , we suppose the public key define $WKDID$ of enriched identities of size ℓ .
- $WKDIB.USKGen(sk, id) = USKGen(sk, \phi(id))$. It should be noted that in case of an $WKDIBE$, some identities are never to be queried to the downgradable IBE: those with 00.
- $WKDIB.USKDel(usk[id], id, id') = USKDown(usk[\phi(id)], \phi(id), \phi(id'))$.
- $WKDIB.Enc(pk, id) = Enc(pk, \phi(id))$.
- $WKDIB.Dec(usk[id], id, C)$ returns $Dec(usk[\phi(id)], \phi(id), C)$ or the reject symbol $\perp$.

The Wicked IBE constructed inherits his security from the DIBE. Due to the construction an adversary against the security of the Wicked IBE will give an adversary against the security of the DIBE.

From Wicked IBE to DIBE We can easily transform a Wicked IBE scheme into DIBE by using only identity made of 0 and *. In fact the element 1 of the DIBE plays the role of the * of the Wicked IBE. Morally a DIBE can be seen as a Wicked IBE where the patterns are made of only 2 distinct elements instead of 3. We will describe this scheme here since it is not relevant for our purpose. We choose to define a new primitive DIBE because the size of the alphabet for the identities is only 2 and it allows a better efficiency for all schemes based on DIBE. The wildcard * is hard to handle, the best way is how we do it with our DIBE, from that remark it is natural to use DIBE instead of Wicked IBE.

3.1.3 Instantiation and Proof of security

We will base instantiate our DIBE scheme on the IBE scheme from [BKP14]. In their scheme, the identity bits are used in a special way in the user secret

Setup(param): $\mathbf{B} \xleftarrow{\$} \mathcal{D}_k$ For $i = 0, \dots, \ell$: $z_i \xleftarrow{\$} \mathbb{Z}_p^{(k+1) \times n}; \mathbf{Z}_i = \mathbf{z}_i^\top \cdot \mathbf{A} \in \mathbb{Z}_p^{n \times k}$ $z' \xleftarrow{\$} \mathbb{Z}_p^{k+1}; \mathbf{Z}' = z'^\top \cdot \mathbf{A} \in \mathbb{Z}_p^{1 \times k}$ $\text{pk} := (\mathcal{G}, [\mathbf{A}]_1, ([\mathbf{Z}_i]_1)_{0 \leq i \leq \ell}, [\mathbf{Z}']_1)$ $\text{sk} := ((z_i)_{0 \leq i \leq \ell}, z')$ Return (pk, sk) USKGen(sk, id $\in$ ID): $t \xleftarrow{\$} \mathbb{Z}_p^n;$ $v = \sum_{i=0}^{l(\text{id})} f_i(\text{id}) \mathbf{z}_i t + z' \in \mathbb{Z}_p^{k+1}$ $\mathbf{S} \xleftarrow{\$} \mathbb{Z}_p^{n' \times \mu}; \mathbf{T} = \mathbf{B} \cdot \mathbf{S} \in \mathbb{Z}_p^{n \times \mu}$ $\mathbf{V} = \sum_{i=0}^{l(\text{id})} f_i(\text{id}) \mathbf{Z}_i \mathbf{T} \in \mathbb{Z}_p^{(k+1) \times \mu}$ For $i, \text{id}[i] = 1$: $e_i = \mathbf{Z}_i t \in \mathbb{Z}_p^{k+1}; E_i = \mathbf{Z}_i \mathbf{T} \in \mathbb{Z}_p^{k+1 \times \mu}$ $\text{usk}[\text{id}] := ([t]_2, [v]_2) \in \mathbb{G}_2^n \times \mathbb{G}_2^{k+1}$ $\text{udk}[\text{id}] := ([\mathbf{T}]_2, [\mathbf{V}]_2, ([e_i]_2, [E_i]_2)_{i, \text{id}[i]=1})$ $\in \mathbb{G}_2^{n \times \mu} \times \mathbb{G}_2^{(k+1) \times \mu} \times (\mathbb{G}_2^{k+1} \times \mathbb{G}_2^{(k+1) \times \mu})^{\text{Ham}(\text{id})}$ Return (usk[id], udk[id]) Enc(pk, id): $r \xleftarrow{\$} \mathbb{Z}_p^k$ $c_0 = \mathbf{A} r \in \mathbb{Z}_p^{k+1}$ $c_1 = (\sum_{i=0}^{l(\text{id})} f_i(\text{id}) \mathbf{Z}_i) \cdot r \in \mathbb{Z}_p^n$ $K = z'_0 \cdot r \in \mathbb{Z}_p$ Return sk = $[K]_T$ and C = $([c_0]_1, [c_1]_1)$	USKDown(usk[id], $\tilde{\text{id}}$): If $\neg(\tilde{\text{id}} \preceq \text{id})$, then return $\perp$ Set $\mathcal{I} = \{i \text{id}[i] = 0 \wedge \tilde{\text{id}}[i] = 1\}$ // Downgrading the key: $\hat{v} = v + \sum_{i \in \mathcal{I}} f_i(\text{id}') e_i \in \mathbb{Z}_p^k$ $\hat{V} = V + \sum_{i \in \mathcal{I}} f_i(\text{id}') E_i \in \mathbb{Z}_p^{k \times \mu}$ // Rerandomization of $(\hat{v}, \hat{V})$: $s' \xleftarrow{\$} \mathbb{Z}_p^\mu; S' \xleftarrow{\$} \mathbb{Z}_p^{\mu \times \mu}$ $t' = t + \mathbf{T} s' \in \mathbb{Z}_p^n;$ $T' = \hat{T} \cdot S' \in \mathbb{Z}_p^{n \times \mu}$ $v' = \hat{v} + \hat{V} \cdot s' \in \mathbb{Z}_p^k;$ $V' = \hat{V} \cdot S' \in \mathbb{Z}_p^{k \times \mu}$ // Rerandomization of e_i : For $i, \tilde{\text{id}}[i] = 1$: $e'_i = e_i + \mathbf{E}_i s' \in \mathbb{Z}_p^{k+1};$ $E'_i = E_i \cdot S' \in \mathbb{Z}_p^{(k+1) \times \mu}$ $\text{usk}[\text{id}'] := ([t']_2, [v']_2)$ $\text{udk}[\text{id}'] := ([T']_2, [V']_2, ([e'_i]_2, [E'_i]_2))$ Return (usk[id'], udk[id']) Dec(usk[id], id, C): Parse usk[id] = $([t]_2, [v]_2)$ Parse C = $([c_0]_1, [c_1]_1)$ $\text{sk} = e([c_0]_1, [v]_2) \cdot e([c_1]_1, [t]_2)^{-1}$ Return sk $\in \mathbb{G}_T$
--	---

FIGURE 3.3: A DOWNGRADABLE IBE BASED ON MDDH

key: a bits set to 0 does not express in the user secret key. This helps us to create a DIBE by creating the same use secret key and adding for each 1s, an element to eliminate this expression of the 1.

Remark 2. *This instantiation is based on a MAC himself based on HPS. There is a MAC from Naor-Reingold PRF which is tight secure but we did not manage to prove its security when transforming it into a downgradable scheme.*

Theorem 1. *Under the $\mathcal{D}_k$ -MDDH assumption, the scheme presented in figure 3.3 is PR-ID-CPA secure. For all adversaries $\mathcal{A}$ there exists an adversary $\mathcal{B}$ with $\mathbf{T}(\mathcal{A}) \approx \mathbf{T}(\mathcal{B})$ and $\text{Adv}_{\text{DIBKEM}, \mathcal{D}_k}^{\text{PR-ID-CPA}}(\mathcal{A}) \leq (\text{Adv}_{\mathcal{D}_k, \text{GGen}}(\mathcal{B}) + 2q_k(\text{Adv}_{\mathcal{D}_k, \text{GGen}}(\mathcal{B}) + 1/q))^6$.*

⁶We recall that q_k is the maximal number of query to the Eval oracle

<u>Initialize:</u> $\text{sk}_{\text{MAC}} = (\mathbf{B}, (x_i)_{0 \leq i \leq \ell}, x'_0) \xleftarrow{\\$} \text{Gen}_{\text{MAC}}(\text{par})$ Return $([\mathbf{B}]_2, ([x_i^\top \mathbf{B}]_2)_{0 \leq i \leq \ell})$ <u>Eval(m):</u> $\mathcal{Q}_{\mathcal{M}} = \mathcal{Q}_{\mathcal{M}} \cup \{\mathbf{m}\}$ $([t]_2, [u]_2) \xleftarrow{\\$} \text{Tag}(\text{sk}_{\text{MAC}}, \mathbf{m})$ For $i, \mathbf{m}_i = 1$: $d_i = x_i^\top t \in \mathbb{Z}_p$ Return $([t]_2, [u]_2, ([d_i]_2))$	<u>Chal(m*):</u> // one query $h \xleftarrow{\\$} \mathbb{Z}_p$ $h_0 = \sum f_i(\mathbf{m}_i^*) x_i \cdot h \in \mathbb{Z}_p^n$ $h_1 = x'_0 \cdot h \in \mathbb{Z}_p$ $h_1 \xleftarrow{\\$} \mathbb{Z}_p$ Return $([h]_1, [h_0]_1, [h_1]_T)$ <u>Finalize($\beta \in \{0, 1\}$):</u> Return $\beta \wedge (\mathbf{m}^* \not\subseteq \mathcal{Q}_{\mathcal{M}})$
---	---

FIGURE 3.4: GAMES $\text{DPR-CMA}_{\text{real}}$, AND $\text{DPR}_0\text{-CMA}_{\text{rand}}$ (BOXED) FOR DEFINING $\text{DPR}_0\text{-CMA}$ SECURITY.

Remark 3. This instantiation respect the formal definition of DIBKEM (definition 17). However for efficiency purpose one can remark that for realizing WIBE or ABE the user's secret keys does not need to be rerandomize during the delegation phase since it will not be used by another user. It introduce the concept of self-delegatable-only scheme. Thus we can avoid the heavy elements T, S, E of the user secret keys. We called this new scheme self-downgradable IBE.

In this proof we will make use of MAC the primitive and security's definitions can be found in Section 2.4.1.

The inner block is a downgradable MAC

Definition 22. An affine MAC over $\mathbb{Z}_p^n$ is downgradable, if the message space is $\mathcal{M} = \{0, 1\}^m$ for some finite base set $\{0, 1\}$, $f'_0(\mathbf{m}) = 1$, and there exists a public function $f : \mathcal{M} \rightarrow \{0, \dots, \ell\}$ such that for all $\mathbf{m}' \preceq \mathbf{m}$,

$$f_i(\mathbf{m}'_i) = \begin{cases} f_i(\mathbf{m}_i) & \text{if } \mathbf{m}_i = \mathbf{m}'_i \\ f_i(0) & \text{otherwise} \end{cases}.$$

Let MAC be a delegatable affine MAC over $\mathbb{Z}_p^n$ with message space $\mathcal{M} = \{0, 1\}^m$. To build a DIBE, we require a new notion denoted as $\text{DPR}_0\text{-CMA}$ security. It differs from the classical security in two ways. Firstly, additional values needed for DIBE downgrade process are provided to the adversary through the call to Initialize and Eval. Secondly, Chal always returns a real $\mathbf{h}_0$. (In fact, the additional values actually allow the adversary to distinguish real from random $\mathbf{h}_0$.)

Let $\mathcal{G} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, q, g_1, g_2, e)$ be an asymmetric pairing group in par . Consider the games from Figure 3.4.

Definition 23. A delegatable affine MAC over $\mathbb{Z}_p^n$ is $\text{DPR}_0\text{-CMA}$ -secure if for all PPT $\mathcal{A}$, $\text{Adv}_{\text{MAC}}^{\text{dpr}_0\text{-cma}}(\mathcal{A}) := \Pr[\text{DPR-CMA}_{\text{real}}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{DPR}_0\text{-CMA}_{\text{rand}}^{\mathcal{A}} \Rightarrow 1]$ is negligible.

<u>Gen_{MAC}(par):</u> $\mathbf{B} \xleftarrow{\\$} \mathcal{D}_k$ $x_0, \dots, x_l \xleftarrow{\\$} \mathbb{Z}_p^{k+1}; x'_0 \xleftarrow{\\$} \mathbb{Z}_p$ $\text{sk}_{\text{MAC}} = (\mathbf{B}, x_0, \dots, x_l, x'_0)$ Return sk_{MAC} <u>Tag(sk_{MAC}, m):</u> $s \xleftarrow{\\$} \mathbb{Z}_p^k, t = \mathbf{B}s$ $u = (x_0^\top + \sum_{i=1}^{ m } \mathbf{m}_i \cdot x_i^\top)t + x'_0 \in \mathbb{Z}_p$ For $i, \mathbf{m}_i = 1, d'_i = (-x_i)t$ Return $\tau = ([t]_2, [u]_2, [d]_2) \in \mathbb{G}_2^{k+1} \times \mathbb{G}_2 \times \mathbb{G}_2^{\text{Ham}(\mathbf{m})}$	<u>Down($\tau, \mathbf{m}, \mathbf{m}'$):</u> If $\mathbf{m}' \preceq \mathbf{m}$, $[u']_2 = [u + \sum_{i, \mathbf{m}'_i \neq \mathbf{m}_i} d_i]_2$ $\forall i, \mathbf{m}'_i = 1, [d_i]_2 = [d_i]_2$ Return $\tau' = ([t]_2, [u']_2, [d']_2) \in \mathbb{G}_2^{k+1} \times \mathbb{G}_2 \times \mathbb{G}_2^{\text{Ham}(\mathbf{m}')}$ <u>Ver(sk_{MAC}, $\tau, \mathbf{m}$):</u> If $u = (x_0^\top + \sum_{i=1}^{ m } \mathbf{m}_i \cdot x_i^\top)t + x'_0$ then return 1; Else return 0.
--	---

FIGURE 3.5: DOWNGRADABLE MAC FROM HPS [BKP14]

We explicit in Figure 3.5 the inner downgradable MAC we consider in our scheme. And then prove its security.

In this proof we will show that an adversary will be at some point against a standard affine MAC thus the security of the MAC we based our instantiation on, ensure the security of our Downgradable MAC. Intuitively, we will replace query by query the answer of the Eval oracle by pure randomness in $\mathbb{G}_2^{k+1} \times \mathbb{G}_2 \times \mathbb{G}_2^{\text{Ham}(\mathbf{m})}$. This proof is close from the proof of security of the affine MAC from HPS in [BKP14]. $\mathbf{G}_0$ is the real security game defined in 3.4. $\mathbf{G}_{1,i}$ the first $i - 1$ answer to the Eval oracle are random and the rest is answered as in the real game. We also need a game to switch from *game* _{$\mathbf{G}_{1,i}$} to the game $\mathbf{G}_{1,i+1}$. This new game will be called $\mathbf{G}'_{1,i}$. Here we will only describe how to come from $\mathbf{G}'_{i,1}$ to $\mathbf{G}_{i+1,1}$ since it is the only part that will differ from the proof in [BKP14].

Let $\mathbf{m}$ be the i -th query of the adversary, since $\mathbf{m}^* \not\preceq \mathbf{m}$ there exists a j such that $\mathbf{m}_j^* \neq \mathbf{m}_j$ and $f_j(\mathbf{m}_j^*) \neq f_j(0)$. In this configuration the adversary not more information about x_j than in a standard affine MAC. We can thus reuse the argument of the original proof: there is an information-theoretic argument to show that $u - x'_0$ is uniformly random. To simplify our proof we assume that the adversary $\mathcal{A}$ knows x'_0 and all x_l with $l \notin \{0, j\}$. He may also know $\mathbf{B}^\top x_0$ and $\mathbf{B}^\top x_j$. We will show that $\mathcal{A}$ is unable to guess x_j and x_0 , $\mathcal{A}$ has to solve the following matrix equation:

$$\begin{pmatrix} \mathbf{B}^\top x_0 \\ \mathbf{B}^\top \\ h_0 \\ u - x'_0 \end{pmatrix} = \underbrace{\begin{pmatrix} \mathbf{B}^\top & 0 \\ 0 & \mathbf{B}^\top \\ h \cdot \mathbf{I}_{k+1} & \mathbf{m}_j^* h \cdot \mathbf{I}_{k+1} \\ t^\top & \mathbf{m}_j t^\top \end{pmatrix}}_{\mathbf{M}} \cdot \begin{pmatrix} x_0 \\ x_j \end{pmatrix} \quad (3.1)$$

The $u - x'_0$ is linearly independent from the other rows: $t^\top$ is independent from $\mathbf{B}^\top$ because $t \notin \text{span}(\mathbf{B})$ with probability $(q-1)/q$, also $\mathbf{m}_j \neq \mathbf{m}_j^*$ which means

that this last row is independent from the rows $(h \cdot \mathbf{I}_{k+1} \quad \mathbf{m}_j^* h \cdot \mathbf{I}_{k+1})$. Thus this system of equations has not enough equations to be solved e.g. $\mathcal{A}$ can not distinguish between a random and u (except for a probability $1/q$).

Finally, we do all the other steps of the proof like in the original proof, and then we end up with the following lemma.

Lemma 2. *For all adversaries $\mathcal{A}$ there exists an adversary $\mathcal{B}$ with $\mathbf{T}(\mathcal{A}) \approx \mathbf{T}(\mathcal{B})$ and $\mathbf{Adv}_{\text{MAC}_{\text{HPS}}, \mathcal{D}_k}(\mathcal{B})^{\text{DPR}_0\text{-CMA}}(\mathcal{A}) \leq 2q_k(\mathbf{Adv}_{\mathcal{D}_k, \text{GGen}}(\mathcal{B}) + 1/q)$.*

Which leads to the security of the downgradable MAC.

Achieving Secure DIBE

We define the sequence of games $\text{G}_0\text{-G}_4$ as in Figure 3.6. Let $\mathcal{A}$ be an adversary against the PR-ID-CPA security of DIBKEM. G_0 is the real attack game.

<u>Initialize:</u> // Games $\text{G}_0\text{-G}_2$, $\boxed{\text{G}_3\text{-G}_4}$ $\mathcal{G} \xleftarrow{\\$} \text{GGen}(\mathcal{R}); \mathbf{A} \xleftarrow{\\$} \mathcal{D}_k$ $\text{sk}_{\text{MAC}} = (\mathbf{B}, x_0, \dots, x_\ell, x'_0) \xleftarrow{\\$} \text{Gen}_{\text{MAC}}(\mathcal{G})$ $\forall i \in \llbracket 0, \ell \rrbracket :$ $\mathbf{Y}_i \xleftarrow{\\$} \mathbb{Z}_p^{k \times n}; \mathbf{Z}_i = (\mathbf{Y}_i^\top \mid x_i) \cdot \mathbf{A} \in \mathbb{Z}_p^{n \times k}$ $d_{i,1} = z_i^\top \cdot \mathbf{B} \in \mathbb{Z}_p^k$ $d_{i,2-n} = z_i^\top \cdot \mathbf{B} \in \mathbb{Z}_p^{k \times n-1}$ $d_{i,2-n'} = (\bar{\mathbf{A}}^{-1})^\top (\mathbf{Z}_i^\top \mathbf{B} - \mathbf{A}^\top x_i \mathbf{B})$ $y'_0 \xleftarrow{\\$} \mathbb{Z}_p^k; z'_0 = (y'_0{}^\top \mid x'_0) \cdot \mathbf{A} \in \mathbb{Z}_p^{1 \times k}$ $\text{pk} := ([\mathbf{A}]_1, ([\mathbf{Z}_i]_1)_{0 \leq i \leq \ell}, [z'_0]_1)$ $\text{dk} := ([\mathbf{B}]_2, ([d_i]_2)_{0 \leq i \leq \ell})$ $\text{sk} := ((\mathbf{Z}_i)_{0 \leq i \leq \ell}, z'_0)$ Return (pk, dk) <u>USKGen(id):</u> // Games $\text{G}_0\text{-G}_2$, $\boxed{\text{G}_3\text{-G}_4}$ $\mathcal{Q}_{\text{ID}} = \mathcal{Q}_{\text{ID}} \cup \{\text{id}\}$ $([t]_2, [u]_2) \xleftarrow{\\$} \text{Tag}(\text{sk}_{\text{MAC}}, \text{id})$ $v = \sum_i f_i(\text{id}) \mathbf{Y}_i t + y'_0 \in \mathbb{Z}_p^k$ $v^\top = (t^\top \sum f_i(\text{id}) \mathbf{Z}_i + z'_0 - u \cdot \mathbf{A}) \cdot \bar{\mathbf{A}}^{-1}$ For $i, \text{id}[i] = 1$: $d_{i,1} = \mathbf{x}_i^\top t \in \mathbb{Z}_p$ $d_{i,2-n} = \mathbf{Y}_i t \in \mathbb{Z}_p^k;$ $d_{i,2-n}^\top = (t^\top \mathbf{Z}_i - d_{i,1} \mathbf{A}) \bar{\mathbf{A}}^{-1} \in \mathbb{Z}_p^{1 \times k}$ $\text{usk}[\text{id}] := ([t]_2, [u]_2, [v]_2) \in \mathbb{G}_2^n \times \mathbb{G}_2^1 \times \mathbb{G}_2^k$ $\text{udk}[\text{id}] := ([d_i]_2)_{\text{id}[i]=1} \in (\mathbb{G}_2^{1+k})^{(\text{Ham}(\text{id}))}$ Return (usk[id], udk[id])	<u>Enc(id*):</u> // Games G_0, $\boxed{\text{G}_1\text{-G}_2}$, $\boxed{\text{G}_2}$, $\boxed{\text{G}_3}$ $r \xleftarrow{\\$} \mathbb{Z}_p^k$ $c_0^* = \mathbf{A}r \in \mathbb{Z}_p^{k+1}$ $c_0^* \xleftarrow{\\$} \mathbb{Z}_p^{k+1}$ $h \xleftarrow{\\$} \mathbb{Z}_p; \bar{c}_0^* \xleftarrow{\\$} \mathbb{Z}_p^k;$ $\bar{c}_0^* := h + \mathbf{A} \cdot \bar{\mathbf{A}}^{-1} c_0^* \in \mathbb{Z}_p$ $c_1^* = (\sum_i f_i(\text{id}^*) \mathbf{Z}_i) r \in \mathbb{Z}_p^n$ $c_1^* = \sum_i f_i(\text{id}^*) (\mathbf{Y}_i^\top \mid x_i) c_0^* \in \mathbb{Z}_p^n$ $c_1^* = \sum_i f_i(\text{id}^*) (\mathbf{Z}_i \cdot \bar{\mathbf{A}}^{-1} \bar{c}_0^* + x_i \cdot h)$ $K^* = z'_0 \cdot r \in \mathbb{Z}_p.$ $K^* = (y'_0{}^\top \mid x'_0) c_0^* \in \mathbb{Z}_p$ $K^* = z'_0 \cdot \bar{\mathbf{A}}^{-1} \bar{c}_0^* + x'_0 \cdot h$ Return $\mathbf{K}^* = [K^*]_T$ and $\mathbf{C}^* = ([c_0^*]_1, [c_1^*]_1)$ <u>Enc(id*):</u> // Game G_3, $\boxed{\text{G}_4}$ $h \xleftarrow{\\$} \mathbb{Z}_p; \bar{c}_0^* \xleftarrow{\\$} \mathbb{Z}_p^k; \bar{c}_0^* := h + \mathbf{A} \cdot \bar{\mathbf{A}}^{-1} c_0^* \in \mathbb{Z}_p$ $c_1^* = \sum_i f_i(\text{id}^*) (\mathbf{Z}_i \cdot \bar{\mathbf{A}}^{-1} \bar{c}_0^* + x_i \cdot h)$ $K^* = z'_0 \cdot \bar{\mathbf{A}}^{-1} \bar{c}_0^* + x'_0 \cdot h$ $K^* \xleftarrow{\\$} \mathbb{Z}_p$ Return $\mathbf{K}^* = [K^*]_T$ and $\mathbf{C}^* = ([c_0^*]_1, [c_1^*]_1)$ <u>Finalize(β):</u> // Games $\text{G}_0\text{-G}_4$ Return $(\text{id}^* \not\in \mathcal{Q}_{\text{ID}}) \wedge \beta$
--	--

FIGURE 3.6: GAMES $\text{G}_0\text{-G}_4$ FOR THE PROOF

We can see that G_1 is simply a rewriting of G_0 .

Lemma 3. $\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \Pr[G_0^{\mathcal{A}} \Rightarrow 1]$.

Lemma 4. *There exists an adversary $\mathcal{B}_1$ with $\mathbf{T}(\mathcal{B}_1) \approx \mathbf{T}(\mathcal{A})$ and*

$$\text{Adv}_{\mathcal{D}_k, \text{GGen}}(\mathcal{B}_1) \geq |\Pr[G_2^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]|.$$

Lemma 5. $\Pr[G_3^{\mathcal{A}} \Rightarrow 1] = \Pr[G_2^{\mathcal{A}} \Rightarrow 1]$.

Proof. G_3 is simulated without using y'_0 and $(Y_i)_{0 \leq i \leq \ell}$. By $\mathbf{Y}_i^\top = (\mathbf{Z}_i - x_i \mathbf{A}) \bar{\mathbf{A}}^{-1}$, we have

$$\begin{aligned} \mathbf{D}_i &= (\bar{\mathbf{A}}^{-1})^\top (\mathbf{Z}_i^\top \mathbf{B} - \mathbf{A}^\top d_i) = \underbrace{(\bar{\mathbf{A}}^{-1})^\top (\mathbf{Z}_i^\top - \mathbf{A}^\top x_i)}_{\mathbf{Y}_i} \mathbf{B} \\ d_i &= (\bar{\mathbf{A}}^{-1})^\top \cdot (\mathbf{Z}_i^\top t - \mathbf{A}^\top \underbrace{x_i^\top t}_{d_i}) = \mathbf{Y}_i t. \end{aligned}$$

as in Game G_2 . And so, we have $[v]_2$, K^* and C^* are identical to G_2 . □

Lemma 6. *There exists an adversary $\mathcal{B}_2$ with $\mathbf{T}(\mathcal{B}_2) \approx \mathbf{T}(\mathcal{A})$ and*

$$\text{Adv}_{\text{MAC}}^{\text{dpr}_0\text{-cma}}(\mathcal{B}_2) \geq |\Pr[G_4^{\mathcal{A}} \Rightarrow 1] - \Pr[G_3^{\mathcal{A}} \Rightarrow 1]|$$

Proof. In G_4 , we answer the $\text{Enc}(\text{id}^*)$ query by choosing random K^* . We construct algorithm $\mathcal{B}_2$ in Figure 3.7 to show the differences between G_4 and G_3 is bounded by the advantage of breaking $\text{dpr}_0\text{-cma}$ security of MAC.

We note that, in games G_3 and G_4 , the values x_i and x'_i are hidden until the call to $\text{Enc}(\text{id}^*)$ (because the adversary is not allowed to query an id such that $\text{id}^* \preceq \text{id}$). In both games $\text{DPR-CMA}_{\text{real}}$ and $\text{DPR}_0\text{-CMA}_{\text{rand}}$, we have $h = \bar{c}_0^* - \mathbf{A} \bar{\mathbf{A}}^{-1} \bar{c}_0^*$. Hence $h_0 = \sum f_i(m_i) x_i \cdot (\bar{c}_0^* - \mathbf{A} \bar{\mathbf{A}}^{-1} \bar{c}_0^*)$ which implies c_1^* is distributed identically in games G_3 and G_4 . If h_1 is uniform (i.e., $\mathcal{B}_2$ is in Game $\text{DPR}_0\text{-CMA}_{\text{rand}}$) then the view of $\mathcal{A}$ is the same as in G_4 . If h_1 is real (i.e., $\mathcal{B}_2$ is in Game $\text{DPR-CMA}_{\text{real}}$) then $K^* = z'_0 \cdot \bar{\mathbf{A}}^{-1} \bar{c}_0^* + x'_0 \cdot h$, which means the view of $\mathcal{A}$ is the same as in G_3 . □

The proof follows by combining Lemmas 3-6.

In this section we compare the schemes obtained by using our instantiation of DIBE and our new schemes obtained by our transformations and our DIBE. We end up with the most efficient scheme for full security in the standard model and under classical hypothesis for WIBE, WKD-IBE and of similar efficiency for HIBE.

To compare efficiency in a simple way, we choose to consider the case where the number of patterns is maximal e.g. the size of pattern is equal to 1, thus the number of patterns is n which is the length of the identity. The value

<u>Initialize:</u> $\mathbf{A} \xleftarrow{\\$} \mathcal{D}_k$ $([\mathbf{B}]_2, ([x_i^\top \mathbf{B}]_2)_{0 \leq i \leq \ell}) \xleftarrow{\\$} \text{Initialize}_{\text{MAC}}$ $\forall i \in \llbracket 0, \ell \rrbracket$: $\mathbf{Z}_i \xleftarrow{\\$} \mathbb{Z}_p^{n \times k}; z'_0 \xleftarrow{\\$} \mathbb{Z}_p^{1 \times k}$ $\text{pk} := (\mathcal{G}, [\mathbf{A}]_1, ([\mathbf{Z}_i]_1)_{0 \leq i \leq \ell}, [z'_0]_1)$ Return (pk, dk) <u>Enc(id*):</u> //only one query $([h]_1, [h_0]_1, [h_1]_T) \xleftarrow{\\$} \text{Chal}(\text{id}^*)$ $\bar{c}_0^* \xleftarrow{\\$} \mathbb{Z}_p^k; \underline{c}_0^* := h + \underline{\mathbf{A}} \cdot \bar{\mathbf{A}}^{-1} \bar{c}_0^* \in \mathbb{Z}_p$ $c_1^* = \sum_i f_i(\text{id}^*) \mathbf{Z}_i \cdot \bar{\mathbf{A}}^{-1} \bar{c}_0^* + h_0$ $K^* = z'_0 \cdot \bar{\mathbf{A}}^{-1} \bar{c}_0^* + h_1$ Return $K^* = [K^*]_T$ and $\mathbf{C}^* = ([c_0^*]_1, [c_1^*]_1)$	<u>USKGen(id):</u> $\mathcal{Q}_{\text{ID}} = \mathcal{Q}_{\text{ID}} \cup \{\text{id}\}$ $([t]_2, [u]_2, [T]_2, [u]_2, ([d_i]_2, [D_i]_2)) \xleftarrow{\\$} \text{Eval}(\text{id})$ $v^\top = (t^\top \sum f_i(\text{id}) \mathbf{Z}_i + z'_0 - u \cdot \underline{\mathbf{A}}) \cdot (\bar{\mathbf{A}})^{-1}$ $V = (\bar{\mathbf{A}}^{-1})^\top (\sum f_i(\text{id}) \mathbf{Z}_i^\top \cdot \mathbf{T} - \underline{\mathbf{A}}^\top \cdot u)$ For $i, \text{id}_i = 1$: $e_i^\top = (t^\top \mathbf{Z}_i - d_i \underline{\mathbf{A}}) \bar{\mathbf{A}}^{-1} \in \mathbb{Z}_p^{1 \times k}$ $E_i = (\bar{\mathbf{A}}^{-1})^\top (\mathbf{Z}_i^\top \mathbf{T} - \underline{\mathbf{A}}^\top \cdot D_i) \in \mathbb{Z}_p^{k \times \mu}$ $\text{usk}[\text{id}] := ([t]_2, [u]_2, [v]_2) \in \mathbb{G}_2^n \times \mathbb{G}_2^1 \times \mathbb{G}_2^k$ $\text{udk}[\text{id}] := ([T]_2, [u]_2, [\mathbf{V}]_2, [e_i]_2, [E_i]_2)$ Return (usk[id], udk[id]) <u>Finalize(β):</u> Return $(\text{id}^* \not\in \mathcal{Q}_{\text{ID}}) \wedge \text{Finalize}_{\text{MAC}}(\beta)$
--	---

FIGURE 3.7: DESCRIPTION OF $\mathcal{B}_2$ (HAVING ACCESS TO THE ORACLES $\text{Initialize}_{\text{MAC}}$, Eval , Chal , $\text{Finalize}_{\text{MAC}}$ FOR THE PROOF OF LEMMA 6.

Name	pk	usk	C	assump.	Sec	Loss
WKD [AKN07]	$n + 4$	$n + 2$	2	BDDH	Sel. standard	$O(Lq_k)$
WKD [AKN07]	$(n + 1)n + 3$	$n + 2$	2	BDDH	Full standard	$O(q_k^L)$
WKD (via our DIBE)	$4n + 2$	$3n + 5$	5	DLin (any $k - \text{MDDH}$)	Full standard	$O(q_k)$
WIBE [BDNS07]	$(n + 1)n + 3$	$n + 1$	$(n + 1)n + 2$	BDDH	Full standard	$O(L^2 q_k^L)$
WIBE (via our DIBE)	$4n + 2$	$3n + 5$	5	DLin (any $k - \text{MDDH}$)	Full standard	$O(q_k)$

FIGURE 3.8: EFFICIENCY COMPARISON BETWEEN OUR TRANSFORMATIONS AND PREVIOUS SCHEMES

Name	$ \text{pk} $	$ \text{usk} $	$ C $	assump.	Loss
HIBE [BBG05]	$n + 4$	$2 + \ell$	5	DLin	sel. $O(n \cdot q_k)$
HIBE [BKP14]	$2n + 1$	$11\ell + 5$	5	DLin (any $k - \text{MDDH}$)	$O(n)$
HIBE (via our DIBE)	$4n + 2$	$11n + 5$	5	DLin (any $k - \text{MDDH}$)	$O(q_k)$

FIGURE 3.9: EFFICIENCY COMPARISON BETWEEN OUR TRANSFORMATIONS AND HIBE SCHEMES

q_k corresponds to the number of derivation key oracle requests made by the adversary⁷.

Efficiency comparison for HIBE The Figure 3.9 compares the HIBE built via our DIBE. Our instantiation of DIBE inherits its efficiency from the HIBE from [BKP14], except we need to artificially double the size of the identities. Here ℓ is the number of free bits in an identity (the ones to delegate). Note that for the case of root of the hierarchy e.g. the user with an empty bit string as identity, $\ell = n$.

It should be noted, that while we rely on the same underlying principle, our scheme is not tightly secure and does not rely on the Naor-Reingold PRF, which allows to circumvent the worrisome parts of their proofs⁸.

3.2 Blind IBE and Fragmented IBE

In our idea of mixing IBE and SPHF we found another application: efficient Blind fragmented IBE⁹ instantiation which will allow to create an efficient adaptive Oblivious Transfer.

3.2.1 Constructing a Blind Fragmented IBKEM from an IBKEM

Shortly speaking a Blind IBKEM is an IBKEM with the possibility to get a user secret key without the authority knowing which identity is related to the request. This feature is in fact very interesting when using IBE as a tool. We will see that in the next chapter 4.3.2.

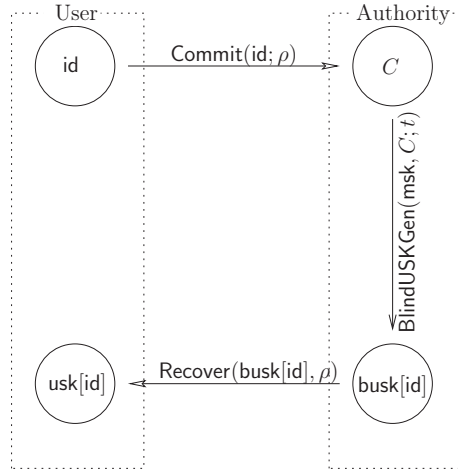
High-Level Idea: achieving Blindness We now show how to obtain a BlindIBKEM scheme from any IBKEM scheme. From a high-level point of view, this transformation mixes two pre-existing approaches.

⁷In the original version of [AKN07] they include an element in the ciphertext to turn their scheme into an encryption scheme. Since our scheme is a Key Encapsulation Mechanism we remove this element when comparing both schemes.

⁸There is problem in the proof which has not been patched at that moment

⁹see Definition 12

First, we are going to consider a reverse Naor transform [BF01, CFH⁺07] which has been described in Section 2.5. At this point we are seeing IBE keys as signature and we will use the Fischlin [Fis06] round-optimal approach to create blind signatures. The whole interaction is done in one pass: First, the user commits to the message, then he recovers a signature linked to his commitment. For sake of simplicity, instead of using a Non-Interactive Zero-Knowledge Proof of Knowledge of a signature, we are going to follow the [BFPV11, BPV12] approach, where thanks to an additional term, the user can extract a signature on the identity from a signature on the committed identity. This two steps transformation is depicted in Figure 3.10, details will follow in the next paragraph.



1. A user commits to the targeted identity id using some randomness ρ .
2. The authority possesses an algorithm allowing it to generate keys for committed identities using its master secret key msk , and some randomness t , in order to obtain a blinded user secret key $busk[id]$.
3. The user using solely the randomness used in the initial commitment is able to recover the requested secret key from the authority's generated value.

FIGURE 3.10: GENERIC TRANSFORMATION OF AN IBKEM INTO A BLIND IBKEM (NAIVE APPROACH)

Generic Transformation of an IBKEM into a Blind IBKEM. To blind a user key request we will use a Smooth Projective Hash Function. We assume the existence of a labeled CCA-encryption scheme

$$\mathcal{E} = (\text{Setup}_{cca}, \text{KeyGen}_{cca}, \text{Encrypt}_{cca}^{\ell}, \text{Decrypt}_{cca}^{\ell})$$

with an SPHF over a language $\mathcal{L}_{id}^c \subset X$, where $\mathcal{L}_{id}^c = \{C \mid \exists \rho \text{ such that } C = \text{Encrypt}_{cca}^{\ell}(id; \rho)\}$, defined by $(\text{Setup}, \text{HashKG}, \text{ProjKG}, \text{Hash}, \text{ProjHash})$ onto a set G (where ℓ is a label defined by the global protocol). The formal description of our BlindUSKGen is described in Figure 3.11. Note that we need a key derivation function¹⁰ KDF to get a long enough bit string to hide the user secret key.

Remark 4. *In this transformation the authority sends every secret key but only one can be recovered by the user: the one requested. This comes directly from the property of the SPHF and this is why we make use of this primitive. This*

¹⁰One can use the Leftover-Hash Lemma [HILL99], with a random seed defined in `param` during the global setup, to extract the entropy from v , then followed by a pseudo-random generator to get a long enough bit-string.

- The user computes an encryption of the expected identity id and keeps the randomness ρ : $C = \text{Encrypt}_{\text{cca}}^\ell(\text{id}; \rho)$.
- For every identity id' , the server computes the key $\text{usk}[\text{id}']$ along with a pair of (secret, public) hash keys $(\text{hk}_{\text{id}'}, \text{hp}_{\text{id}'})$ for a smooth projective hash function on the language $\mathcal{L}_{\text{id}'}^c$:
 $\text{hk}_{\text{id}'} = \text{HashKG}(\ell, \mathcal{L}_{\text{id}'}^c, \text{param})$ and $\text{hp}_{\text{id}'} = \text{ProjKG}(\text{hk}_{\text{id}'}, \ell, (\mathcal{L}_{\text{id}'}^c, \text{param}), \text{id}')$.
He also compute the corresponding hash value
 $H_{\text{id}'} = \text{Hash}(\text{hk}_{\text{id}'}, (\mathcal{L}_{\text{id}'}^c, \text{param}), (\ell, C))$.
Finally, he sends $(\text{hp}_{\text{id}'}, \text{usk}[\text{id}'] \oplus \text{KDF}(H_{\text{id}'}))$ for every id' , where $\oplus$ is a compatible operation.
- Thanks to hp_{id} , the user is able to compute the corresponding projected hash value $H'_{\text{id}} = \text{ProjHash}(\text{hp}_{\text{id}}, (\mathcal{L}_{\text{id}}^c, \text{param}), (\ell, C), \rho)$. He then recovers $\text{usk}[\text{id}]$ for the initially committed identity id since $H_{\text{id}} = H'_{\text{id}}$.

FIGURE 3.11: GENERIC CONSTRUCTION OF
 $\text{BlindUSKGen}((\mathcal{S}, \text{msk})(U, \text{id}, \ell; \rho))$ FOR A BLIND IBE

idea of many receiving/one understandable will be reused in this Thesis. We will generalize this with the concept of OLBE developed in the Section 5.1.

Theorem 2. *If IBKEM is a PR-ID-CPA-secure identity-based key encapsulation scheme and $\mathcal{E}$ a labeled CCA-encryption scheme compatible with an SPHF, then BlindIBKEM is leak free and weak blind.*

Proof. First, BlindIBKEM satisfies leak-free secret key generation since it relies on the CCA security on the encryption scheme, forbidding a user to open it to another identity than the one initially encrypted. Furthermore, the pseudo-randomness of the SPHF ensures that the blinded user key received for id is indistinguishable from random if he encrypted $\text{id}' \neq \text{id}$. Finally, the weak blindness also relies on the CCA security on the encryption scheme, since an encryption of id is indistinguishable from a encryption of $\text{id}' \neq \text{id}$. $\square$

Generic Transformation of a Fragmented IBKEM into a Blind Fragmented IBKEM. Now we go a step further and have a look at Fragmented IBE

¹¹. In a Fragmented IBE a user key is made of pieces from the list $(\text{usk}[0], \text{usk}[0, \text{id}_0], \dots, \text{usk}[m-1, \text{id}_{m-1}])$ if id_i is the i -th bit of the identity id (for example $\text{usk}[\text{id}] = \text{usk}[0] \cdot \prod_{i=0}^n \text{usk}[i, \text{id}_i]$). For Fragmented IBE we are able to improve the efficiency of the Blind Fragmented IBKEM transformation: instead of having a response of the authority linear in the number of user's secret key we will be linear in the number of pieces of key. This new communication cost is logarithmic in the number of user's secret key. We describe this transformation in figure 3.12.

For security reasons, one cannot give directly the evaluation value, but as we are considering the sum of the evaluations for each bit, we simply add a Shamir-like secret sharing, by adding randomness that is going to cancel out at the end.

¹¹They were defined in [BKP14]. Affine IBE derive their name from the fact that only affine operations are done on the identity bits (no hashing, square rooting, inverting... are allowed).

- The user computes a bit-per-bit encryption of the requested line identity id and keeps the randomness ρ : $\{C = \text{Encrypt}_{\text{cca}}^\ell(\text{id}; \rho)\}$.
- The server computes a fragmented version of all the keys $\text{usk}[\text{id}']$, *i.e.* all the values $\text{usk}[i, b]$ for i from 0 up to the length m of the keys and $b \in \{0, 1\}$. He also computes a pair of (secret, public) hash keys $(\text{hk}_{i,b}, \text{hp}_{i,b})$ for a smooth projective hash function on the language $\mathcal{L}_{i,b}^c$: “The i -th bit of the value encrypted into C is b ”, *i.e.* $\text{hk}_{i,b} = \text{HashKG}(\ell, \mathcal{L}_{i,b}^c, \text{param})$ and $\text{hp}_{i,b} = \text{ProjKG}(\text{hk}_{i,b}, \ell, (\mathcal{L}_{i,b}^c, \text{param}))$. He also computes the corresponding hash value $H_{i,b} = \text{Hash}(\text{hk}_{i,b}, (\mathcal{L}_{i,b}^c, \text{param}), (\ell, C))$ and chooses random values z_i . Finally, he sends, for each (i, b) , $(\text{hp}_{i,b}, \text{busk}[i, b])$, where $\text{busk}[i, b] = \text{usk}[i, b] \oplus \text{KDF}(H_{i,b}) \oplus z_i$, together with $Z = \text{usk}_0 \ominus \left(\bigoplus_i z_i \right)$, where $\oplus$ is a compatible operation and $\ominus$ its inverse.
- Thanks to the $\text{hp}_{i, \text{id}_i}$ for the initially committed identity id , the user is able to compute the corresponding projected hash value
$$H'_{i, \text{id}_i} = \text{ProjHash}(\text{hp}_{i, \text{id}_i}, (\mathcal{L}_{i, \text{id}_i}^c, \text{param}), (\ell, C), \rho),$$
that should be equal to H_{i, id_i} for all i . From the values $\text{busk}[i, \text{id}_i]$, he then recovers $\text{usk}[i, \text{id}_i] \oplus z_i$. Finally, with the operation
$$\left(\bigoplus_i (\text{usk}[i, \text{id}_i] \oplus z_i) \right) \oplus Z,$$
he recovers the expected $\text{usk}[\text{id}]$.

FIGURE 3.12: GENERIC CONSTRUCTION OF $\text{BlindUSKGen}(\langle (\mathcal{S}, \text{msk})(U, \text{id}, \ell; \rho) \rangle)$ FROM A BLIND AFFINE IBE

3.2.2 Pairing-Based Instantiation

Affine Bit-Wise Blind IBE. We are going to use the family of IBE from [BKP14] described in the following picture (Figure 3.13), which is their instantiation derived from a Naor-Reingold MAC¹². In the following, $h_i()$ are injective deterministic public functions mapping a bit to a scalar in $\mathbb{Z}_p$.

We propose a blind user key generation in Figure 3.14.

To fit the framework we will use in the next chapter, we are going to consider the equivocable language of each chameleon hash of the identity bits (a_i, b_{i, m_i}) , and then a Cramer-Shoup like encryption of b into d (more details in Figure 2.5). We denote this process as **Har** in the following protocol, and by $\mathcal{L}_{\text{Har}, i, \text{id}_i}$ the language on identity bits. And then we will apply the SPHF described in figure 2.14. We thus obtain the following security results.

Theorem 3. *This construction achieves both the weak Blindness, and the leak-free secret key generation requirements under the $k - \text{MDDH}$ assumption.*

Proof. Given an adversary $\mathcal{A}$ against the weak Blindness security or the leak-free secret key generation of the blinded scheme, we are going to build an adversary

¹²For the reader familiar with the original result, we combine x, y into a bigger y to lighten the notations, and compact the (x'_i, y'_i) values into a single y' as this has no impact on their construction.

<u>Setup($\mathcal{R}$):</u> $A \xleftarrow{\\$} \mathcal{D}_k, B = \overline{A}$ For $i \in \llbracket 0, \ell \rrbracket : Y_i \xleftarrow{\\$} \mathbb{Z}_p^{k+1}; Z_i = Y_i^\top \cdot A \in \mathbb{Z}_p^k$ $y' \xleftarrow{\\$} \mathbb{Z}_p^{k+1}; z' = y'^\top \cdot A \in \mathbb{Z}_p^k$ $\text{mpk} := (\mathcal{G}, [A]_1, ([Z_i]_1)_{i \in \llbracket 0, \ell \rrbracket}, [z']_1)$ $\text{msk} := (Y_i)_{i \in \llbracket 0, \ell \rrbracket}, y'$ Return (mpk, msk) <u>USKGen(msk, id):</u> $s \xleftarrow{\\$} \mathbb{Z}_p^k, t = Bs$ $w = (Y_0 + \sum_{i=1}^\ell h_i(\text{id}_i)Y_i)t + y' \in \mathbb{Z}_p^{k+1}$ Return $\text{usk}[\text{id}] := ([t]_2, [w]_2) \in \mathbb{G}_2^{k+k+1}$	<u>Enc(mpk, id):</u> $r \xleftarrow{\\$} \mathbb{Z}_p^k$ $c_0 = Ar \in \mathbb{Z}_p^{k+1}$ $c_1 = (Z_0 + \sum_{i=1}^\ell h_i(\text{id}_i)Z_i) \cdot r \in \mathbb{Z}_p$ $K = z' \cdot r \in \mathbb{Z}_p$ Return $[K]_T$ and $C = ([c_0]_1, [c_1]_1) \in \mathbb{G}_1^{k+1+1}$ <u>Dec(usk[id], id, C):</u> Parse $\text{usk}[\text{id}] = ([t]_2, [w]_2)$ Parse $C = ([c_0]_1, [c_1]_1)$ $K = e([c_0]_1, [w]_2) \cdot e([c_1]_1, [t]_2)^{-1}$ Return $K \in \mathbb{G}_T$
---	---

FIGURE 3.13: A FRAGMENTED IBKEM [BKP14].

- First flow: $\mathcal{U}$ starts by computing $\rho \xleftarrow{\\$} \mathbb{Z}_p^{1+4 \times \ell}$, $a, d = \text{Har}(\text{id}, \ell; \rho) \in \mathbb{Z}_p^\ell \times \mathbb{Z}_p^{2 \times (k+3)\ell}$, Sends $C = ([a]_1, [d]_2)$ to $\mathcal{S}$ - Second Flow: $\mathcal{S}$ then proceeds $s \xleftarrow{\\$} \mathbb{Z}_p^k, t = Bs, f \xleftarrow{\\$} \mathbb{Z}_p^{\ell \times k+1}$, For each $i \in \llbracket 1, \lceil \log n \rceil \rrbracket, b \in \llbracket 0, 1 \rrbracket$: $\text{hk}_{i,b} = \text{HashKG}(\mathfrak{L}_{\text{Har}, i, b}, C)$ $\text{hp}_{i,b} = \text{ProjKG}(\text{hk}_{i,b}, \mathfrak{L}_{\text{Har}, i, b}, C)$ $H_{i,b} = \text{Hash}(\text{hk}_{i,b}, \mathfrak{L}_{\text{Har}, i, b}, C)$ $\omega_{i,b} = (bY_i)t + f_i + H_{i,b}$	Then sets $w_0 = Y_0t + y' - \sum_{i=1}^\ell f_i \in \mathbb{Z}_p^{k+1}$ Returns $\text{busk} := ([t]_2, [w_0]_2, \{[\omega_{i,b}]_2\}, \{[\text{hp}_{i,b}]_2\})$ BlindUSKGen₃: $\mathcal{U}$ then recovers his key For each $i \in \llbracket 1, \ell \rrbracket$: $H'_i = \text{ProjHash}(\text{hp}_{i, \text{id}_i}, \mathfrak{L}_{\text{Har}, i, \text{id}_i}, C, \rho_i)$ $w_i = \omega_{i, \text{id}_i} - H'_i$ $w = w_0 + \sum_{i=1}^\ell w_i$ And then recovers $\text{usk}[\text{id}] := [t]_2, [w]_2$
--	--

FIGURE 3.14: BlindUSKGen($((\mathcal{S}, \text{msk})(\mathcal{U}, \text{id}, \ell; \rho))$).

$\mathcal{B}$ against the PR-ID-CPA security of the initial IBE.

From the challenge, $\mathcal{B}$ receives the parameter from an IBE scheme mpk , has access to a user key generation oracle $\text{USKGen}_{\mathcal{O}}$, and for a given fresh id^* , a tuple $(\mathbf{K}^*, \mathbf{C}^*)$ whose consistency he needs to decide.

In the initial game $\mathcal{G}_0$, $\mathcal{B}$ behaves normally, generating mpk , msk , and answering $\mathcal{A}$ blinded request honestly.

$\mathcal{G}_1$ In this game, $\mathcal{B}$ using the extraction procedure on the CCA commitment, is able to recover the identity id queried by $\mathcal{A}$. If for a bit i , there is two valid openings, then $\mathcal{A}$ broke the collision resistance property of the underlying chameleon hash (and so MDDH) and $\mathcal{B}$ aborts. Thus this game is equivalent to the previous one under $k - \text{MDDH}$.

$\mathcal{G}_2$ In this game, $\mathcal{B}$ starts altering his answer. It is able to recover the identity id queried by $\mathcal{A}$ with the extraction procedure. Now, for each bit of identity there is one binary value not committed $\bar{\text{id}}_i$. For these bits $\mathcal{B}$'s answer will be modified and sets to a random values $\omega_{i, \bar{\text{id}}_i} \xleftarrow{\$} \mathbb{Z}_p^{k+1}$. Under the smoothness of the underlying smooth projective hash function, this game is indistinguishable from the previous one.

$\mathcal{G}_3$ Now $\mathcal{B}$ continues to extract the requested id , picks random $f \xleftarrow{\$} \mathbb{Z}_p^{\ell \times k+1}$, and sets $w_0 = \text{usk}[\text{id}] - \sum_{i=1}^{\ell} f_i$, then for $i \in \llbracket 1, n \rrbracket$, $\omega_{i, \text{id}_i} = f_i + H_{i, \text{id}_i}$ while $\omega_{i, \bar{\text{id}}_i} \xleftarrow{\$} \mathbb{Z}_p^{k+1}$ as before. If $\mathcal{B}$, does not recover a valid identity, he simply only sends dummy values.

This game is indistinguishable from the previous one, as this is just a rewriting of the vector f_i . (Noting $\hat{f}_i$ the old one, we have $f_i = \hat{f}_i - (\text{id}_i Y_i)t$ which follows the same distribution).

$\mathcal{G}_4$ $\mathcal{B}$ can now forget about msk , since the user secret key will be whole and hidden in w_0 . When receiving a BlindUSKGen request, $\mathcal{B}$ extracts the request identity, and if it is not $\perp$ he forwards it to the $\text{USKGen}_{\mathcal{O}}$ oracle, and plugs the received $\text{usk}[\text{id}]$ as before.

Now $\mathcal{A}$ can request a challenge from $\mathcal{B}$, $\mathcal{B}$ forwards this request to the initial non-blinded IBE challenge for a fresh id^* , and returns the challenge $(\mathbf{K}^*, \mathbf{C}^*)$ to $\mathcal{A}$ which leads to the conclusion. $\square$

For sake of generality, any bit-wise affine IBE could work (like for example Waters IBE [Wat05]). We preferred here a tightly secure IBE giving an even more practical scheme.

Chapter 4

Applications of Identity-based Encryption

In this chapter we will see how Identity-based encryption can be used in different ways. We have already seen that IBE can be transformed into a signature. By adding one more layer of IBE (using HIBE of depth 2) we are able to create Identity-based Signature. This is a known construction [CFH⁺07] but we will use it in the particular context of constructing Identity-based Designated Verifier Signature (along with other primitives). In a second part we will use our new primitive namely Downgradable Identity-based Encryption and achieve what this primitive has been made for: construct an Attribute-based Encryption scheme from a variant of Identity-based Encryption. At the end of this chapter we describe a construction of adaptive Oblivious Transfer. This technique has already been used but we managed to highly improve the communication cost of this result, achieving a generic construction from any fragmented IBE. We end up with a very efficient and practical adaptive OT (which was not done before).

4.1 Identity Based Designated Verifier Signature

The first section of this chapter is about Designated Verifier Signature (DVS). Our objective is to achieve Identity-based DVS. First we will describe a generic way to construct DVS. In a second time we will show how to move from the normal setup to the Identity-based one. In this first part we will propose a generic way to construct DVS then give an efficient new instantiation.

4.1.1 Definition and Security

The principle of DVS is to allow a signer to sign a message that only one¹ chosen user can verify. We will here describe the scheme as a universal designated verifier signature introduced in [SBWP03].

There are many applications to a such primitive, for example it has the ability to provide a fully deniable email authentication solution where parties can exchange authenticated mail without non repudiation evidence. Thanks to the use of DVS, both parties involved in a mail conversation can authenticate the party they are talking to and are assured that no proof of this conversation can be disclosed. Indeed, if an attacker can gain access to the email inboxes of the trusted parties, he cannot provide a proof of authenticity of the contained information when disclosing emails. ID-based cryptography usage eases the overall process with the reduction of key exchanges needed. Another application is the use of DVS for cloud-based consultation of digitally signed documents. For instance, Electronic Health Records (EHR) and Personal Health Records (PHR) are now widely deployed in western societies. With this feature, the potential leakage of medical records cannot result in blackmail or insurance fraud as cyber criminals cannot provide a proof of authenticity for the stolen data. Of course, the potential embarrassment caused by the disclosure of medical information without authenticity verification cannot be avoided.

Definition 24. *A Universal Designated Verifier Signature Scheme DVS consists of seven PPT algorithms (Setup, KGS, KGV, Sign, Verify, Des, DesV) with the following properties:*

- **Setup**($1^{\mathfrak{K}}$): where $\mathfrak{K}$ is the security parameter, generates the global parameters **param** of the scheme.
- **KGS**(**param**): outputs a signing key pair $(\text{sk}_1, \text{pk}_1)$ for the signer.
- **KGV**(**param**): outputs a verifier key pair $(\text{sk}_2, \text{pk}_2)$ for the verifier.
- **Sign**($\text{sk}_1, \mathcal{M}$): outputs a signature σ on $\mathcal{M}$ publicly verifiable with pk_1 .
- **Verify**($\text{pk}_1, \mathcal{M}, \sigma$): checks the validity of σ .
- **Des**($\sigma, \mathcal{M}, \text{pk}_1, \text{pk}_2$): outputs a designated verifier signature $\hat{\sigma}$ for the verifier.
- **DesV**($\hat{\sigma}, \mathcal{M}, \text{sk}_2, \text{pk}_1$): checks the validity of the designated signature.

Using only **Setup**, **KGS**, **Sign**, and **Verify** we have a simple digital signature scheme. A Universal Designated Verifier Signature Scheme should satisfy the following properties:

¹it can easily be extended to a set of multiple users, for example it can be possible to use many times the algorithm **Des**

- The simple signature scheme (**Setup**, **KGS**, **Sign**, **Verify**) should be unforgeable (definition at section 2.4.1).
- Only the signer or the designated verifier should be able to generate a verifiable message signature pair for $(\mathcal{M}, \hat{\sigma})^2$. This is called **DV-Unforgeability** and expressed by the following security experiment ($\text{Exp}_{\text{dvuf}, \mathcal{A}}^{\text{UDVS}}$) for an adversary $\mathcal{A}$, where $\mathcal{SM}$ depicts the set of previously signed messages:

$\text{Exp}_{\text{dvuf}, \mathcal{A}}^{\text{UDVS}}(\mathcal{K})$:

$\text{param} \leftarrow \text{Setup}(1^{\mathcal{K}})$
 $(\text{sk}_1, \text{pk}_1) \leftarrow \text{KGS}(\text{param})$
 $(\text{sk}_2, \text{pk}_2) \leftarrow \text{KGV}(\text{param})$
 $(\mathcal{M}, \hat{\sigma}) \leftarrow \mathcal{A}^{\text{OSign}, \text{OVerify}}(\text{pk}_1, \text{sk}_2)$
 Return $\text{DesV}(\hat{\sigma}, \mathcal{M}, \text{sk}_2, \text{pk}_1)$ if $\mathcal{M} \notin \mathcal{SM}$

The probability of success against this game is denoted by

$$\text{Succ}_{\text{dvuf}, \mathcal{A}}^{\text{UDVS}}(\mathcal{K}) = \Pr[\text{Exp}_{\text{dvuf}, \mathcal{A}}^{\text{UDVS}}(\mathcal{K}) = 1],$$

- An adversary should not be able to convince a third party about the validity/invalidity of a designated signature, this is called **Non-transferability**³. In terms of advantage it means that an adversary must have at best a negligible advantage in distinguishing the two following distributions. We note $\mathcal{S}$ the signature space, and $\xleftarrow{\$}$ to indicate a random uniform sampling:

$$\begin{aligned} \Delta_0 &= \left\{ (\mathcal{M}, \hat{\sigma}) \left| \begin{array}{l} (\text{sk}_1, \text{pk}_1), (\text{sk}_2, \text{pk}_2) \leftarrow \text{KGS}, \text{KGV} \\ \hat{\sigma} = \text{Des}(\text{Sign}(\text{sk}_1, \mathcal{M}), \mathcal{M}, \text{pk}_1, \text{pk}_2) \end{array} \right. \right\} \\ \Delta_1 &= \left\{ (\mathcal{M}, \hat{\sigma}) \left| \begin{array}{l} (\text{sk}_1, \text{pk}_1), (\text{sk}_2, \text{pk}_2) \leftarrow \text{KGS}, \text{KGV} \\ \hat{\sigma} \xleftarrow{\$} \mathcal{S}. \end{array} \right. \right\} \end{aligned}$$

Throughout our security experiments, we will use different oracles:

²We allow the verifier to be able to create a such signature because if he was not able to then by giving his secret key, he would be able to prove to someone that the signer signed the message.

³This is the key property of this primitive

- **OSign** takes as input $\mathcal{M}$, a signer public key pk_1 and returns a signature σ on this message and adds $\mathcal{M}$ to the set of already signed message $\mathcal{SM}$.
- **OVerify** takes as input a designated verifier signature $\hat{\sigma}$, a message $\mathcal{M}$ and a verification key and checks their validity.

Remark 5. *The hard-to-get property is the **Non-transferability**. It will force us to use heavy tools to satisfy this property as we describe in the next paragraph. Note that at this point DVS and standard digital signature are very different when proving the security: for a standard digital security we can rely on computational problem since the adversary has to produce a value. In the case of the DVS the **Non-transferability** property does not force the adversary to produce anything since it is an indistinguishability between two distributions. It means that we will need to rely the security on a decisional assumption.*

4.1.2 A New Construction

It is natural to start from a signature to build a DVS since the inner scheme should satisfy regular signature's security property (e.g. Unforgeability). This is our starting point, but now we want to transform this signature to have the **Non-transferability** and the **DV-Unforgeability** properties. We will interest in the use of SPHF along with an extractable commitment (an encryption scheme for simplicity). It is an interesting approach because the verifier will have to compute the value (using the SPHF's Hashing algorithm) and verify the equality with the value sent by the signer. Thus the verifier will not be able to convince a third party that he owns a signature on a precise message from the signer because it could have computed it itself. We will detail this in the following.

Starting from the result presented by Blazy et al. [BPV12], we achieve UDVS in 3 steps:

- The signer will sign $\mathcal{M}$ using sk_1 resulting in a signature σ .
- The signer then encrypts σ with an encryption key ek and sends it to the verifier (ek will be a public parameter and no user knows the corresponding private key).
- Both the sender and the verifier will use a SPHF on the language $\mathcal{L}$ the set composed of encrypted valid signatures from the signer on the message $\mathcal{M}$. In our proposition, the Universal Designated Verifier keys are generated thanks to the SPHF, namely $\text{sk}_2 = \text{hk}$ and $\text{pk}_2 = \text{hp}$. The signature is valid if the verifier receives the same value as the one he computed.

We describe this scheme in the Figure 4.1.

High Level Security Analysis We briefly give security sketches for the various expected security properties, and show how they reduce in a black-box way to the security of the underlying building blocks.

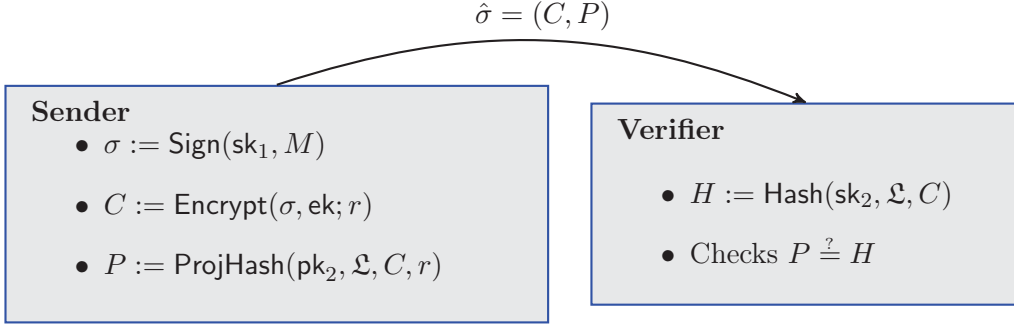


FIGURE 4.1: UNIVERSAL DESIGNATED VERIFIER SIGNATURE BASED ON SPHF

Theorem 4. *Under the unforgeability of the signature scheme, the proposed Universal Designated Verifier Signature construction is Unforgeable.*

Theorem 5. *Under the unforgeability of the signature scheme, and the Smoothness of the SPHF, the proposed Universal Designated Verifier Signature construction is DV-Unforgeable.*

Theorem 6. *Under the indistinguishability (IND-CPA) of the encryption scheme and the pseudo randomness of the SPHF, the proposed Universal Designated Verifier Signature construction is Non-Transferable.*

Proof of Unforgeability

The inner signing mechanisms of the UDVS is kept unchanged from the underlying signature scheme hence the security remains the same.

Proof of DV-Unforgeability

An adversary $\mathcal{A}$ breaking the DV-unforgeability has a non-negligible probability to produce a valid tuple $(\mathcal{M}, \sigma)$ for a fresh unsigned message (for example verifier).

Proof. We will construct an adversary $\mathcal{B}$ against the unforgeability of the signature scheme. For signatures, $\mathcal{B}$ uses a signing oracle from the initial signature scheme, this is indistinguishable from the real game, and he knows the decryption key sk_1 associated with the encryption key in the common reference string.

When receiving the challenge signature, first $\mathcal{B}$ check if the message is fresh, and the pair valid. In this case, he parses $\hat{\sigma}$ in (C, P) . Under the smoothness of the underlying SPHF, C is a valid encryption of a valid signature on M^* for the given public key. Using sk_1 , $\mathcal{B}$ can recover a fresh σ^* on M^* , and so breaks the unforgeability of the inner signature scheme. $\square$

Proof of Non-Transferability

An adversary $\mathcal{A}$ breaking the Non-Transferability has a non-negligible probability to distinguish a valid designated signature from an invalid one. We are going to show how to build a simulator $\mathcal{B}$ breaking the IND-CPA security of the underlying encryption scheme.

Proof. For signatures, $\mathcal{B}$ uses a signing oracle from the initial signature scheme, this is indistinguishable from the real game, and he knows the decryption key sk_1 associated with the encryption key in the common reference string.

We are going to proceed in a sequence of games moving from $\mathcal{G}_0$ where the adversary is presented with a valid designated signature, to $\mathcal{G}_2$ where he sees a random value.

- $\mathcal{G}_0$ This is the real game, the signer queries a signature to the signing oracle, encrypts it, and computes honestly the projective hash for the designated identity.
- $\mathcal{G}_1$ The signer now picks a random value for the projected hash, under the pseudo-randomness of the SPHF (implied by definition by the indistinguishability of the encryption), this is indistinguishable from the valid value.
- $\mathcal{G}_2$ The signer now encrypts a random value, once again under the indistinguishability of the encryption, this is indistinguishable from the previous game.

Hence

$$\text{Adv}_{\mathcal{A}}^{NT} \leq 2 * \text{Adv}_{\mathcal{A}}^{\text{ind-cpa}}$$

□

Instantiation The main difficulty to build this scheme is to find a SPHF on the right language *i.e.* encryptions of a valid signature on $\mathcal{M}$. Here we use the linear encryption over Waters signature [Wat05] followed by the SPHF⁴ from [BPV12]. In this instantiation we are able to combine the encryption and the signature in a more efficient scheme by combining these primitives while reusing part of the randomness (see Figure 4.2). Reusing the randomness s allows to have $\sigma_2 = C_2$, this redundancy reduces the overall communication cost by one element without altering the security proof. This optimization is called **Sign&Crypt** leads to a slightly modified version of the language $\mathfrak{L}$ ⁵. We want to stress out that this new primitive is different from both the Signcrypt introduced by Zheng in [Zhe97] and Signature on Randomizable Ciphertext introduced by Blazy et al. in [BFPV11].

⁴see Figures 2.3 , 2.10 and 2.13 respectively

⁵ $\mathfrak{L} := \text{WLin}(\text{ek}, \text{pk}, M)$ denotes the set of elements computed as **Sign&Crypt**(M, ek, sk): this is the set of encryption of valid signature of the message M with the optimization in the writing

<u>Setup(1^κ)</u> $\mathcal{G}_g = (p, \mathbb{G}, g, h, \nu, \mathbb{G}_T, e) \leftarrow \text{GGen}(1^\kappa)$ $\text{ek} \leftarrow \text{Lin.KeyGen}(\mathcal{G}_g)$ $U = (u_0, \dots, u_k) \xleftarrow{\$} \mathbb{G}^{k+1}$ $\text{param} = (\mathcal{G}_g, \text{ek}, U)$ <u>KGV(param)</u> $\text{sk}_2 \leftarrow \text{HashKG}(\mathcal{L}_{\text{ek}, \text{pk}_1})$ $\text{pk}_2 \leftarrow \text{ProjKG}(\text{hk}, \mathcal{L}_{\text{ek}, \text{pk}_1})$ Return $(\text{sk}_2, \text{pk}_2)$	<u>KGS(param)</u> Return $(\text{sk}_1, \text{pk}_1) := \text{Wat.KeyGen}(\text{param})$ <u>Sign($\mathcal{M}, \text{sk}_1, \text{pk}_2$)</u> $\sigma \leftarrow \text{Wat.Sign}(\text{sk}_1, \mathcal{M}, s)$ $C \leftarrow \text{Lin.Encrypt}(\text{ek}, \sigma_1, (r, s))$ $P \leftarrow \text{ProjHash}(\text{pk}_2, \mathcal{L}, C, r)$ Return (C, P) <u>Verify($\text{sk}_2, \mathcal{L}, C, P$)</u> $H \leftarrow \text{Hash}(\text{sk}_2, \mathcal{L}_{\text{ek}, \text{pk}_1}, C)$ Return 1 if $H = P$ and 0 otherwise.
---	--

FIGURE 4.2: INSTANTIATION OF SIGN&CRYPT

Additional Properties: In fact, the scheme is more than a mere universal designated verifier signature. By giving the decryption key dk to an authority we allow it to recover a standard signature. Thus, our UDVS scheme is both undeniable and convertible. The undeniability is achieved because the signer can not deny that he signed a message. The convertibility comes from the fact that the authority is able to transform a designated signature into a regular signature.

Security Proof of the SPHF over the Sign&Crypt

Theorem 7. *This Smooth Projective Hash Function (Figure 2.13) is smooth.*

Proof. Let us show that from an information theoretic point of view, $v = \text{Hash}(\text{sk}_2; \mathcal{L}, \mathcal{C})$ is unpredictable, even knowing hp , when Σ is not a correct Sign&Crypt, in other words $\mathcal{C} = (C_1 = Y_1^r, C_2 = Y_2^{r_2} = \sigma_2, C_3 = \nu^t \sigma_1)$, for $t \neq r_1 + r_2$. (We stress that every incorrect tuple can be written like this)

We recall that we earlier defined $\text{Hash}(\text{hk}, \mathcal{L}, W) = H := e(C_1, \nu)^{x_1} \cdot (e(C_3, g) / (e(h, \text{pk}_1) \cdot e(\nu \mathcal{F}(\mathcal{M}), C_2)))^{x_2} = e(C_1, \nu)^{x_1} \cdot e(\nu, g^t / C_2)$.

If we denote $Y_1 = g^{y_1}$, $\nu = g^{y_3}$, and consider the discrete logarithm in $\mathbb{G}_T$ we have:

$$\begin{pmatrix} \log \text{pk}_2 \\ \log H \end{pmatrix} = \begin{pmatrix} y_1 & 1 \\ r_1 y_1 y_3 & (t - r_2) y_3 \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

The determinant of this matrix is non-zero iff $\mathcal{C}$ does not belong to the language in other words when $(t \neq r_1 + r_2)$. So H is independent from pk_2 and $\mathcal{C}$. $\square$

4.1.3 Moving To The Identity-based Context

Generic Approach Obviously to move in the identity-based setting we are going to transform the building blocks into an identity-based signature (IBS), and an identity-based SPHF (ID-SPHF). Using identity-based building blocks

we are able to construct ID-DVS the same way we constructed DVS, it is describe in Figure 4.3.

<u>Setup($\mathcal{R}$)</u> $\text{param} \leftarrow \text{GGen}(\mathcal{R})$ $\text{mpk}_1, \text{msk}_1, \text{mdk}_1 = \text{HIBKEM.Gen}(\mathcal{R})$ $\text{mpk}_2, \text{msk}_2 = \text{ID-SPHF.Setup}(\mathcal{R})$ <u>KGS($\text{id}_1, \text{msk}_1$)</u> Return $\text{sk}_1, \text{dk}_1 := \text{HIBKEM.USKGen}(\text{msk}_1, \text{id}_1)$ <u>KGV($\text{id}_2, \text{msk}_2$)</u> Return $\text{sk}_2 := \text{ID-SPHF.USKGen}(\text{msk}_2, \text{id}_2)$	<u>Sign_d((sk_1, dk_1), $\mathcal{M}, (\text{mpk}_2, \text{id}_2)$)</u> $\sigma \leftarrow \text{HIBKEM.USKDel}(\text{mdk}_1, \text{sk}_1, \text{dk}_1, \text{id}_1, \mathcal{M})$ $C, P \leftarrow \text{ID-SPHF.Encap}(\text{mpk}_2, \text{id}_2; \sigma)$ Return (C, P) <u>Verify($\text{msk}_2, \mathcal{L}, C, P, \text{id}_1$)</u> $H \leftarrow \text{ID-SPHF.Decap}(\text{msk}_2, C)$ Return 1 if $H = P$ and 0 otherwise
---	--

FIGURE 4.3: ID-DVS FROM ID-SPHF AND HIBKEM

The security will not be different from the scheme in a standard setting:

Theorem 8. *Under the respective security of the HIBKEM and the ID-SPHF, the Designated Verifier Signature is unforgeable, DV-unforgeable and non-transferable.*

Sketch of Proof: An adversary breaking the (DV)-unforgeability of the ID-DVS either breaks the smoothness of the ID-SPHF (by sending an invalid inner signature with an adapted proof), or the unforgeability of the IBS (by sending a forgery of the inner signature), and so the indistinguishability of the HIBKEM.

Identity-based Signature The difficulty thus lies in the constructions of these Identity-based primitives. We have another constraint, the primitives chosen have to be compatible. For the signature scheme, we use the transformation from a Hierarchical Identity-based Key Encapsulation Mechanism scheme (HIBKEM) to an IBS [CFH⁺07]. The concept is quite simple, the signature from id_i on $\mathcal{M}$ will be the user secret key $\text{usk}[\text{id}_i || \mathcal{M}]$ derived from $\text{usk}[\text{id}_i]$. Thus the verification is made by decrypting a dummy message encrypted for this new identity.

We supersede the SPHF in the previous approach by an ID-SPHF to achieve ID-DVS described in Figure 4.3. Roughly speaking, the two primitives will be compatible because the SPHF will implicitly verify the signature by computing pairings on the commitment of the signature.

Hierarchical ID-based Key Encapsulation: We use the one presented in [BKP14]. In the present scheme (see Figure 4.4), we only consider a hierarchy of depth 2 which avoids the need to rerandomize the delegation key⁶.

Identity-based Designation ID-based Smooth Projective Hash Func-

⁶There is a potential flaw/imprecision in the original proof around the simulation of the randomization of the delegation term, However, this does not arise here as we consider only one delegation step.

<u>Gen($\mathcal{R}$):</u> $\mathbf{A} \xleftarrow{\\$} \mathbb{Z}_p^{2 \times 1}; \mathbf{B} \xleftarrow{\\$} \mathbb{Z}_p$ For $i = 1, \dots, 2\ell$: $\mathbf{Y}_i \xleftarrow{\\$} \mathbb{Z}_p^2; \mathbf{Z}_i = \mathbf{Y}_i^\top \cdot \mathbf{A} \in \mathbb{Z}_p$ $\mathbf{E}_i = \mathbf{Y}_i \cdot \mathbf{B} \in \mathbb{Z}_p^2$ $y'_0 \xleftarrow{\\$} \mathbb{Z}_p^2; z'_0 = y'_0{}^\top \cdot \mathbf{A} \in \mathbb{Z}_p$ $\text{pk} := (\mathcal{G}, [\mathbf{A}]_1, ([\mathbf{Z}_i]_1)_{0 \leq i \leq \ell}, [z'_0]_1)$ $\text{dk} := ([\mathbf{B}]_2, ([\mathbf{E}_i]_2)_{0 \leq i \leq 2\ell})$ $\text{sk} := (\mathbf{B}, (\mathbf{Y}_i)_{0 \leq i \leq \ell}, y'_0)$ Return $(\text{mpk}_1 = (\text{pk}, \text{dk}), \text{msk}_1 = \text{sk})$ <u>USKGen($\text{msk}_1, \text{id}_1 \in \text{ID}$):</u> $t \in \mathbb{Z}_p$ $v = \sum_{i=1}^{\ell} f_i(\text{id}_1) \mathbf{Y}_i t + y'_0 \in \mathbb{Z}_p^2$ For $i \in [0, \ell - 1]$: $e_i = \mathbf{Y}_{\ell+i} t \in \mathbb{Z}_p^2$ $\text{usk}[\text{id}_1] := ([t]_2, [v]_2) \in \mathbb{G}_2 \times \mathbb{G}_2^2$ $\text{udk}[\text{id}_1] := ([e_i]_2)_{\ell < i \leq 2\ell} \in (\mathbb{G}_2^2)^\ell$ Return $\text{sk}_1 = (\text{usk}[\text{id}_1], \text{udk}[\text{id}_1])$	<u>USKDel($\text{dk}, \text{sk}_1, \text{id}_1, \mathcal{M}$):</u> $\hat{\text{id}}_1 := \text{id}_1 \mathcal{M}$ $\hat{v} = v + \sum_{i=\ell+1}^{2\ell} f_i(\hat{\text{id}}_1) e_i \in \mathbb{Z}_p^2$ $s' \xleftarrow{\\$} \mathbb{Z}_p, t' = t + \mathbf{B} s' \in \mathbb{Z}_p$ $v' = \hat{v} + \sum_{i=1}^{2\ell} f_i(\hat{\text{id}}_1) \mathbf{E}_i s' \in \mathbb{Z}_p^2$ $\text{usk}[\text{id}'] := ([t']_2, [v']_2) \in \mathbb{G}_2 \times \mathbb{G}_2^2$ Return $\sigma = \text{usk}[\hat{\text{id}}_1]$ <u>Enc($\text{mpk}_1, \text{id}_1$):</u> $r \xleftarrow{\\$} \mathbb{Z}_p; c_0 = \mathbf{A} r \in \mathbb{Z}_p^2$ $c_1 = (\sum_{i=1}^{2\ell} f_i(\text{id}_1) \mathbf{Z}_i) \cdot r \in \mathbb{Z}_p$ $K = z'_0 \cdot r \in \mathbb{Z}_p$ Return $\mathbf{K} = [K]_T$ and $\mathbf{C} = ([c_0]_1, [c_1]_1)$ <u>Dec($\text{usk}[\hat{\text{id}}_1], \hat{\text{id}}_1, \mathbf{C}$):</u> Parse $\text{usk}[\hat{\text{id}}_1] = ([t]_2, [v]_2)$ Parse $\mathbf{C} = ([c_0]_1, [c_1]_1)$ $\mathbf{K} = e([c_0]_1, [v]_2) \cdot e([c_1]_1, [t]_2)^{-1}$ Return $\mathbf{K} \in \mathbb{G}_T$
---	---

FIGURE 4.4: HIBKEM UNDER SXDH ($k = \text{MDDH}$ FOR $k = 1$).

tion

We are now going to describe such a compatible ID-based Hash Proof System, which is inspired from the one in the penultimate appendix of [BKP14], except we need some extra tweaking to be compatible with a pairing based verification.

In [BKP14], they presented an ID-Hash proof system based on SXDH. We note that **Setup**, **USKGen**, **Encap** and **Decap** are the same as in HIBKEM from earlier and **Encap**^{*} returns a random ciphertext $\mathbf{C}$ from the ciphertext space $\mathbf{CS} = \mathbb{G}_1^3$. Correctness of ID-SPHF follows from the correctness of the underlying HIBKEM.

Revisiting the techniques from [BBC⁺13a], one can extend their construction to handle language more complex like encryption of elements $\alpha \in \mathcal{L}$. The language corresponding to valid signature is the set of solutions to a pairing product equation which requires some extra care. We need to supersede the USKGen in the ID-SPHF to be compatible with the verification equation of the inner signature. For that, instead of computing t directly in the ID-SPHF the authority receives the values $\mathbf{Z}_i$ from the HIBKEM, and also sends t' , a set of $[t\mathbf{Z}_i]_1$.

To sign, a user first computes $\sigma_1 = t_i, \sigma_2 = \sum \text{id}'_i Y_i t_i + y'$. He then uses the ID-SPHF to hide t , for user j , as $c = (\mathbf{G}s, \sum \text{id}_j W_j s + \sum \text{id}'_i \mathbf{Z}_i t_i)$ and $P = [w's]_2$.

User j checks the validity with $[c_0^\top v_j - c_1^\top t_j - P + (\sum \text{id}'_i \mathbf{Z}_i t_i)^\top t_j]_T \stackrel{?}{=} 1_T$, this is done via the ID-SPHF presented in Figure 4.5.

We added some elements compared to the original ID-SPHF but the scheme

<u>Setup($\mathcal{R}$):</u> $\mathbf{G} \xleftarrow{\\$} \mathbb{Z}_p^{2 \times 1}$ For $i = 1, \dots, \ell$: $\mathbf{X}_i \xleftarrow{\\$} \mathbb{Z}_p^2, \mathbf{W}_i = \mathbf{X}_i^\top \cdot \mathbf{G} \in \mathbb{Z}_p$ $x' \xleftarrow{\\$} \mathbb{Z}_p^2; w' = x_i^\top \cdot \mathbf{G} \in \mathbb{Z}_p^2$ $\text{mpk}_2 := (\mathcal{G}, [\mathbf{G}]_2, ([\mathbf{W}_i]_2)_{1 \leq i \leq \ell}, [w']_2)$ $\text{msk}_2 := ((\mathbf{X}_i)_{1 \leq i \leq \ell}, x')$ Return $(\text{mpk}_2, \text{msk}_2)$ <u>USKGen($\text{msk}_2, \text{id}_2$):</u> $[t]_2 \xleftarrow{\\$} \mathbb{Z}_p$ $v = \sum_{i=0}^{\ell} \text{id}_{2,i} \mathbf{X}_i t + x'_i \in \mathbb{Z}_p^2$ Return $\text{sk}_2 := ([t]_1, [v]_1) \in \mathbb{G}_1^3$	<u>Encap($\text{mpk}_2, \text{id}_2; \alpha$):</u> $r \xleftarrow{\\$} \mathbb{Z}_p$ $c_0 = \mathbf{G}r \in \mathbb{Z}_p^2$ $c_1 = (\sum_{i=0}^{\ell} \text{id}_{2,i} \mathbf{W}_i) \cdot r + \alpha \in \mathbb{Z}_p$ $K = z'_i \cdot r \in \mathbb{Z}_p$ Return $\mathbf{C} = ([c_0]_1, [c_1]_2)$ and $\mathbf{K} = [K]_T$ <u>Encap* ($\text{mpk}_2, \text{id}_2$):</u> $(c_0, c_1) \xleftarrow{\\$} \mathbb{Z}_p^3$ Return $\mathbf{C} = ([c_0]_2, [c_1]_2) \in \mathbb{G}_2^3$ <u>Decap($\text{sk}_2, \mathbf{C}$):</u> Parse $\text{sk}_2 = ([t]_1, [v]_1)$ Parse $\mathbf{C} = ([c_0]_2, [c_1]_2)$ $\mathbf{K} = [c_0 v - (c_1 - \alpha)t]_T$ Return $\mathbf{K} \in \mathbb{G}_T$
---	---

FIGURE 4.5: EXAMPLE OF AN ID-BASED HPS FOR THE LANGUAGE OF ENCRYPTION.

remains secure since this elements lies in the span of those already given in the original scheme. Under the Self Random Reducibility of the SXDH assumption those extra elements can be simulated.

In terms of communication cost , we manage to keep a lightweight signature made of 5 group elements under SXDH, plus a 128 bits string while we keep a high level security. It is hard to compare with the state of the art for an ID-DVS since they are almost all proven secure in the random oracle model.

Additional properties

The obtained signature in the previous part is a little more than a simple ID-DVS, and reveals to possess various interesting properties. The signature is fully randomizable. So in case a user $\mathcal{A}$ is not able to reach $\mathcal{B}$ directly, he can send something to an intermediate proxy $\mathcal{C}$ which will be unable to learn the validity of the signature. Then $\mathcal{C}$ is able to fully randomize it before sending it to user $\mathcal{B}$ who will be able to learn that it is a signature from $\mathcal{A}$. This would, for instance, allow to hide metadatas in communication preventing competitors to learn that a given enterprise is communicating with a prospective government.

The signature is also an *Undeniable Signature* as the ID-SPHF master authority can revoke the obfuscation of a given signature, and determine whether it is valid or not. It is interesting to note, that the HIBKEM authority has no power at this step, meaning that while the *Judge* has to be unique (e.g. the one with the power of revealing which has signed the message), there can be many sub-authorities in charge of signature (e.g. the HIBE authorities).

4.2 Attribute-based Encryption from Downgradable IBE

Beyond the Notion of Identities, a Step Toward ABE

In an IBE the idea is to describe any user by a bit string. Mail addresses are a classical example. Looking at an address: *Alice@etu.crypto-uni.edu*, one can see that identities englobes some kind of attributes. The user is a student in the university "crypto". From this, it seems natural to extend the construction of IBE to ABE.

This bring us to the link between WIBE and Attribute-based Encryption: using the example of Alice's mail address one can encrypt for every user of the university using the identity *"*...*crypto-uni.edu"*. The problem with this approach is that WIBE are not practical enough to be used. Moreover an identity representing a lot of attributes could be quite big and thus leads to big ciphertext. Instead we propose to use DIBE and rethink the building of the identity string: now each bit will refer to one attribute. Following our example with Alice, her identity is now a bit string made of 0s for every bits but the ones corresponding to: {university=crypto, rank=student}. The power of this re-conceptualization lies in the construction of the DIBE, in the DIBE a bit set to 0 will not express in the user secret key and thus only the 1s will matter. A user is now completely defined by what he owns and what he does not own.

4.2.1 Definitions and Security

To define Attribute-based Encryption we need to first define what is an access structure⁷:

Definition 25 (Access Structure). *An access structure $\mathbb{F}$ is a collection non-empty subsets of the universe of attributes U . For simplicity we will index our attributes. Thus we are able to express access structure as follow: $\mathbb{F} \subseteq 2^U \setminus \{0\}$.*

Definition 26 (Attribute-based Encryption). *An Attribute-based encryption (ABE) scheme $\mathbf{ABE}$ consists of four PPT algorithms $\mathbf{ABKEM} = (\text{Setup}, \text{USKGen}, \text{Enc}, \text{Dec})$ with the following properties.*

- *The probabilistic key generation algorithm $\text{Setup}(\mathfrak{K})$ returns the (master) public/secret key (pk, sk) . We assume that pk implicitly defines a message space $\mathcal{M}$, an Attribute space U , and ciphertext space $\mathcal{CS}$.*
- *The probabilistic user secret key generation algorithm $\text{USKGen}(\text{sk}, \mathbb{A})$ that takes as input the master secret key sk and a set of attributes $\mathbb{A} \subset U$ and returns the user secret-key $\text{usk}[\mathbb{A}]$.*
- *The probabilistic encryption algorithm $\text{Enc}(\text{pk}, \mathbb{F}, M)$ returns a ciphertext $C \in \mathcal{CS}$ with respect to the access structure $\mathbb{F}$.*

⁷In our case it will be a boolean formula in Disjunctive Normal Form (DNF) it means that the formulae is made of disjunction of conjunction of attribute

Procedure Initialize: $(pk, sk) \xleftarrow{s} \text{Setup}(\mathfrak{K})$ Return pk	Procedure Enc($\mathbb{F}^*$): //one query $(sk^*, C^*) \xleftarrow{s} \text{Enc}(pk, \mathbb{F}^*, M^*)$ $C^* \xleftarrow{s} CS$ Return (C^*)
Procedure USKGen($\mathbb{A}$): $\mathcal{Q}_A \leftarrow \mathcal{Q}_A \cup \{\mathbb{A}\}$ Return $usk[\mathbb{A}] \xleftarrow{s} \text{USKGen}(sk, \mathbb{A})$	Procedure Finalize(β): Return $(\forall \mathbb{A} \in \mathcal{Q}_A, \mathbb{A} \text{ does not verify } \mathbb{F}) \wedge \beta$

FIGURE 4.6: SECURITY GAMES $\text{PR-A-CPA}_{\text{real}}$ AND $\text{PR-A-CPA}_{\text{rand}}$ (BOXED) FOR DEFINING PR-A-CPA-SECURITY .

- The deterministic decryption algorithm $\text{Dec}(usk[\mathbb{A}], \mathbb{F}, \mathbb{A}, C)$ returns the decrypted message $M \in \mathcal{M}$ or the reject symbol $\perp$.

For perfect correctness we require that for all $\mathfrak{K} \in \mathbb{N}$, all pairs (pk, sk) generated by $\text{Setup}(\mathfrak{K})$, all access structure $\mathbb{F}$, all set of attribute $\mathbb{A} \subset U$ satisfying $\mathbb{F}$, all $usk[\mathbb{A}]$ generated by $\text{USKGen}(sk, \mathbb{A})$ and all C output by $\text{Enc}(pk, \mathbb{F}, M)$:

$$\Pr[\text{Dec}(usk[\mathbb{A}], \mathbb{F}, \mathbb{A}, C) = M] = 1.$$

Like before, we encompass the classical security hypotheses for an ABE, with a PR-A-CPA one as described in Figure 4.6.

Definition 27 (PR-A-CPA Security). *An attribute-based key encapsulation scheme ABKEM is PR-A-CPA -secure if for all PPT $\mathcal{A}$, $\text{Adv}_{\text{ABKEM}}^{\text{PR-A-CPA}}(\mathcal{A}) := |\Pr[\text{PR-A-CPA}_{\text{real}}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{PR-A-CPA}_{\text{rand}}^{\mathcal{A}} \Rightarrow 1]|$ is negligible.*

Note that we distinguish two types of ABE: the Key Policy ABE (KP-ABE) and the Ciphertext Policy ABE (CP-ABE). The former allows to encrypt for attributes and the user can decrypt if the attributes verify his personal policy lying in his user secret key. The latter is way more practical because the policy lies in the ciphertext, thus the policy can be adapted at every new encryption to target different users. In this thesis we will only talk about CP-ABE.

4.2.2 Transformation from DIBE to ABE

In a usual notion of (ciphertext-policy) ABE, a key is associated with a set $\mathbb{A}$ of attributes in the attribute universe $\mathcal{U}$, while a ciphertext is associated with an access policy $\mathbb{F}$ (or called access structure) over attributes. The decryption can be done if $\mathbb{A}$ satisfies $\mathbb{F}$. We can see that IBE is a special case of ABE where both $\mathbb{A}$ and $\mathbb{F}$ are singletons.

In this thesis, we confine ABE in the two following aspects. First, we restrict the universe $\mathcal{U}$ to be of polynomial size in security parameter; this is often called

small-universe ABE (as opposed to large-universe ABE where $\mathcal{U}$ can be of super polynomial size.). Second, we allow only DNF formulae in expressing policies (as opposed to any boolean formulae, or equivalently, any access structures).

Our idea for obtaining a (small-universe) ABE scheme for DNF formulae from any DIBE scheme is as follows. For simplicity and without lost of generality, we set the universe as $\mathcal{U} = \{1, \dots, n\}$. We will use DIBE with identity length n . For any set $S \subseteq \mathcal{U}$, we define $\text{id}_S \in \{0, 1\}^n$ where its i -th position is defined by

$$\text{id}_S[i] := \begin{cases} 1 & \text{if } i \in S \\ 0 & \text{if } i \notin S \end{cases}.$$

To issue an ABE key for a set $\mathbb{A} \subseteq \mathcal{U}$, we use a DIBE key for $\text{id}_{\mathbb{A}}$. On the other hand, to encrypt a message M in ABE with a DNF policy $\mathbb{F} = \bigvee_{j=1}^k (\bigwedge_{a \in S_j} a)$, where each attribute a is in $\mathcal{U}$, we encrypt the same message M in DIBE each with id_{S_j} for all $j \in [1, k]$; this will result in k ciphertexts of the DIBE scheme. Note that k is the number of OR, the disjunction, in the DNF formula.

Decryption can be done as follows. Suppose $\mathbb{A}$ satisfies $\mathbb{F}$. Hence, we have that there exists S_j (defined in the formula $\mathbb{F}$) such that $S_j \subseteq \mathbb{A}$. We then derive a DIBE key for id_{S_j} from our ABE key for $\mathbb{A}$ (which is then a DIBE key for $\text{id}_{\mathbb{A}}$); this can be done since $S_j \subseteq \mathbb{A}$ implies that any positions of 1 in id_{S_j} will also contain 1 in $\text{id}_{\mathbb{A}}$ (and thus the derivation is possible). We finally decrypt the ciphertext associated with id_{S_j} to obtain the message M . We summarize this transformation in Figure 4.7.

<u>Setup(param):</u> Run $\text{Setup}_{\text{DIBE}}(\mathfrak{R})$ Return (pk, sk) <u>KeyGen(sk, $\mathbb{A}$):</u> Return $\text{usk}[\mathbb{A}] \leftarrow \text{USKGen}_{\text{DIBE}}(\text{sk}, \text{id}_{\mathbb{A}})$	<u>Encrypt(pk, $\mathbb{F}$, M):</u> Parse $\mathbb{F} = \bigvee_{j=1}^k (\bigwedge_{a \in S_j} a)$ For all $j \in [1, k]$, compute: $(C_j, K_j) \leftarrow \text{Enc}_{\text{DIBE}}(\text{pk}, \text{id}_{S_j})$ and $C'_j \leftarrow M \oplus K_j$ Return $\mathbf{C} = (C_1, \dots, C_k, C'_1, \dots, C'_k)$ <u>Decrypt(usk[$\mathbb{A}$], $\mathbb{F}$, $\mathbf{C}$):</u> Parse $\mathbb{F} = \bigvee_{j=1}^k (\bigwedge_{a \in S_j} a)$ Find $j \in [1, k]$ s.t. $S_j \subseteq \mathbb{A}$ Compute $U \leftarrow \text{USKDown}_{\text{DIBE}}(\text{usk}[\mathbb{A}], \text{id}_{S_j})$ Compute $K_j \leftarrow \text{Dec}_{\text{DIBE}}(U, \text{id}_{S_j}, C_j)$ Return $M = C'_j \oplus K_j$
--	--

FIGURE 4.7: ABE FROM DIBE

We have the following security theorem for the above ABE scheme. The proof is very simple and is done by a straightforward hybrid argument over k ciphertexts of DIBE.

Theorem 9. *The above ABE from DIBE is PR-A-CPA secure under the pr-id-cpa security of the DIBE scheme used. In particular for all adversaries $\mathcal{A}$, we have*

that $\text{Adv}_{\text{ABE}}^{\text{PR-A-CPA}}(\mathcal{A}) \leq k \cdot \text{Adv}_{\text{DIBE}}^{\text{pr-id-cpa}}(\mathcal{A})$ where k is the number of OR in the DNF formula (associated to the challenge ciphertext).

Proof. We prove our transformation via a sequence of games beginning with the real game for the PR-A-CPA security of the ABE and ending up with a game where the ciphertext of the ABE is uniformly chosen at random e.g. a game where adversary's advantage is reduce to 0.

Let $\mathcal{A}$ be an adversary against the PR-A-CPA security of our transformation. Let C be the simulator of the PR-A-CPA experience.

Game G_0 : This is the real security game.

Game $G_{1,1}$: In this game the simulator generates correctly every ciphertexts but the first one. The first ciphertext is replaced by a random element of the ciphertext space. $G_{1,1}$ is indistinguishable from G_0 if the pr-id-cpa security holds for the DIBE used.

$$\text{Adv}^{G_0, G_{1,1}}(\mathcal{A}) \leq \text{Adv}_{\text{DIBE}}^{\text{pr-id-cpa}}(\mathcal{A})$$

Game $G_{1,i}$: This game is the same than the game $G_{1,i-1}$ but the i -th ciphertext is replaced by a random element of the ciphertext space. $G_{1,i}$ is indistinguishable from $G_{1,i-1}$ if the pr-id-cpa security holds for the DIBE used.

$$\text{Adv}^{G_{1,i-1}, G_{1,i}}(\mathcal{A}) \leq \text{Adv}_{\text{DIBE}}^{\text{pr-id-cpa}}(\mathcal{A})$$

Game $G_{1,k}$: in this game all ciphertexts are random elements, $G_{1,k}$ is indistinguishable from $G_{1,k-1}$ if the pr-id-cpa security holds for the DIBE used.

$$\text{Adv}^{G_{1,k-1}, G_{1,k}}(\mathcal{A}) \leq \text{Adv}_{\text{DIBE}}^{\text{pr-id-cpa}}(\mathcal{A})$$

At this point our current game $G_{1,k}$ has for challenge encryption only random elements. This means that an adversary has no advantage in winning this game. We finally end up with the advantage of $\mathcal{A}$ in winning the original security game:

$$\begin{aligned} \text{Adv}_{\text{ABE}}^{\text{PR-A-CPA}}(\mathcal{A}) &\leq \text{Adv}^{G_0, G_{1,k}}(\mathcal{A}) \\ &\leq \sum_{i=1}^k \text{Adv}^{G_{1,i-1}, G_{1,i}}(\mathcal{A}) \\ &\leq k \times \text{Adv}_{\text{DIBE}}^{\text{pr-id-cpa}}(\mathcal{A}) \end{aligned}$$

□

4.2.3 Efficiency comparison for ABE

Our instantiation leads to a very efficient ABE scheme with tight reduction to a classical primitive. This scheme would be one of the most practical. However

Name	$ pk $	$ sk $	$ C $	pairing	exp $\mathbb{G}$	exp $\mathbb{G}_t$	Reduction Loss
[OT10]	$4U + 2$	$3U + 3$	$7m + 5$	$7m + 5$	0	m	$O(q_k)$
[LW12]	$24U + 12$	$6U + 6$	$6m + 6$	$6m + 9$	0	m	$O(q_k)$
[CGW15]	$6UR + 12$	$3UR + 3$	$3m + 3$	6	$6m$	0	$O(q_k)$
[Att16] scheme 10	$6UR + 12$	$3UR + 6$	$3m + 6$	9	$6m$	0	$O(q_k)$
[Att16] scheme 13	$96(M+TR)^2 + \log(UR)$	$3UR + 6$	$3m + 6$	9	$6m$	0	$O(q_k)$
Our DNF- ABE	$4U + 2$	$3U + 3$	$3k + 2$	13	0	0	$O(k)$

FIGURE 4.8: EFFICIENCY COMPARISON OF PRACTICAL CP-ABE SCHEMES

we achieve ABE where the access structure has to be a boolean formula in the DNF which is less practical than allowing any kind of access structure (which is done in others practical schemes).

Figure 4.8 presents a non exhaustive comparison of our ABE schemes with efficient ones. They are all fully secure under the classical assumption DLin . U is the size of the universe of attributes. m is the number of attributes in a policy. t is the size of an attribute set, and T is the maximum size of t (if bounded). R is the maximum number of attributes multi used in one policy (if bounded). q_k is again the number of all the key queries made by the adversary during security game. For our scheme, k is the number of OR, the disjunction, in the associated DNF formula.

4.3 From IBE To Oblivious Transfer

4.3.1 Oblivious Transfer Formalism and Main Issues

The Blind IBE (BIBE) we built can instantiate an efficient adaptive Oblivious Transfer, in this section we will see how. In the latest OT in the UC framework [CKWZ13, ABB⁺13, BC15], the server is required to send an encryption of the complete database for each line required by the user (thus $O(n)$ each time). We here give a protocol requiring $O(\log(n))$ for each line (except the first one, still in $O(n)$), in the UC framework with adaptive corruptions under classical assumptions (MDDH). Our protocol is an adaptive version of the one from [BC15] using an improvement of the ideas from [GH07].

The idea is to encrypt the database with an IBE (each line encrypted for the identity: "number of this line"), and send it once and for all to the recipient. Then the recipient will ask for IBE's private keys in an oblivious way. This second phase can be seen as an oblivious transfer on the database {users' secret keys}. Most of the reduction of the communication cost is done by using IBE with structure on the users' secret keys (Fragmented IBE for example).

Technical issue: In the Blind Fragmented IBKEM transformation, we use implicit decommitment. But in the UC framework it implies a very strong commitment primitive (formalized as SPHF-friendly commitments in [ABB⁺13]), which is both extractable and equivocable. Our idea is here to split these two properties by using on the one hand an equivocable commitment and on the other hand an (extractable) CCA encryption scheme by generalizing the way to access a line in the database. But this is infeasible with simple line numbers⁸. The change is tiny, we will not consider refer to a number for a target a line of the database (e.g. the identity) but we will use encode them: to request an a line we will commit a word W_i in the language $\mathcal{L}_i$.

4.3.2 High Level Idea of the Construction:

Our construction builds upon the UC-secure OT scheme from [BC15], with ideas inspired from [GH07], who propose a neat framework allowing to achieve *adaptive* Oblivious Transfer (but not in the UC framework). Their construction is quite simple: It requires a Blind property for the IBE allowing the recovery of the secret key to be oblivious. Note that this method can be seen as realizing an oblivious transfer with a database made of secret keys.

This approach is round-optimal: After the database preparation, the first flow is sent by the user as a commitment to the identity i , and the second one is sent by the server with the blinded expected information. There was several issues to have a UC-secure OT from this construction. We have to handle the possible adaptive corruption request from the adversary in the security games. To do so, as explained before, we use extractable and equivocable commitments using word in languages instead of simple number. Moreover using fragmented IBKEM and blinding them with our transformation we improve the efficiency of our scheme.

Precisely handling adaptive corruptions: To deal with corruptions of the user, recall that a simulated server (knowing the secret key of the encryption scheme) is already able to extract the identity committed to. But we now consider that, for all id , $\mathcal{L}_{\text{id}}$ is the language of the equivocable commitments on words in the inner language $\tilde{\mathcal{L}}_{\text{id}} = \{\text{id}\}$. We assume them to be a *Trapdoor Collection of Languages*, which means that it is computationally hard to sample an element in $\mathcal{L}_1 \cap \dots \cap \mathcal{L}_n$, except for the simulator, who possesses a trapdoor tk (the equivocation trapdoor) allowing it to sample an element in the intersection of languages. This allows a simulated user (knowing this trapdoor) not to really bind to any identity during the commitment phase. The only difference with the algorithm described in Figure 3.12 is that the user now encrypts this word W (which is an equivocable commitment on his identity id) rather than directly encrypting his identity id : $C = \text{Encrypt}_{\text{cca}}^\ell(W; \rho)$.

⁸Thus we have to very slightly change the scheme in figure 3.12

4.3.3 Generic Construction of Adaptive OT

We describe our generic construction of OT in Figure 4.9. We additionally assume the existence of a Pseudo-Random Generator (PRG) F with input size equal to the plaintext size, and output size equal to the size of the messages in the database and an IND-CPA encryption scheme $\mathcal{E} = (\text{Setup}_{\text{cpa}}, \text{KeyGen}_{\text{cpa}}, \text{Encrypt}_{\text{cpa}}, \text{Decrypt}_{\text{cpa}})$ with plaintext size at least equal to the security parameter. This additional step making use of the encryption scheme is aimed to handle corruptions of any party before the first flow. This leads to the following security result.

Theorem 10. *Assuming that BlindUSKGen is constructed as described above, the adaptive Oblivious Transfer protocol described in Figure 4.9 UC-realizes the functionality $\mathcal{F}_{\text{OT}}^{\mathcal{E}}$ presented in Figure 2.16 with adaptive corruptions assuming reliable erasures.*

Proof. We prove the security of this protocol via a sequence of games, starting from the real game, where the adversary $\mathcal{A}$ interacts with the real players, and ending with the ideal game, where we have built a simulator $\mathcal{S}$ that makes the interface between the ideal functionality $\mathcal{F}$ and the adversary $\mathcal{A}$. The simulator is explicitly given in Game 11. Recall that we consider adaptive corruptions.

We denote as $\mathcal{S}$ the server and U the user. The main idea is that, by assumption, the simulator can always obtain the common trapdoor tk of the collection of languages $(\mathcal{L}_1, \dots, \mathcal{L}_n)$ and use it to commit to a word simultaneously belonging to all the languages. In case of adaptive corruption, we face two cases: either the word was not correct, in which case, following the real protocol, the user should have erased his randomness, so that the simulator does not have anything to reveal. Or the word was correct and should belong to a certain language $\mathcal{L}_s$, and the simulator can then adapt the word and randomness so that it seems to belong to $\mathcal{L}_s$ (only). This enables us to avoid the use of commitments both extractable and equivocal (which is the usual tool for adaptive corruptions).

Due to the construction of the protocol, we have to prove that the user recovers the secret key $\text{usk}[s]$ corresponding to the s -th line of the database in an oblivious way, which means on the one hand that the user gains no information on the other keys, and on the other hand that the server gains no information on the key required by the user. Assuming this is the case (the proof follows), the adaptive security of the global oblivious transfer relies on the security of the underlying IBE scheme: The indistinguishability of the ciphertexts ensure that the user only recovers the s -th line for which he knows the secret key $\text{usk}[s]$.

Since the channels are authenticated, we know whether a flow was sent by an honest player (and received without any alteration) or not.

Game G_0 : This is the real game.

Game G_1 : In this game, the simulator generates correctly every flow on behalf of the honest players, as they would do themselves, knowing the

CRS generation:

$\text{crs} \xleftarrow{\$} \text{SetupCom}(1^{\mathfrak{R}}), \text{param}_{\text{cpa}} \xleftarrow{\$} \text{Setup}_{\text{cpa}}(1^{\mathfrak{R}})^a.$

Database Preparation:

1. Server runs $\text{Setup}(\mathfrak{R})$, to obtain mpk, msk .
2. For each line t , he computes $(D_t, K_t) = \text{Enc}(\text{mpk}, t)$, and $L_t = K_t \oplus \text{DB}(t)$.
3. He also computes $\text{usk}[i, b]$ for all $i = 1 \dots, m$ and $b = 0, 1$ and erases msk .
4. Server generates a key pair $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}_{\text{cpa}}(\text{param}_{\text{cpa}})$ for $\mathcal{E}$, stores sk and completely erases the random coins used by KeyGen .
5. He then publishes $\text{mpk}, \{(D_t, L_t)\}_t, \text{pk}$.

Index query on s :

1. User chooses a random value S , computes $R \leftarrow F(S)$ and encrypts S under pk :
 $c \xleftarrow{\$} \text{Encrypt}_{\text{cpa}}(\text{pk}, S)$
2. User computes $\mathcal{C}$ with the first flow of $\text{BlindUSKGen}(\langle (\mathcal{S}, \text{msk})(U, s, \ell; \boldsymbol{\rho}) \rangle)$ with $\ell = (\text{sid}, \text{ssid}, U, \mathcal{S})$ (see Figure 3.14).
3. User stores the random $\boldsymbol{\rho}_s = \{\boldsymbol{\rho}_*\}$ needed to open $\mathcal{C}$ to s , and completely erases the rest, including the random coins used by $\text{Encrypt}_{\text{cpa}}$ and sends $(c, \mathcal{C})$ to the Server

IBE input msk:

1. Server decrypts $S \leftarrow \text{Decrypt}_{\text{cpa}}(\text{sk}, c)$ and computes $R \leftarrow F(S)$
2. Server runs the second flow of $\text{BlindUSKGen}(\langle (\mathcal{S}, \text{msk})(U, s, \ell; \boldsymbol{\rho}) \rangle)$ on $\mathcal{C}$ (see Figure 3.14).
3. Server erases every new value except $(\text{hp}_{i,b})_{i,b}, (\text{busk}[i, b])_{i,b}, Z \oplus R$ and sends them over a secure channel.

Data recovery:

1. User then using, $\boldsymbol{\rho}_s$ recovers $\text{usk}[s]$ from the values received from the server.
2. He can then recover the expected information with $\text{Dec}(\text{usk}[s], s, D_s) \oplus L_s$ and erases everything else.

^ausing a commitment and a *cpa* secure encryption

FIGURE 4.9: ADAPTIVE UC-SECURE 1-OUT-OF- n OT FROM A FRAGMENTED BLIND IBE

database $(DB(1), \dots, DB(n))$ and the word W sent by the environment to the server and the user. In all the subsequent games, the players use the label $\ell = (\text{sid}, \text{ssid}, \mathcal{S}, U)$. In case of corruption of an honest player (either server or user), the simulator can give the internal data generated on behalf of the honest users.

Game G_2 : In this game, we replace the setup algorithm **Setup** by **SetupT**, allowing the existence of a trapdoor to find words in the intersection of the collection of languages. We also allow the simulator to program **Setup_{CCA}** in the CRS, enabling it to learn the extraction trapdoor of the CCA encryption scheme. The indistinguishability of the setups makes this game indistinguishable from the former one for the environment. Corruptions are handled the same way.

Game G_3 : We first deal with honest servers $\mathcal{S}$: he computes everything honestly during the database preparation. When receiving a commitment C , the simulator extracts the committed value W . By testing with the help of the algorithm **Verify**, it recovers s such that $W \in \mathcal{L}_s$. If it recovers $s \neq t$ such that $W \in \mathcal{L}_s \cap \mathcal{L}_t$, then the adversary has broken the infeasibility of finding a word in an intersection of languages without knowing the trapdoor and we abort the game. Otherwise, instead of computing the keys $H_{i,b}$, for $i = 1, \dots, \ell$ and $b = 0, 1$ with the hash function, the simulator then chooses $H_{i,b} \xleftarrow{\$} G$ when b is not equal to the i -th bit of W .

With an hybrid proof, applying the smoothness for every honest sender, on every index (i, b) such that $b \neq W_i$, since C is extracted to $W \in \mathcal{L}_i$, for any (i, b) such that $b \neq W_i$, the hash value is indistinguishable from a random value.

In case of corruption, everything has been erased (except after the pre-flow, where the simulator can reveal the keys (pk, sk) generated honestly). This game is thus indistinguishable from the previous one under the smoothness.

Game G_4 : Still in this case, when receiving a commitment C , the simulator extracts the committed value W . By testing with the help of the algorithm **Verify**, it recovers s such that $W \in \mathcal{L}_s$. Instead of proceeding as the server would do with $(\text{usk}[i, b])$, the simulator proceeds on $(\text{usk}'[i, b])$, with $\text{usk}'[i, b] = 0$ except if $b = W_i$. Since the masks $H_{i,b}$, for $b \neq W_i$, are random, this game is perfectly indistinguishable from the previous one.

Game G_5 : We now deal with honest users U : the simulator now uses the trapdoor tk to find a word W' in the intersection of all languages.

With an hybrid proof, applying a security game in each session in which the simulator does not know the trapdoor tk , one can show the indistinguishability of the two games. In case of corruption of the receiver, one learns the already known value W , thus s .

Game G_6 : We deal with the generation of R for honest servers $\mathcal{S}$ where the users U are honest: if $\mathcal{S}$ and U are honest at least until $\mathcal{S}$

received the second flow, the simulator sets $R = F(S')$ for both $\mathcal{S}$ and U , with S' a random value, instead of $R = F(S)$.

With an hybrid proof, applying the IND-CPA property for each session, one can show the indistinguishability of this game with the previous one.

Game G_7 : Still in the same case, the simulator sets R as a random value, instead of $R = F(S')$.

With an hybrid proof, applying the PRF property for each session, one can show the indistinguishability of this game with the previous one.

Game G_8 : We now deal with **the generation of H_{i,W_i} for honest servers $\mathcal{S}$ with honest users U** . Thanks to the additional random mask R , one can send random $(\text{usk}[i, W_i])_i$, and H_{i,W_i} can be computed later (when U actually receives its flow).

As above, but only if U has not been corrupted before receiving its flow, the simulator chooses $H_{i,W_i} \xleftarrow{\$} G$. With an hybrid proof, applying the pseudo-randomness, for every honest sender, the hash value is indistinguishable from a random value, because the adversary does not know any decommitment information for C . If the player U involved in the pseudo-randomness game gets corrupted (but the decommitment information is unknown) we are not in this case, and we can thus abort it.

In case of corruption of $\mathcal{S}$, everything has been erased (except after the pre-flow, where the simulator can reveal the keys (pk, sk) generated honestly).

In case of corruption of the receiver U , and thus receiving the value $\widetilde{DB}(s)$, the simulator computes $\widetilde{K}_s$ such that $\widetilde{K}_s \oplus \widetilde{DB}(s) = K_s \oplus DB(s)$ and the corresponding $\widetilde{\text{usk}}[W]$. It chooses R (because it was a random value unknown to the adversary and all the other $H_{i,b}$ are independent random values too) such that $\left(\bigoplus_i \text{busk}[i, W_i] \oplus H'_{i,W_i} \right) \oplus Z \oplus R = \widetilde{\text{usk}}[W]$.

This game is thus indistinguishable from the previous one under the pseudo-randomness.

Remark. We now explain how, in the pairing instantiation which will follow, given already sent values $\widetilde{D_s}, K_s \oplus DB(s)$, a simulator recovering from the environment the value $\widetilde{DB}(s)$, can adaptively be able to change his memory so as to compute a user key $\widetilde{\text{usk}}[W]$ such that $\text{Dec}(\widetilde{\text{usk}}[W], W, D_s) = DB(s) \oplus K_s \oplus \widetilde{DB}(s)$. This is exactly where we use the restriction on the size of DB elements so that we can manage to find a vector $\delta_s \in \mathbb{G}_2^{2k+1}$ such that $DB(s) \oplus \widetilde{DB}(s) = e(D_s, \delta_s)$. Thus, this allows the server to update his memory into $\widetilde{\text{usk}}[W] = \text{usk}[W] \cdot \delta_s$.

Game G_9 : Still in this case, the simulator proceeds on $(\text{usk}[i, b])$, with $\text{usk}[i, b] = 0$ for all i, b . Since the masks $H_{i,b}, Z \oplus R$, for any i, b , are independent random values (the $\text{busk}[i, b]$, for $b \neq W_i$ are independent random values, and

R is independently random), this game is perfectly indistinguishable from the previous one.

We remark that it is therefore no more necessary to know the index s given by the ideal functionality to the honest receiver U , to simulate $\mathcal{S}$ (but it is still necessary to simulate U).

Game G_{10} : We do not use anymore the knowledge of s when simulating an **honest user** U : the simulator generates a word W' in the intersection of the languages and $C \xleftarrow{\$} \text{Encrypt}_{\text{cca}}^\ell(W'; \rho)$, with $\ell = (\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{R})$, to send C during the first phase of honest users. This does not change anything from the previous game since the randomness needed to open to a word in another language is never revealed.

When it thereafter receives $(\text{Send}, \text{sid}, \text{ssid}, \mathcal{S}, \mathcal{R}, (\text{hp}_{i,b}, \text{busk}[i, b]))$ from the adversary, the simulator computes, for all lines t , $\text{usk}[W_t]$ and recovers K_t and finally $DB(t)$, which provides the database $(DB(1), \dots, DB(n))$ submitted by the sender. It uses them to send a **Send**-message to the ideal functionality.

Game G_{11} : We can now make use of the functionality, which leads to the following simulator:

- when receiving a **Send**-message from the ideal functionality, which means that an honest server has sent a pre-flow and a database, the simulator generates a key pair $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}(1^{\mathcal{K}})$ and sends pk as pre-flow;
- after receiving a pre-flow pk (from an honest or a corrupted sender) and a **Receive**-message from the ideal functionality, which means that an honest receiver has sent a first flow, the simulator generates a word W' in the intersection of languages, $C \xleftarrow{\$} \text{Encrypt}_{\text{cca}}^\ell(W; \rho)$ with $\ell = (\text{sid}, \text{ssid}, \mathcal{R}, \mathcal{S})$ and $c \xleftarrow{\$} \text{Encrypt}(\text{pk}, S)$ where S is a random value;
- when receiving a commitment C and a ciphertext c , generated by the adversary (from a corrupted receiver), the simulator extracts the committed value W and recovers s (aborting in case of multiple values), and uses it to send a **Receive**-message to the ideal functionality (and also decrypts the ciphertext c as S , and computes $R = F(S)$);
- when receiving $(\text{hp}_{i,b}, \text{busk}[i, b])$ from the adversary, the simulator computes, for all lines t , $\text{usk}[W_t]$ and recovers K_t and finally $DB(t)$, which provides the database $(DB(1), \dots, DB(n))$ submitted by the sender. It uses them to send a **Send**-message to the ideal functionality.
- when receiving a **Received**-message from the ideal functionality, together with $\widehat{DB}(s)$, on behalf of a corrupted receiver, from the extracted W leading to s , instead of proceeding as the sender would do on $(\text{usk}[i, b])$, the simulator proceeds on $(\text{usk}'[i, b])$, with $\text{usk}'[i, b] = 0$ except if $b = W_i$.

- when receiving a commitment C and a ciphertext c , generated by an honest sender (*i.e.*, by the simulator itself), the simulator proceeds as above on $(\text{usk}'[i, b])$, with $\text{usk}'[i, b] = 0$ except if $b = W_i$, but it chooses R uniformly at random instead of choosing it as $R = F(S)$; in case of corruption afterward, the simulator will adapt R such that $\left(\bigoplus_i \text{busk}[i, W_i] \oplus H'_{i, W_i}\right) \oplus Z \oplus R = \widetilde{\text{usk}[W]}$, where $\widetilde{\text{usk}[W]}$ leads to $\tilde{K}_s$ such that $\tilde{K}_s \oplus \widetilde{DB(s)} = K_s \oplus DB(s)$, where $\widetilde{DB(s)}$ is the message actually received by the receiver.

Any corruption either reveals s earlier, which allows a correct simulation of the receiver, or reveals $(DB(1), \dots, DB(n))$ earlier, which allows a correct simulation of the server. When the server has sent his flow, he has already erased all his random coins.

However, there would have been an issue when the user is corrupted after the server has sent his flow, but before the user receives it, since he has kept ρ : this would enable the adversary to recover $\text{usk}[W]$ from $\text{busk}[i, W_i]$ and hp_{i, W_i} . This is the goal of the ephemeral mask R that provides a secure channel.

□

Adaptive UC-Secure Oblivious Transfer. Using the instantiations of the fragmented IBE in figure 3.13 and the transformation 3.14, we finally get our instantiation by combining this $k - \text{MDDH}$ -based blind IBE with a $k - \text{MDDH}$ variant of El Gamal for the CPA encryption needed.

The requirement on the IBE blind user secret key generation (being able to adapt the key if the line changes) is achieved assuming that the server knows the discrete logarithms of the database lines⁹. For practical applications, one could imagine to split all 256-bit lines into 8 pieces for a decent/constant trade-off in favor of computational efficiency.

Efficiency: for $k = 1$, so under the classical SXDH assumption, the first flow requires $8 \log |DB|$ elements in $\mathbb{G}_1$ for the CCA encryption part and $\log(|DB| + 1)$ in $\mathbb{G}_2$ for the chameleon one, while the second flow would now require $1 + 4 \log |DB|$ elements in $\mathbb{G}_1$, $1 + 2 \log |DB|$ for the fragmented masked key, and $2 \log |DB|$ for the projection keys. As said before other OT scheme are under non classical security assumption [GH07] or have a communication cost linear in the size of the database (there is a comparison of non adaptive OT in Figure 5.11).

⁹This is quite easy to achieve by assuming that for all line s , $DB(s) = [db(s)]_1$ where $db(s)$ is the real line (thus known). It implies a few more computation on the user's side in order to recover $db(s)$ from $DB(s)$, but this remains completely feasible if the lines belong to a small space.

Chapter 5

Oblivious Transfer Generalization and New Approach

During the study of Oblivious as a stand alone (e.g. not as an IBE's application) we remarked that an OT and many other protocols such as Oblivious Signature-based Envelope¹ can be generalized into one protocol that we called Oblivious Language-based Envelope. We also give a general instantiation of this new kind of protocol. In this section too SPHF will be a very important tool for our instantiations. Secondly, we link two primitives: Password authenticated key exchange and Oblivious Transfer. Precisely we show how to transform an efficient PAKE into an efficient OT.

5.1 Oblivious Language-based Envelope

In this section we generalize many existing primitives such as Oblivious Signature-Based Envelope, Oblivious Transfer and even Blind IBKEM into a new protocol that we call Oblivious Language-based Envelope. First we describe what is an Oblivious Signature-Based Envelope to see the likelihood with our Oblivious transfer constructed in the previous Chapter. Then we describe our new protocol and give a generic instantiation secure in the UC-framework.

5.1.1 Oblivious Signature-Based Envelope

The construction from 4.3 opens new efficient applications to the already known Oblivious-Transfer protocols. But what happens when someone wants some additional access control by requesting extra properties, like if the user is only allowed to ask two lines with the same parity bits, the user can only request

¹defined in the next section.

lines for whose number has been signed by an authority, or even finer control provided through credentials? A first example could be with a database made of only one line but the access is delivered only if the recipient owns a special signature. This is called Oblivious Signature-based envelope and has already been studied in [LDB03,BPV12]. In other words in an OSBE protocol a sender $\mathcal{S}$ wants to send a private message $m \in \{0,1\}^{\mathfrak{K}}$ to a recipient $\mathcal{R}$ in possession of a valid certificate/signature on a public message M (given by a certification authority):

Definition 28 (Oblivious Signature-Based Envelope). *An OSBE scheme is defined by four algorithms (Setup, KeyGen, Sign, Verify), and one interactive protocol $\text{Protocol}(\mathcal{S}, \mathcal{R})$:*

- **Setup**($1^{\mathfrak{K}}$), where $\mathfrak{K}$ is the security parameter, generates the global parameters **param**;
- **KeyGen**($\mathfrak{K}$) generates the keys (vk, sk) of the certification authority;
- **Sign**(sk, M) produces a signature σ on the input message M , under the signing key sk ;
- **Verify**(vk, M, σ) checks whether σ is a valid signature on M , w.r.t. the public key vk ; it outputs 1 if the signature is valid, and 0 otherwise.
- **Protocol**($\mathcal{S}(\text{vk}, M, P), \mathcal{R}(\text{vk}, M, \sigma)$) between the sender $\mathcal{S}$ with the private message P , and the recipient $\mathcal{R}$ with a certificate σ . If σ is a valid signature under vk on the common message M , then $\mathcal{R}$ receives M , otherwise it receives nothing. In any case, $\mathcal{S}$ does not learn anything.

An OSBE scheme should fulfill the following security properties². To the best of our knowledge, no UC functionality has already been given.

- *correct*: the protocol actually allows $\mathcal{R}$ to learn P , whenever σ is a valid signature on M under vk ;
- *semantically secure*: the recipient learns nothing about $\mathcal{S}$'s input m if it does not use a valid signature σ on M under vk as input. More precisely, if $\mathcal{S}_0$ owns P_0 and $\mathcal{S}_1$ owns P_1 , the recipient that does not use a valid signature cannot distinguish an interaction with $\mathcal{S}_0$ from an interaction with $\mathcal{S}_1$ even if he has eavesdropped on several interactions $\langle \mathcal{S}(\text{vk}, M, P), \mathcal{R}(\text{vk}, M, \sigma) \rangle$ with valid signatures, and the same sender's input P ;
- *escrow-free (oblivious with respect to the authority)*: the authority (owner of the signing key sk), playing as the sender or just eavesdropping, is unable to distinguish whether $\mathcal{R}$ used a valid signature σ on M under vk as input.
- *semantically secure w.r.t. the authority*: after the interaction, the authority (owner of the signing key sk) learns nothing about m from a passive access to a challenge transcript³.

²The formal security games are given in [BPV12], in this thesis will formally recall this properties through the UC functionality of OLBE described in figure 5.1.

³This property is different from the semantic security described above, indeed in the context of OSBE the signer and the sender can be different entities

5.1.2 Definition of an Oblivious Language-Based Envelope

Oblivious Language-Based Envelope (OLBE) generalizes Oblivious Transfer and OSBE, for n messages (with n polynomial in the security parameter κ) to provide the best of both worlds.

Languages. Let X be a set of elements. The language $\mathcal{L} \subset X$ used in the definition of an OLBE should be a hard-partitioned subset of X , *i.e.* it is computationally hard to distinguish a random element in $\mathcal{L}$ from a random element not in $\mathcal{L}$. We here mainly consider languages which are hard-partitioned subsets, for instance, encryptions of publicly verifiable languages. These sublanguages should fulfill the following properties:

- *Publicly Verifiable:* Given a word x in X along with a witness w , anyone should be able to decide in polynomial time whether $x \in \mathcal{L}$ or not.
- *Self-Randomizable:* Given a word in the language, anyone should be able to sample a new word in the language⁴, and the distribution of this resampling should be indistinguishable from an honestly computed distribution. This will be used in order to prevent an adversary, or the authority in charge of distributing the words, to learn which specific form of the word was used by the user.

In case we consider several languages $(\mathcal{L}_1, \dots, \mathcal{L}_n)$, we also assume it is a *Trapdoor Collection of Languages*: It is computationally hard to sample an element in $\mathcal{L}_1 \cap \dots \cap \mathcal{L}_n$, except if one possesses a trapdoor tk (without the knowledge of the potential secret keys)⁵. For example, in the previous chapter we used this property to handle adaptive corruptions.

This concept of languages is very flexible and allows a lot of different applications to the concept of OLBE. For example we can consider a language where it is hard to create a word in the language⁶. This is the case for the OSBE protocol: the signature is given by the authority and under the unforgeability of the signature scheme, it is hard to create a fresh and valid signature.

OLBE. In such a protocol, a sender $\mathcal{S}$ wants to send one or several private messages to a recipient $\mathcal{R}$. $\mathcal{R}$ will be able to recover any message for which he possesses a word in the corresponding language. Let's say that the sender sends $n_{max} \leq n$ for a fixed n . In such a scheme, the languages $(\mathcal{L}_1, \dots, \mathcal{L}_n)$ are assumed to be a trapdoor collection of languages, publicly verifiable and self-randomizable.

Following the definitions for OSBE recalled above and given in [LDB03, BPV12], we give the following definition for OLBE. Due to the proximity of the primitives we shape our definition of OLBE as the definition of OSBE.

⁴It should be noted that this property is not incompatible with the potential secret key of the language in case it is keyed (see below).

⁵This implicitly means that the languages are compatible, in the sense that one can indeed find a word belonging to all of them.

⁶We refer to this kind of languages as *Keyed* languages

Definition 29 (Oblivious Language-based Envelope). An OLBE scheme is defined by five algorithms (Setup , SetupT , KeyGen , Samp , Verify), and one interactive protocol $\text{Protocol}(\mathcal{S}, \mathcal{R})$:

- $\text{Setup}(1^{\mathfrak{K}})$, where $\mathfrak{K}$ is the security parameter, generates the global parameters param , among which the numbers n and $n_{\max}$;
- or $\text{SetupT}(1^{\mathfrak{K}})$, where $\mathfrak{K}$ is the security parameter, additionally allows the existence⁷ of a trapdoor tk for the collection of languages $(\mathfrak{L}_1, \dots, \mathfrak{L}_n)$.
- $\text{KeyGen}(\text{param}, \mathfrak{K})$ generates, for all $i \in \{1, \dots, n\}$, the description of the language $\mathfrak{L}_i$ (as well as the language key $\text{sk}_{\mathfrak{L}_i}$ if need be). If the parameters param were defined by SetupT , this implicitly also defines the common trapdoor tk for the collection of languages $(\mathfrak{L}_1, \dots, \mathfrak{L}_n)$.
- $\text{Samp}(\text{param}, I)$ or $\text{Samp}(\text{param}, I, (\text{sk}_{\mathfrak{L}_i})_{i \in I})$ such that $I \subset \{1, \dots, n\}$ and $|I| = n_{\max}$, generates a list of words $(W_i)_{i \in I}$ such that $W_i \in \mathfrak{L}_i$ for all $i \in I$;
- $\text{Verify}_i(W_i, \mathfrak{L}_i)$ checks whether W_i is a valid word in the language $\mathfrak{L}_i$. It outputs 1 if the word is valid, 0 otherwise;
- $\text{Protocol}(\mathcal{S}((\mathfrak{L}_1, \dots, \mathfrak{L}_n), (m_1, \dots, m_n)), \mathcal{R}((\mathfrak{L}_1, \dots, \mathfrak{L}_n), (W_i)_{i \in I}))$, which is executed between the sender $\mathcal{S}$ with the private messages $(m_1, \dots, m_n)$ and corresponding languages $(\mathfrak{L}_1, \dots, \mathfrak{L}_n)$, and the recipient $\mathcal{R}$ with the same languages and the words $(W_i)_{i \in I}$ with $I \subset \{1, \dots, n\}$ and $|I| = n_{\max}$, proceeds as follows. For all $i \in I$, if the algorithm $\text{Verify}_i(W_i, \mathfrak{L}_i)$ returns 1, then $\mathcal{R}$ receives m_i , otherwise it does not. In any case, $\mathcal{S}$ does not learn anything.

The collections of words can be a single certificate/signature on a message M (encompassing OSBE, with $n = n_{\max} = 1$), a password, a credential, an equivocal commitment of a line number (encompassing our UC secure 1-out-of- n oblivious transfer, with $n_{\max} = 1$), k line numbers (encompassing k -out-of- n oblivious transfer, with $n_{\max} = k$), etc... Like OSBE and our adaptive OT, we allow a simulated setup SetupT to be run instead of the classical setup Setup in order to allow the simulator to possess some trapdoors.

5.1.3 Security Properties and Ideal Functionality of OLBE

We describe here the properties that an OLBE should satisfy, we further describe them more formally with an ideal functionality (see Figure 5.1):

- *correctness*: the protocol actually allows $\mathcal{R}$ to learn $(m_i)_{i \in I}$, whenever $(W_i)_{i \in I}$ are valid words of the languages $(\mathfrak{L}_i)_{i \in I}$, where $I \subset \{1, \dots, n\}$ and $|I| = n_{\max}$;
- *semantically secure (sem)*: the recipient learns nothing about the input m_i of $\mathcal{S}$ if it does not use a word in $\mathfrak{L}_i$. More precisely, if $\mathcal{S}_0$ owns $m_{i,0}$ and $\mathcal{S}_1$ owns $m_{i,1}$, the recipient that does not use a word in $\mathfrak{L}_i$ cannot

⁷The specific trapdoor will depend on the languages and be computed in the KeyGen algorithm.

distinguish between an interaction with $\mathcal{S}_0$ and an interaction with $\mathcal{S}_1$ even if the receiver has seen several interactions

$$\langle \mathcal{S}((\mathcal{L}_1, \dots, \mathcal{L}_n), (m_1, \dots, m_n)), \mathcal{R}((\mathcal{L}_1, \dots, \mathcal{L}_n), (W'_j)_{j \in I}) \rangle$$

with valid words $W'_i \in \mathcal{L}_i$, and the same sender's input m_i ;

- *escrow free (oblivious with respect to the authority)*: the authority corresponding to the language $\mathcal{L}_i$ (owner of the language secret key $\text{sk}_{\mathcal{L}_i}$ – if it exists), playing as the sender or just eavesdropping, is unable to distinguish whether $\mathcal{R}$ used a word W_i in the language $\mathcal{L}_i$ or not. This requirement also holds for anyone holding the trapdoor key tk .
- *semantically secure w.r.t. the authority (sem*)*: after the interaction, the trusted authority (owner of the language secret keys if they exist) learns nothing about the values $(m_i)_{i \in I}$ from the transcript of the execution. This requirement also holds for anyone holding the trapdoor key tk .

Moreover, the setup algorithms should be indistinguishable and it should be infeasible to find a word belonging to two or more languages without the knowledge of tk .

The functionality $\mathcal{F}_{\text{OLBE}}$ is parametrized by a security parameter κ and a set of languages $(\mathcal{L}_1, \dots, \mathcal{L}_n)$ along with the corresponding public verification algorithms $(\text{Verify}_1, \dots, \text{Verify}_n)$. It interacts with an adversary $\mathcal{S}$ and a set of parties $\mathcal{P}_1, \dots, \mathcal{P}_N$ via the following queries:

- **Upon receiving from party $\mathcal{P}_i$ an input of the form $(\text{Send}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, (m_1, \dots, m_n))$** , with $m_k \in \{0, 1\}^\kappa$ for all k : record the tuple $(\text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, (m_1, \dots, m_n))$ and reveal $(\text{Send}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j)$ to the adversary $\mathcal{S}$. Ignore further **Send**-message with the same **ssid** from $\mathcal{P}_i$.
- **Upon receiving an input of the form $(\text{Receive}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, (W_i)_{i \in I})$ where $I \subset \{1, \dots, n\}$ and $|I| = n_{\max}$ from party $\mathcal{P}_j$** : ignore the message if $(\text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, (m_1, \dots, m_n))$ is not recorded. Otherwise, reveal $(\text{Receive}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j)$ to the adversary $\mathcal{S}$ and send the message $(\text{Received}, \text{sid}, \text{ssid}, \mathcal{P}_i, \mathcal{P}_j, (m'_k)_{k \in I})$ to $\mathcal{P}_j$ where $m'_k = m_k$ if $\text{Verify}_k(W_k, \mathcal{L}_k)$ returns 1, and $m'_k = \perp$ otherwise. Ignore further **Received**-message with the same **ssid** from $\mathcal{P}_j$.

FIGURE 5.1: IDEAL FUNCTIONALITY FOR OBLIVIOUS LANGUAGE-BASED ENVELOPE $\mathcal{F}_{\text{OLBE}}$

The ideal functionality is inspired from the oblivious transfer functionality given in [Can01, CKWZ13, ABB⁺13]. As for oblivious transfer (Figure 2.16), we adapt them to the simple UC framework for simplicity.

Remark 6. *If we specify the language to a set of signature over a known message this ideal functionality becomes an ideal functionality for an OSBE scheme which has never been defined before.*

5.1.4 Generic UC-Secure Instantiation of OLBE with Adaptive Security

For simplicity we focus on the case $n_{\max} = 1$. It encompasses OSBE and 1-out-of- n OT. To get the more general cases one could run many $n_{\max} = 1$ -OLBE in parallel. This simplifies the algorithms **Samp** and **Verify**.

Building Blocks. We assume the existence of a labeled CCA-encryption scheme $\mathcal{E} = (\text{Setup}_{\text{cca}}, \text{KeyGen}_{\text{cca}}, \text{Encrypt}_{\text{cca}}^\ell, \text{Decrypt}_{\text{cca}}^\ell)$ ⁸ compatible with an SPHF onto a set G . In the **KeyGen** algorithm, the description of the languages $(\mathcal{L}_1, \dots, \mathcal{L}_n)$ thus implicitly defines the languages $(\mathcal{L}_1^c, \dots, \mathcal{L}_n^c)$ of CCA-encryptions of elements of the languages $(\mathcal{L}_1, \dots, \mathcal{L}_n)$.

We additionally use a key derivation function **KDF** to derive a pseudo-random bit-string $K \in \{0, 1\}^{\kappa}$ from a pseudo-random element $v \in G$.

We also assume the existence of a Pseudo-Random Generator (PRG) F with input size equal to the plaintext size, and output size equal to the size of the messages in the database and an IND-CPA encryption scheme $\mathcal{E} = (\text{Setup}_{\text{cpa}}, \text{KeyGen}_{\text{cpa}}, \text{Encrypt}_{\text{cpa}}, \text{Decrypt}_{\text{cpa}})$ with plaintext size at least equal to the security parameter.

We follow the ideas of the oblivious transfer constructions given in [ABB⁺13, BC15], giving the protocol presented on Figure 5.2. For the sake of simplicity, we only give the version for adaptive security, in which the sender generates a public key pk and ciphertext c to create a somewhat secure channel (they would not be used in the static version).

All the instantiations given are pairing-based but techniques explained in [BC15] could be used to rely on other families of assumptions, like decisional quadratic residue or even LWE.

Theorem 11. *The oblivious language-based envelope scheme described in Figure 5.2 is UC-secure in the presence of adaptive adversaries, assuming reliable erasures, an IND-CPA encryption scheme, and an IND-CCA encryption scheme admitting an SPHF on the language of valid ciphertexts of elements of $\mathcal{L}_i$ for all i ⁹.*

The proof of this theorem is close to the proof of the Theorem 10, the main difference comes from the fact that we do not have the blind IBE construction. The other difference is the nature of the protocols: the language for Oblivious transfer is precise (commitments of a number of line), but in the OLBE it is general. The techniques remain the same for the rest of the proof.

⁸the notion of labeled encryption is very close to a simple encryption. In a labelled encryption the algorithm **Encrypt** and **Decrypt** take as input a label. A ciphertext encrypted with a label ℓ can only be decrypted with the decryption algorithm using the same label.

⁹as soon as the languages are self-randomizable, publicly-verifiable and admit a common trapdoor

CRS: $\text{param} \xleftarrow{\$} \text{Setup}(1^{\mathbb{K}})$, $\text{param}_{\text{cca}} \xleftarrow{\$} \text{Setup}_{\text{cca}}(1^{\mathbb{K}})$, $\text{param}_{\text{cpa}} \xleftarrow{\$} \text{Setup}_{\text{cpa}}(1^{\mathbb{K}})$.

Pre-flow:

1. Sender generates a key pair $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}_{\text{cpa}}(\text{param}_{\text{cpa}})$ for $\mathcal{E}$, stores sk and completely erases the random coins used by KeyGen .
2. Sender sends pk to User.

Flow From the Receiver $\mathcal{R}$:

1. User chooses a random value J , computes $R \leftarrow F(J)$ and encrypts J under pk : $c \xleftarrow{\$} \text{Encrypt}_{\text{cpa}}(\text{pk}, J)$.
2. User computes $C \xleftarrow{\$} \text{Encrypt}_{\text{cca}}^{\ell}(W; r)$ with $\ell = (\text{sid}, \text{ssid}, \mathcal{R}, \mathcal{S})$.
3. User completely erases J and the random coins used by $\text{Encrypt}_{\text{cpa}}$ and sends C and c to Sender. He also checks the validity of his words: the receiver only keeps the random coins used by $\text{Encrypt}_{\text{cca}}$ for the j such that $\text{Verify}_j(W, \mathcal{L}_j) = 1$ (since he knows they will be useless otherwise).

Flow From the Sender $\mathcal{S}$:

1. Sender decrypts $J \leftarrow \text{Decrypt}_{\text{cpa}}(\text{sk}, c)$ and then $R \leftarrow F(J)$.
2. For all $j \in \{1, \dots, n\}$, sender computes $\text{hk}_j = \text{HashKG}(\ell, \mathcal{L}_j^c, \text{param})$, $\text{hp}_j = \text{ProjKG}(\text{hk}_j, \ell, (\mathcal{L}_j^c, \text{param}))$, $v_j = \text{Hash}(\text{hk}_j, (\mathcal{L}_j^c, \text{param}), (\ell, C))$, $Q_j = m_j \oplus \text{KDF}(v_j) \oplus R$.
3. Sender erases everything except $(Q_j, \text{hp}_j)_{j \in \{1, \dots, n\}}$ and sends them over a secure channel.

Message recovery:

Upon receiving $(Q_j, \text{hp}_j)_{j \in \{1, \dots, n\}}$, $\mathcal{R}$ can recover m_i by computing $m_i = Q_i \oplus \text{ProjHash}(\text{hp}_i, (\mathcal{L}_i^c, \text{param}), (\ell, C), r) \oplus R$.

FIGURE 5.2: UC-SECURE OLBE FOR ONE MESSAGE (SECURE AGAINST ADAPTIVE CORRUPTIONS)

Proof. We prove the adaptive security of this protocol via a sequence of games, starting from the real game, where the adversary $\mathcal{A}$ interacts with the real players, and ending with the ideal game, where we have built a simulator $\mathcal{S}$ that makes the interface between the ideal functionality $\mathcal{F}$ and the adversary $\mathcal{A}$.

The main idea is that, by assumption, the simulator can always obtain the common trapdoor tk of the collection of languages $(\mathfrak{L}_1, \dots, \mathfrak{L}_n)$ and use it to commit to a word simultaneously belonging to all the languages. In case of adaptive corruption, we face two cases: either the word was not correct, in which case, following the real protocol, the user should have erased his randomness, so that the simulator does not have anything to reveal. Or the word was correct and should belong to a certain language $\mathfrak{L}_i$, and the simulator can then adapt the word and randomness so that it seems to belong to $\mathfrak{L}_i$ (only). This enables us to avoid the use of commitments both extractable and equivocal (which is the usual tool for adaptive corruptions).

We say that a flow is *oracle-generated* if it was sent by an honest player (or the simulator) and received without any alteration by the adversary. It is said *non-oracle-generated* otherwise.

Game G_1 : This is the real game.

Game G_2 : In this game, the simulator generates correctly every flow on behalf of the honest players, as they would do themselves, knowing the inputs $(m_1, \dots, m_n)$ and W sent by the environment to the sender and the receiver. In all the subsequent games, the players use the label $\ell = (\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{R})$. In case of corruption, the simulator can give the internal data generated on behalf of the honest players.

Game G_3 : In this game, we replace the setup algorithm Setup by SetupT , allowing the existence of a trapdoor to find words in the intersection of the collection of languages. We also allow the simulator to program $\text{Setup}_{\text{CCA}}$ in the CRS, enabling it to learn the extraction trapdoor of the CCA encryption scheme. The indistinguishability of the setup makes this game indistinguishable from the former one for the environment. Corruptions are handled the same way.

Game G_4 : We first deal with oracle-generated flows from the senders $\mathcal{S}$: when receiving a commitment C , the simulator extracts the committed value W . By testing with the help of the algorithm Verify , it recovers i such that $W \in \mathfrak{L}_i$. If it recovers $i \neq j$ such that $W \in \mathfrak{L}_i \cap \mathfrak{L}_j$, then the adversary has broken the infeasibility of finding a word in an intersection of languages without knowing the trapdoor and we abort the game. Otherwise, instead of computing the key v_t , for $t = 1, \dots, n$ with the hash function, the simulator then chooses $v_t \xleftarrow{\$} G$ for $t \neq i$.

With an hybrid proof, applying the smoothness for every honest sender, on every index $t \neq i$, since C is extracted to $W \in \mathfrak{L}_i$, for any $t \neq i$, the hash value is indistinguishable from a random value.

In case of corruption, everything has been erased (except after the pre-flow, where the simulator can reveal the keys $(\mathbf{pk}, \mathbf{sk})$ generated honestly). This game is thus indistinguishable from the previous one under the smoothness.

Game G_5 : Still in this case, when receiving a commitment C , the simulator extracts the committed value W , giving it the number i . Instead of proceeding as the sender would do on $(m_1, \dots, m_n)$, the simulator proceeds on $(m'_1, \dots, m'_n)$, with $m'_i = m_i$, but $m'_t = 0$ for all $t \neq i$. Since the masks v_t , for $t \neq i$, are random, this game is perfectly indistinguishable from the previous one.

Game G_6 : We now deal with oracle-generated flows from the receivers $\mathcal{R}$: the simulator now uses the trapdoor $\mathbf{tk}$ to find a word W in the intersection of all languages.

With an hybrid proof, applying a security game in each session in which the simulator does not know the trapdoor $\mathbf{tk}$, one can show the indistinguishability of the two games. In case of corruption of the receiver, one learns the already known value i .

Game G_7 : We deal with **the generation of R for honest senders $\mathcal{S}$ on oracle-generated queries (adaptive case only)**: if $\mathcal{S}$ and $\mathcal{R}$ are honest at least until $\mathcal{S}$ received the second flow, the simulator sets $R = F(J')$ for both $\mathcal{S}$ and $\mathcal{R}$, with J' a random value, instead of $R = F(J)$.

With an hybrid proof, applying the IND-CPA property for each session, one can show the indistinguishability of this game with the previous one.

Game G_8 : Still in the same case, the simulator sets R as a random value, instead of $R = F(J')$.

With an hybrid proof, applying the PRF property for each session, one can show the indistinguishability of this game with the previous one.

Game G_9 : We now deal with **the generation of v_i for honest senders $\mathcal{S}$ on oracle-generated queries**:

- in the static case (the pre-flow is only needed to compute $(\mathbf{vk}, \mathbf{vtk})$, and thus we assume $R = 0$) the simulator chooses $v_i \xleftarrow{\$} G$ (for $t \neq i$, the simulator already chooses $v_t \xleftarrow{\$} G$), where i is the index given by the ideal functionality to the honest receiver $\mathcal{R}$.

With an hybrid proof, applying the pseudo-randomness for every honest sender, the hash value is indistinguishable from a random value, because the adversary does not know any decommitment information for C ;

- in the adaptive case, and thus with the additional random mask R , one can send a random m_i , and v_i can be computed later (when $\mathcal{R}$ actually receives its flow).

As above, but only if $\mathcal{R}$ has not been corrupted before receiving its flow, the simulator chooses $v_s \xleftarrow{\$} G$. With an hybrid proof, applying the

pseudo-randomness, for every honest sender, the hash value is indistinguishable from a random value, because the adversary does not know any decommitment information for C . If the player $\mathcal{R}$ involved in the pseudo-randomness game gets corrupted (but the decommitment information is unknown) we are not in this case, and we can thus abort it.

In case of corruption of $\mathcal{S}$, everything has been erased (except after the pre-flow, where the simulator can reveal the keys $(\mathbf{pk}, \mathbf{sk})$ generated honestly).

In case of corruption of the receiver $\mathcal{R}$, and thus receiving the value m_i , the simulator chooses R (because it was a random value unknown to the adversary and all the other v_t are independent random values too) such that

$$R \oplus \text{ProjHash}(\text{hp}_i, (L_i, \text{param}), (\ell, C), r_i) \oplus m_i = Q_i.$$

This game is thus indistinguishable from the previous one under the pseudo-randomness.

Game G_{10} : Still in this case, the simulator proceeds on $(m'_1, \dots, m'_n)$, with $m'_t = 0$ for all i . Since the masks $v_t \oplus R$, for any $t = 1, \dots, n$, are independent random values (the v_t , for $t \neq i$ are independent random values, and v_i is also independently random in the static case, while R is independently random in the adaptive case), this game is perfectly indistinguishable from the previous one.

We remark that it is therefore no more necessary to know the index i given by the ideal functionality to the honest receiver $\mathcal{R}$, to simulate $\mathcal{S}$ (but it is still necessary to simulate $\mathcal{R}$).

Game G_{11} : We do not use anymore the knowledge of i when simulating an **honest receiver $\mathcal{R}$** : the simulator generates a word W in the intersection of the languages and $C \xleftarrow{\$} \text{Encrypt}_{\text{cca}}^\ell(W; r)$, with $\ell = (\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{R})$, to send C during the first phase of honest receivers. This does not change anything from the previous game since the randomness needed to open to a word in another language is never revealed.

When it thereafter receives $(\text{Send}, \text{sid}, \text{ssid}, \mathcal{S}, \mathcal{R}, (Q_1, \text{hp}_1, \dots, Q_n, \text{hp}_n))$ from the adversary, the simulator computes, for $i = 1, \dots, k$, r_i ,

$$v_i \leftarrow \text{ProjHash}(\text{hp}_i, (\mathcal{L}_i, \text{param}), (\ell, C), r_i)$$

and $m_i = v_i \oplus R \oplus Q_i$. This provides the database submitted by the sender.

Game G_{12} : We can now make use of the functionality, which leads to the following simulator:

- when receiving a **Send**-message from the ideal functionality, which means that an honest sender has sent a pre-flow, the simulator generates a key pair $(\mathbf{pk}, \mathbf{sk}) \xleftarrow{\$} \text{KeyGen}(1^{\mathbb{K}})$ and sends $\mathbf{pk}$ as pre-flow;

- after receiving a pre-flow pk (from an honest or a corrupted sender) and a **Receive**-message from the ideal functionality, which means that an honest receiver has sent a first flow, the simulator generates a word W in the intersection of languages, $C \xleftarrow{\$} \text{Encrypt}_{\text{cca}}^\ell(W; r)$ with $\ell = (\text{sid}, \text{ssid}, \mathcal{R}, \mathcal{S})$ and $c \xleftarrow{\$} \text{Encrypt}(\text{pk}, J)$ where R is a random value;
- when receiving a commitment C and a ciphertext c , generated by the adversary (from a corrupted receiver), the simulator extracts the committed value W and recovers i (aborting in case of multiple values), and uses it to send a **Receive**-message to the ideal functionality (and also decrypts the ciphertext c as J , and computes $R = F(J)$);
- when receiving $(Q_1, \text{hp}_1, \dots, Q_n, \text{hp}_n)$ from the adversary (a corrupted sender), the simulator computes, for $i = 1, \dots, n$, r_i ,

$$v_i \leftarrow \text{ProjHash}(\text{hp}_i, (\mathcal{L}_i, \text{param}), (\ell, C), r_i)$$

and $m_i = v_i \oplus R \oplus Q_i$. It uses them to send a **Send**-message to the ideal functionality.

- when receiving a **Received**-message from the ideal functionality, together with m_i , on behalf of a corrupted receiver, from the extracted W leading to i , instead of proceeding as the sender would do on $(m_1, \dots, m_n)$, the simulator proceeds on $(m'_1, \dots, m'_n)$, with $m'_i = m_i$, but $m'_j = 0$ for all $j \neq i$;
- when receiving a commitment C and a ciphertext c , generated by an honest sender (*i.e.*, by the simulator itself), the simulator proceeds as above on $(m'_1, \dots, m'_n)$, with $m'_j = 0$ for all j , but it chooses R uniformly at random instead of choosing it as $R = F(J)$; in case of corruption afterward, the simulator will adapt R such that $R \oplus \text{ProjHash}(\text{hp}_i, (\mathcal{L}_i, \text{param}), (\ell, C), r_i) \oplus Q_i = m_i$, where m_i is the message actually received by the receiver.

Any corruption either reveals i earlier, which allows a correct simulation of the receiver, or reveals $(m_1, \dots, m_n)$ earlier, which allows a correct simulation of the sender. When the sender has sent his flow, he has already erased all his random coins.

However, there would have been an issue when the receiver is corrupted after the sender has sent his flow, but before the receiver receives it, since he has kept r_i : this would enable the adversary to recover m_i from Q_i and hp_i . This is the goal of the ephemeral mask R that provides a secure channel.

□

5.1.5 Oblivious Primitives Obtained by the Framework

More than the two examples already discussed in this chapter, the framework also enables us to give a new instantiation of Access Controlled Oblivious Transfer under classical assumptions [CDN09]. In such a primitive, the user does not

automatically gets the line he asks for, but has to prove that he possesses one of the credential needed to access this particular line.

5.2 Very Efficient Oblivious Transfer

The key block needed for our construction of an oblivious transfer/OLBE is the Smooth Projective Hash Functions. It allows to compute a value by two different ways. We can notice that the 2 parties never know if they computed the same value or not. SPHF are heavy tools, very powerful but they come with high computation and communication costs. In this section we investigate a way to replace the role of the SPHF with a lighter primitive. We will first construct OT from PAKE. Indeed PAKE fulfil the property discussed before (the values are equal if they used the same password) and there exists instantiations of PAKE that does not involve any SPHF [JR15].

A second motivation is the willingness to link Oblivious Transfer and PAKE. Finding the link between primitives and showing which primitives can be used to build other ones is a useful trend in proven cryptography. This has been done many times before (IBE-Signature, IBE-IBS,...) and even during this PhD (ABE-IBE, IBE-Fragmented Blind IBE, Fragmented Blind IBE-OT). We remark that many transformations involved IBE confirming the central place of this primitive.

This section will be divided into 4 parts. First we will define the primitive PAKE and the security requirements for it. Then we will see how to generically transform a PAKE into an Oblivious Transfer and afterwards apply it on different examples. Finally we will apply our transformation on a QA-NIZK-based PAKE to obtain our very efficient OT.

5.2.1 Password Authenticated Key Exchange

Roughly speaking a PAKE is an interactive protocol allowing two parties to share a common secret by using passwords.

We present the PAKE ideal functionality $\mathcal{F}_{pwKE}$ on Figure 5.3. It was described in [CHK⁺05]. The main idea behind this functionality is as follows: If neither party is corrupted and the adversary does not attempt any password guess, then the two players both end up with either the same uniformly-distributed session key if the passwords are the same, or uniformly-distributed independent session keys if the passwords are distinct. In addition, the adversary does not know whether this is a success or not. However, if one party is corrupted, or if the adversary successfully guessed the player's password (the session is then marked as **compromised**), the adversary is granted the right to fully determine its session key. There is in fact nothing lost by allowing it to determine the key. In case of a wrong guess (the session is then marked as **interrupted**), the two players are given independently-chosen random keys. A session that is not **compromised** nor **interrupted** is called **fresh**, which is its initial status.

The functionality $\mathcal{F}_{\text{pwKE}}$ is parametrized by a security parameter k . It interacts with an adversary $\mathcal{S}$ and a set of parties $P_1, \dots, P_n$ via the following queries:

- **Upon receiving a query (NewSession, sid, ssid, P_i, P_j , pw) from party P_i :**

Send (NewSession, sid, ssid, P_i, P_j) to $\mathcal{S}$. If this is the first NewSession query, or if this is the second NewSession query and there is a record (sid, ssid, P_j, P_i , pw'), then record (sid, ssid, P_i, P_j , pw) and mark this record **fresh**.

- **Upon receiving a query (TestPwd, sid, ssid, P_i , pw') from the adversary $\mathcal{S}$:**

If there is a record of the form (P_i, P_j , pw) which is **fresh**, then do: If pw = pw', mark the record **compromised** and reply to $\mathcal{S}$ with “correct guess”. If pw $\neq$ pw', mark the record **interrupted** and reply with “wrong guess”.

- **Upon receiving a query (NewKey, sid, ssid, P_i , sk) from the adversary $\mathcal{S}$:**

If there is a record of the form (sid, ssid, P_i, P_j , pw), and this is the first NewKey query for P_i , then:

- If this record is **compromised**, or either P_i or P_j is corrupted, then output (sid, ssid, sk) to player P_i .
- If this record is **fresh**, and there is a record (P_j, P_i , pw') with pw' = pw, and a key sk' was sent to P_j , and (P_j, P_i , pw) was **fresh** at the time, then output (sid, ssid, sk') to P_i .
- In any other case, pick a new random key sk' of length κ and send (sid, ssid, sk') to P_i .

Either way, mark the record (sid, ssid, P_i, P_j , pw) as **completed**.

FIGURE 5.3: IDEAL FUNCTIONALITY FOR PAKE $\mathcal{F}_{\text{pwKE}}$

The passwords are supposed to be distributed before the protocol, our security holds for every distribution of passwords.

Before the UC framework, the classical security model for PAKE protocols was the BPR model [BPR00]. The first ideal functionality for PAKE protocols in the UC framework [Can01, CK02] was proposed by Canetti *et al.* [CHK⁺05], who showed how a simple variant of the Gennaro-Lindell methodology [GL03] could lead to a secure protocol¹⁰.

In case of corruption, the adversary learns the password of the corrupted player; After the **NewKey**-query, it additionally learns the session key.

Description of a UC-Secure PAKE Scheme

For sake of simplicity we are going to consider PAKE protocols in two flows, since there already exists a multitude of PAKE schemes satisfying this requirement, and since minimizing the number of flows is one of the most important issues in modern instantiations. This allows us to generically give an optimization, leading to more efficient protocols.

In order to present the framework of our transformation, we formally split a PAKE scheme into the following four algorithms (note that many algorithm use two random element, one for each password): .

- **Setup**: An algorithm that generates the initial **crs**.
- **flow₁(pw_i; ρ_i, ρ'_i)**: The algorithm run by the user P_i , using some randomness ρ_i and ρ'_i and possessing the password pw_i , to generate the first flow of the protocol. We note ρ_i the part of the randomness involved in hiding the password, and ρ'_i the remainder.
- **flow₂(flow₁, pw_j; ρ_j, ρ'_j)**: The algorithm run by the user P_j , using some randomness ρ_j and possessing the password pw_j , after receiving the flow flow_1 , to generate the second flow of the protocol. This algorithm also produces P_j 's view of the key: $K_i^{(j)}$.
- **Decap(flow₂, pw_i, f(ρ_i, ρ'_i))**: The algorithm run by P_i when receiving the flow flow_2 from P_j , using his password pw_i and a function $f(\rho_i, \rho'_i)$ of the randomness initially used and the remainder (which function can be anything from the identity to the null function), to recover P_i 's view of the key: $K_j^{(i)}$.

5.2.2 Generic Construction of a UC-Secure OT Scheme

In Canetti et al. in [CDVW12], they show how a UC-secure Oblivious Transfer scheme can be transformed into a UC-secure PAKE scheme. In this section we are going to go the other way, and show how a UC-secure PAKE can be transformed into a UC-secure Oblivious Transfer scheme. While considering this direction may seem like transforming a very strong primitive into a weaker one, this will

¹⁰we will later apply our transformation to a PAKE secure in the BPR model to show that our transformation can apply also in a lattice-based instantiation

allow us to prepare a framework to propose the most efficient Oblivious Transfer scheme to date.

We denote by $\text{DB} = (L_1, \dots, L_n)$ the database of the server containing n lines, and by s the line requested by the user in an oblivious way. As for OT and OLBE, we assume the existence of a CPA-Secure encryption scheme $(\text{Setup}_{\text{cpa}}, \text{KeyGen}_{\text{cpa}}, \text{Encrypt}_{\text{cpa}}, \text{Decrypt}_{\text{cpa}})$ ¹¹ and the existence of a Pseudo-Random Generator (PRG) F . The public parameters of the encryption scheme are going to be included in the public common reference string crs generated by the setup algorithm of the PAKE scheme.

Using the PAKE scheme, the encryption scheme and the PRG, one can now obtain an OT scheme from a PAKE scheme, as described in Figure 5.4. The idea of the protocol is as follows:

- First, a setup phase enables to generate the CRS, both for the PAKE and encryption schemes.
- The pre-flow is a technical requirement for the UC proof.
- When querying for line s , the receiver proceeds in three steps. The first one consists in generating a masking value R (technical requirement for the UC proof), the second one consists in running the first flow of the PAKE scheme for password s to generate the first flow denoted¹² as flow_0 , and the third one consists in erasing anything but the needed values, and sending flow_0 to the sender.
- When answering to this query, the sender also proceeds in three steps. The first one consists in recovering the value R and the second one consists in running, for each line $k \in \{1, \dots, n\}$, the second flow of the PAKE scheme to generate the second flow denoted as flow_k (using k as the password) as well as $\mathcal{S}$'s view of the session key, denoted as $(K_k)_{\mathcal{R}}^{(\mathcal{S})}$. Finally, in the third step, the server computes a xor Q_k of the line L_k and this session key and the value R and sends the set (flow_k, Q_k) back to $\mathcal{R}$.
- When receiving this set of values, the receiver uses flow_s to run the decapsulation algorithm of the PAKE scheme (with password s) to recover his view of the session key, denoted as $(K_s)_{\mathcal{S}}^{(\mathcal{R})}$. He finally uses R , Q_s and $(K_s)_{\mathcal{S}}^{(\mathcal{R})}$ to recover the expected line L_s .

The correctness easily follows from the correctness of the PAKE protocol, since the views of $\mathcal{R}$ and $\mathcal{S}$ of the session key for the PAKE scheme executed for password s are the same. The scheme is oblivious with respect to $\mathcal{S}$ since the first flow of the PAKE scheme executed by $\mathcal{R}$ does not reveal any information on s . And it is oblivious with respect to $\mathcal{R}$ since the correctness of the PAKE scheme gives him no information on the session keys $(K_k)_{\mathcal{R}}^{(\mathcal{S})}$ for $k \neq s$. The security is stated in Theorem 12.

¹¹Here again this encryption scheme handle corruptions before the first flow.

¹²Note that we now denote as flow_0 (and not flow_1) the flow generated by the PAKE algorithm flow_1 , in order to avoid the confusion with the flow_1 message generated by the sender (for line 1) during the database answer phase.

CRS generation:

$\text{crs} \xleftarrow{\$} \text{Setup}(1^{\mathfrak{K}}), \text{param}_{\text{cpa}} \xleftarrow{\$} \text{Setup}_{\text{cpa}}(1^{\mathfrak{K}}).$

Pre-Flow:

1. $\mathcal{S}$ generates a key pair $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}_{\text{cpa}}(\text{param}_{\text{cpa}})$ for $\mathcal{E}$, stores sk and completely erases the random coins used by $\text{KeyGen}_{\text{cpa}}$.
2. $\mathcal{S}$ publishes pk .

Index query on s :

1. $\mathcal{R}$ picks a random J , computes $R \leftarrow F(J)$ and sets $c \xleftarrow{\$} \text{Encrypt}_{\text{cpa}}(\text{pk}, J)$.
2. $\mathcal{R}$ picks a random (ρ, ρ') , and runs $\text{flow}_1(s; \rho, \rho')$ to obtain flow_0 .
3. $\mathcal{R}$ stores $f(\rho, \rho')$ needed for the Decap algorithm and R and completely erases the rest, including the random coins used by $\text{Encrypt}_{\text{cpa}}$ and sends (c, flow_0) to $\mathcal{S}$.

Database answer:

1. $\mathcal{S}$ decrypts $J \leftarrow \text{Decrypt}_{\text{cpa}}(\text{sk}, c)$ and computes $R \leftarrow F(J)$.
2. For each line k :
 - $\mathcal{S}$ picks a random (ρ_k, ρ'_k) , runs $\text{flow}_2(\text{flow}_0, k; \rho_k, \rho'_k)$ to generate flow_k and $(K_k)_{\mathcal{R}}^{(\mathcal{S})}$.
 - $\mathcal{S}$ then computes $Q_k = L_k \oplus (K_k)_{\mathcal{R}}^{(\mathcal{S})} \oplus R$.
3. $\mathcal{S}$ erases everything except $(\text{flow}_k, Q_k)_{k \in \llbracket 1, n \rrbracket}$ and sends it to $\mathcal{R}$.

Data recovery:

$\mathcal{R}$ then using $f(\rho, \perp)$ computes $(K_s)_{\mathcal{S}}^{(\mathcal{R})} = \text{Decap}(\text{flow}_s, s, f(\rho, \rho'))$, sets $L_s = Q_s \oplus (K_s)_{\mathcal{S}}^{(\mathcal{R})} \oplus R$, and erases everything else.

FIGURE 5.4: UC-SECURE 1-OUT-OF- n OT FROM A UC-SECURE PAKE

Theorem 12. *Under the UC-security¹³ of the PAKE protocol, the existence of a Pseudo-Random Generator (PRG) F with input size equal to the plaintext size, and output size equal to the size of the messages in the database, and the IND-CPA security of the encryption scheme, the transformation presented in Figure 5.4 achieves a UC-secure 1-out-of- n Oblivious Transfer scheme, with the same handling of corruptions as in the PAKE protocol.*

The high-level idea of the proof is quite simple. Recall that in a UC proof, one has to simulate one of the two players in front of a corrupted player. The first flow allows the simulated server to extract the requested line. One will then be able to send random noise on all the lines besides the correct one. The PAKE functionality ensures that this is possible. Then again for honest users, the simulator will replace the remaining line by a random noise. In case of corruptions of the server, one relies on the security of the CPA encryption, to ensure that everything was done honestly (the adversary is not able to test the validity of the retrieved R , and so for the correct line as $L \oplus K \oplus R$ where L and K are the honest values, and R is simply set as $Q \ominus (L \oplus K)$). In case of corruption of the user, we rely on the PAKE handling of user corruption to adapt the memory.

Proof. To prove this theorem, we exhibit a sequence of games. The sequence starts from the real game, where the adversary $\mathcal{A}$ interacts with real players and ends with the ideal game, where we have built a simulator $\mathcal{S}$ that makes the interface between the ideal functionality $\mathcal{F}$ and the adversary $\mathcal{A}$. We prove the adaptive version of the protocol. The proof of the static framework version can be obtained by removing the parts related to adaptive version from the proof below.

When the sender submits its values, the simulator can extract all the message thanks to the crs trapdoor and get the witnesses for each indices. This allows to simulate the **Send-query** to the ideal functionality.

Eventually, when simulating the honest senders, the simulator recovers the expected line value s from flow_0 , to set flow_s and L_s honestly, the other values can be random. More details follow:

Game G_1 : This is the real game.

Game G_2 : In this game, the simulator generates correctly every flow from the honest players, as they would do themselves, knowing the inputs $(L_1, \dots, L_n)$ and s sent by the environment to the sender and the receiver. In all the subsequent games, the players use the label $\ell = (\text{sid}, \text{ssid}, P_i, P_j)$. In case of corruption, the simulator can give the internal data generated on behalf of the honest players.

¹³Note that the transformation also allows to obtain an OT scheme from a BPR-secure PAKE scheme in a game-based security model, but since this is of less interest, we do not formally prove it due to lack of space.

Game G_3 : In this game, we just replace the **Setup** algorithm (if any), by the one used in the **PAKE** proof, as this is transparent in the **PAKE** protocol, this is also transparent in the Oblivious Transfer.

Game G_4 : We first deal with **honest senders** P_i : when receiving a first flow flow_0 , the simulator recovers the prospective line value s (this is because, we assume the UC-PAKE secure, hence after flow_1 , the simulator should be able to send $\text{TestPwd}, \text{sid}, \text{ssid}, P_i, s$, hence knows said s). Instead of computing the key K_t , for $t = 1, \dots, n$ for all possible line/passwords, it chooses K_t at random in the key space for $t \neq s$.

With an hybrid proof, for every honest sender, on every index $t \neq s$, since flow_0 leads to s , for any $t \neq s$, the new key is indistinguishable from a random value, under the **PAKE** security.

In case of corruption, everything is either already handled by the **PAKE**, or erased. This game is thus indistinguishable from the previous one under the **PAKE** functionality.

Game G_5 : Still in this case, when receiving flow_0 , the simulator recovers the expected value s . Instead of proceeding as the sender would do on lines $(L_1, \dots, L_n)$, the simulator proceeds on $(L'_1, \dots, L'_n)$, with $L'_s = L_s$, but $m'_t = 0$ for all $t \neq s$. Since the masks K_t , for $t \neq s$, are random, this game is perfectly indistinguishable from the previous one.

Game G_6 : We now deal with **honest receivers** P_j : we replace flow_1 for in Step 1 of the index query phase of honest receivers by simulated flow_1 , using the fact that the UC-Proof requires the simulator at this step to send a non committing flow compatible with every possible password, and keep the **PAKE** memory.

Using the **PAKE** security, one can show the indistinguishability of the two games. In case of corruption of the receiver, one learns the already known value s .

Game G_7 : We deal with **the generation of R for honest senders P_i on honestly-generated queries (adaptive case only)**: if P_i and P_j are honest at least until P_i received the second flow, the simulator sets $R = F(S')$ for both P_i and P_j , with S' a random value, instead of $R = F(S)$.

With an hybrid proof, applying the IND-CPA property for each session, one can show the indistinguishability of this game with the previous one.

Game G_8 : Still in the same case, the simulator sets R as a random value, instead of $R = F(S')$.

With an hybrid proof, applying the PRF property for each session, one can show the indistinguishability of this game with the previous one.

Game G_9 : We now deal with **the generation of K_s for honest senders P_i on honestly-generated queries**:

- in the static case (the pre-flow is not necessary, and thus we assume $R = 0$) the simulator chooses K_s at random in the Key Space (for $t \neq s$, the simulator already chooses K_t random), where s is the index given by the ideal functionality to the honest receiver P_j .

Under the PAKE security, for every honest sender, the key value is indistinguishable from a random value, because the adversary does not know any secret information on flow_0 ;

- in the adaptive case, and thus with the additional random mask R , one can send a random Q_s , and K_s can be computed later (when P_j actually receives its flow, and the simulator possibly learns the expected L_s).

As above, but only if P_j has not been corrupted before receiving its flow, the simulator chooses K_s at random. Once again, invoking the PAKE security, and the lack of secret information known by the adversary, this value is indistinguishable from a random value. If the player P_j gets corrupted we are not in this case, and we can thus abort it.

In case of corruption of P_i , everything has been erased and / or handled by the PAKE simulator. In case of corruption of the receiver P_j , and thus receiving the value L_s , the simulator chooses R (because it was a random value unknown to the adversary and all the other K_t are independent random values too) such that $R \oplus K_s \oplus Q_s = L_s$.

This game is thus indistinguishable from the previous one under the PAKE Security (External Adversary, Adaptive Corruption of the second User).

Game G_{10} : Still in this case, the simulator proceeds on $(L'_1, \dots, L'_n)$, with $L'_t = 0$ for all t . Since the masks $K_t \oplus R$, for any $t = 1, \dots, n$, are independent random values (the K_t , for $t \neq s$ are independent random values, and K_s is also independently random in the static case, while R is independently random in the adaptive case), this game is perfectly indistinguishable from the previous one.

We remark that it is therefore no more necessary to know the index s given by the ideal functionality to the honest receiver P_j , to simulate P_i (but it is still necessary to simulate P_j).

Game G_{11} : We do not use anymore the knowledge of s when simulating an **honest receiver** P_j : the simulator generates an equivocal first flow flow_0 during the index query phase of honest receivers. We essentially break the atomic flow_1 from the PAKE in the two separated processes **SimFlow** and **OpenFlow**, used by the simulator in the PAKE proof, and stores the equivocation information Λ , instead of just $f(\rho)$. This does not change anything from the previous game since the memory before flow_0 is never revealed. It can be updated with correct values in case of corruption of the receiver.

When it thereafter receives the message $(\text{Send}, \text{sid}, \text{ssid}, P_i, P_j, (Q_1, \dots, Q_n))$ from the adversary, the simulator computes, for $i = 1, \dots, n$, the value $f(\rho_i) \leftarrow \text{OpenFlow}^\ell(\text{flow}_0, i, \Lambda)$, $K_i \leftarrow \text{Decap}(\text{flow}_i, i, f(\rho_i))$ and $m_i = K_i \oplus R \oplus Q_i$. This provides the database submitted by the sender.

Game G_{12} : We can now make use of the functionality, which leads to the following simulator:

- when receiving a **Send**-message from the ideal functionality, which means that an honest sender has sent a pre-flow, the simulator generates a key pair $(pk, sk) \xleftarrow{\$} \text{KeyGen}(1^{\mathcal{K}})$ and sends pk as pre-flow;
- after receiving a pre-flow pk (from an honest or a corrupted sender) and a **Receive**-message from the ideal functionality, which means that an honest receiver has sent an index query, the simulator generates $(\text{flow}_0, \Lambda) \xleftarrow{\$} \text{SimFlow}^\ell()$ and $c \xleftarrow{\$} \text{Encrypt}(pk, S)$, with $\ell = (\text{sid}, \text{ssid}, P_i, P_j)$ and R a random value, to send flow_1 and c during the index query phase of the honest receiver;
- when receiving flow_0 and a ciphertext c , generated by the adversary (from a corrupted receiver), the simulator recovers the committed value s , and uses it to send a **Receive**-message to the ideal functionality (and also decrypts the ciphertext c as S , and computes $R = F(S)$);
- when receiving $(Q_1, \dots, Q_n)$ from the adversary (a corrupted sender), the simulator computes, for $i = 1, \dots, n$, $f(\rho_i) \leftarrow \text{OpenFlow}^\ell(\text{flow}_0, i, \Lambda)$, $K_i \leftarrow \text{Decap}(\text{flow}_i, i, f(\rho_i))$ and $L_i = K_i \oplus R \oplus Q_i$. It uses them to send a **Send**-message to the ideal functionality.
- when receiving a **Received**-message from the ideal functionality, together with L_s , on behalf of a corrupted receiver, from the recovered s , instead of proceeding as the sender would do on $(L_1, \dots, L_n)$, the simulator proceeds on $(L'_1, \dots, L'_n)$, with $L'_s = L_s$, but $L'_i = 0$ for all $i \neq s$;
- when receiving a a flow flow_0 and a ciphertext c , generated by an honest sender (*i.e.*, by the simulator itself), the simulator proceeds as above on $(L'_1, \dots, L'_k)$, with $L'_i = 0$ for all i , but it chooses R uniformly at random instead of choosing it as $R = F(S)$; in case of corruption afterward, the simulator will adapt R such that $R \oplus K_s \oplus Q_s = L_s$, where L_s is the message actually received by the receiver.

Any corruption either reveals s earlier, which allows a correct simulation of the receiver, or reveals $(L_1, \dots, L_n)$ earlier, which allows a correct simulation of the sender. When the sender has sent his flow, he has already erased all his random coins (or at least enough to be compatible with the PAKE key). However, there would have been an issue when the receiver is corrupted after the sender has sent is flow, but before the receiver receives it, since he has kept some witness $f(\rho)$: this would enable the adversary to recover L_s from Q_s . This is the goal of the ephemeral mask R that provides a secure channel.

As a consequence, the distance between the first and the last games is bounded by q times the PAKE security, and the encryption CPA indistinguishability.

□

Generic Optimization

In the previous transformation we never used the fact that the first user in the PAKE scheme (here, the receiver) does not learn the password from the second

(here, the sender). This is a property required by PAKE but not useful in the transformation (since the sender executes n parallel PAKE schemes, using the (public) number k of the lines as the password). This argument is the key of the optimization we now propose in Figure 5.5. The difference is that in the index query, the receiver does not pick the second randomness ρ' and in the database answer, the randomness ρ_k is not used anymore. The proof remains the same as the previous one.

5.2.3 A Warm Up

Since in [ABB⁺13], the authors proposed both a UC-secure PAKE and a UC-secure Oblivious Transfer constructions, this section serves as an example of application of our framework. More precisely, we apply here our (optimized) transformation to their UC-secure PAKE and show that we obtain exactly their UC-secure Oblivious Transfer (which is the Oblivious Transfer that inspired the instantiation of OLBE in the previous Chapter).

Description of Their UC-Secure PAKE Scheme

Description of the scheme (see Figure 5.6): Both parties (denoted as P_b and $P_{\bar{b}}$) want to share a key by using a common password pw . This will thus fix a language $\mathfrak{L}_{\text{pw}}$, which is the set of valid commitments (the linear encryption in their instantiation) of pw , for a smooth projective hash function. Both flows are done simultaneously. In flow_b , each party P_b will generate a pair of hash keys $(\text{hk}_b, \text{hp}_b)$ and then commit to the password pw_b in C_b . Note that P_b stores the witness δ_b that this value C_b is a valid commitment of pw_b and sends (hp_b, C_b) to the other party.

In the decapsulation phase, each party P_b then receives the commitment and the projection key of the other party and computes two hash values: On the one hand, the projected hash value using his own commitment C_b along with the witness δ_b and the projected key $\text{hp}_{\bar{b}}$ sent by the other party, and on the other hand, the (regular) hash value using the received commitment $C_{\bar{b}}$ and the secret hash key hk_b . The shared session key is then defined as the product of these two values.

The commitment scheme used has specific properties explained in [ABB⁺13] but we interest only needs it to be compatible with SPHF, and both extractable and equivocal.

Completeness: If the two parties used the same password pw , then at the end of the protocol they will share the same value. Thanks to the property of the SPHF, the hash value and the projected value (on the same commitment C_b) will be equal because the commitment belongs to the good language $\mathfrak{L}_{\text{pw}}$ with the good witness δ_b .

Applying the Framework to Obtain a UC-Secure OT Scheme

CRS generation:

$\text{crs} \xleftarrow{\$} \text{Setup}(1^{\mathbb{R}}), \text{param}_{\text{cpa}} \xleftarrow{\$} \text{Setup}_{\text{cpa}}(1^{\mathbb{R}}).$

Pre-Flow:

1. $\mathcal{S}$ generates a key pair $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}_{\text{cpa}}(\text{param}_{\text{cpa}})$ for $\mathcal{E}$, stores sk and completely erases the random coins used by $\text{KeyGen}_{\text{cpa}}$.
2. $\mathcal{S}$ publishes pk .

Index query on s :

1. $\mathcal{R}$ picks a random J , computes $R \leftarrow F(J)$ and sets $c \xleftarrow{\$} \text{Encrypt}_{\text{cpa}}(\text{pk}, J)$.
2. $\mathcal{R}$ picks a random ρ , and runs $\text{flow}_1(s; \rho, \perp)$ to obtain flow_0 .
3. $\mathcal{R}$ stores $f(\rho, \perp)$ needed for the Decap algorithm and R and completely erases the rest, including the random coins used by $\text{Encrypt}_{\text{cpa}}$ and sends (c, flow_0) to $\mathcal{S}$.

Database answer:

1. $\mathcal{S}$ decrypts $J \leftarrow \text{Decrypt}_{\text{cpa}}(\text{sk}, c)$ and computes $R \leftarrow F(J)$.
2. For each line k :
 - $\mathcal{S}$ picks a random ρ'_k , and runs $\text{flow}_2(\text{flow}_0, k; \perp, \rho'_k)$ to generate flow_k and $(K_k)_{\mathcal{R}}^{(\mathcal{S})}$.
 - $\mathcal{S}$ then computes $Q_k = L_k \oplus (K_k)_{\mathcal{R}}^{(\mathcal{S})} \oplus R$.
3. $\mathcal{S}$ erases everything except $(\text{flow}_k, Q_k)_{k \in \llbracket 1, n \rrbracket}$ and sends it to $\mathcal{R}$.

Data recovery: $\mathcal{R}$ then using $f(\rho, \perp)$ computes $(K_s)_{\mathcal{S}}^{(\mathcal{R})} = \text{Decap}(\text{flow}_s, s, f(\rho, \perp))$, sets $L_s = Q_s \oplus (K_s)_{\mathcal{S}}^{(\mathcal{R})} \oplus R$, and erases everything else.

FIGURE 5.5: OPTIMIZED UC-SECURE 1-OUT-OF- n OT FROM A UC-SECURE PAKE

- Setup: $\rho \xleftarrow{\$} \text{SetupCom}(1^{\mathfrak{K}})^a$.
- $\text{flow}_b(\text{pw}_b; \rho_b)$:
 - Generates $\text{hk}_b \xleftarrow{\$} \text{HashKG}(\mathfrak{L}_{\text{pw}_b})$, $\text{hp}_b \leftarrow \text{ProjKG}(\text{hk}_b, \mathfrak{L}_{\text{pw}_b}, \perp)$
 - picks additional ρ'_b and computes $(C_b, \delta_b) \xleftarrow{\$} \text{Com}^{\ell_b}(\text{pw}_b; \rho'_b)$ with $\ell_b = (\text{sid}, \text{ssid}, P_b, P_{\bar{b}}, \text{hp}_b)$. Sets $\rho_b = (\text{hk}_b, \rho'_b)$.
 - stores hk_b, δ_b , completely erases random coins used before and sends (hp_b, C_b) .
- Decap($\text{flow}_b, \text{pw}_b, f(\rho_b) = \text{hk}_b, \delta_b$):
 - Computes $H'_b \leftarrow \text{ProjHash}(\text{hp}_{\bar{b}}, \mathfrak{L}_{\text{pw}_b}, (\ell_b, C_b), \delta_b)$ and $H_{\bar{b}} \leftarrow \text{Hash}(\text{hk}_b, \mathfrak{L}_{\text{pw}_b}, (\ell_{\bar{b}}, C_{\bar{b}}))$ with $\ell_{\bar{b}} = (\text{sid}, \text{ssid}, P_{\bar{b}}, P_b, \text{hp}_{\bar{b}})$
 - Computes $\text{sk}_b = H'_b \cdot H_{\bar{b}}$ and erases everything else, except pw_b .

^aIt makes use of linear encryption as commitment and use their SPHF based on the language: linear encryption of valid password.

FIGURE 5.6: UC-SECURE PAKE FROM [ABB⁺13]

We described in section 2.8.2 the Oblivious Transfer from [ABB⁺13]. In this paper they proposed a PAKE and an OT as applications to their SPHF. With our (optimized) transformation we manage to come from one to the other.

Theorem 13. *As the PAKE is UC-Secure in presence of adaptive corruption under SXDH, the Oblivious Transfer scheme presented in Figure 2.17 is a secure 1-out-of- n Oblivious Transfer, with the same handling of corruptions as the PAKE protocol under SXDH and the IND-CPA security of the encryption scheme.*

On Another PAKE Scheme [KV09]

We are now going to show what happens when our general technique is used even on lattice-based PAKE. As an interesting example, we will apply it to the PAKE from [KV09] (see Figure 5.7). Lattices are drastically lacking a UC-PAKE, however we are going to apply our framework to this BPR secure scheme [BPR00].

We can see that this PAKE protocol is made of three flows. However the last flow is there to perform a key confirmation. The approximate SPHF used ensures that the two values (tk, tk') completed are close-enough (in term of Hamming weight), so that the random key sk generated and coded into c can then be decoded.

Applying the Framework to Obtain a Secure OT Scheme

CRS generation:

$(pk, sk) \xleftarrow{\$} \text{KeyGen}(\text{param})$ for $\mathcal{E}$. $\text{crs} = (pk)$

$\text{flow}_1(\text{pw}; r_1)$:

1. P_1 computes $C_1 = \text{Encrypt}(pk, \ell_1, \text{pw}; r_1)$ where $\ell_1 = (\text{sid}, \text{ssid}, P_1, P_2)$
2. P_1 sends C_1 to P_2

$\text{flow}_2(\text{pw}; r_2)$:

1. P_2 computes $C_2 = \text{Encrypt}(pk, \epsilon, \text{pw}; r_2)$.
2. P_2 generates $hk_2 \leftarrow \text{HashKG}(\mathcal{L}_{\text{pw}, \ell_1})$, $hp_2 \leftarrow \text{ProjKG}(hk_2, \mathcal{L}_{\text{pw}, \ell_1}, C_2)$.
3. P_2 sends (C_2, hp_2) to P_1 .

$\text{flow}_3(\text{pw}; r_2)$:

1. P_1 generates $hk_1 \xleftarrow{\$} \text{HashKG}(\mathcal{L}_{\text{pw}, \epsilon})$, $hp_1 \xleftarrow{\$} \text{ProjKG}(hk_1, \mathcal{L}_{\text{pw}, \epsilon}, C_1)$.
2. P_1 computes $\text{tk} = \text{Hash}(hk_1, \mathcal{L}_{\text{pw}, \epsilon}, C_2) \cdot \text{ProjHash}(hp_2, \mathcal{L}_{\text{pw}, \ell_1}, C_1, r_1)$
3. P_1 chooses $sk \xleftarrow{\$} \{0, 1\}^k$, computes $c = \text{ECC}(sk)$
4. P_1 set $\Delta = \text{tk} \oplus c$.
5. P_1 sends (Δ, hp_1) to P_2 .

Key computing for P_2 :

1. P_2 computes $\text{tk} = \text{Hash}(hk_2, \mathcal{L}_{\text{pw}, \ell_1}, C_1) \cdot \text{ProjHash}(hp_1, \mathcal{L}_{\text{pw}, \ell_2}, C_2, r_2,)$
2. P_2 recover the key $sk = \text{ECC}^{-1}(\text{tk} \oplus \Delta)$

FIGURE 5.7: LATTICE-BASED PAKE FROM [KV09]

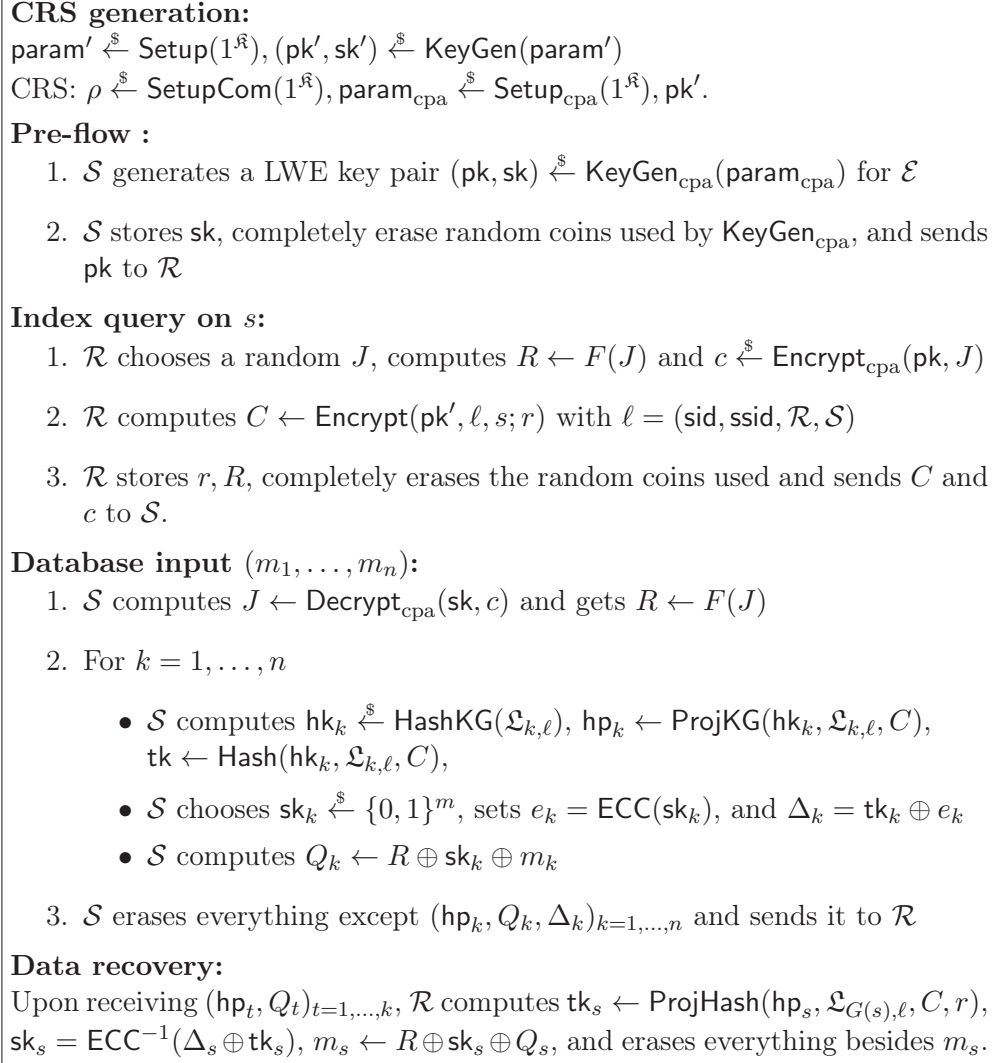


FIGURE 5.8: 1-OUT-OF- n OT FROM LATTICE-BASED PAKE

However, since with our framework the server does not need to commit to his password (the line index here), we can slightly modify the protocol and end up with the OT protocol exposed on Figure 5.8.

The security proof of this construction is a little beyond the point of this transformation, as the resulting scheme is not UC-Secure¹⁴, however the BPR security of the PAKE ensures some minimum requirements.

As a participant in the PAKE cannot learn the other password, this ensures that even after seeing the receiver's flows, the server does not learn the line requested. Similarly, since in case of password mismatch, a user cannot guess the key computed by the other one, this ensures that the user accesses at most one line.

¹⁴There is a problem to simulate an honest user. The Index query cannot be equivocated as the approximate SPHF Smoothness prevents the simulator to this circumvent this requirement.

In terms of efficiency, the resulting Oblivious Transfer ends up being slightly more efficient than both UC constructions from [GH08, BC15], but this comes at no surprise considering the security requirement gap. However this remains promising for the validity of our framework.

5.2.4 Very Efficient Oblivious Transfer from QA-NIZK

We now consider the instantiation from [JR15] in which the UC-PAKE allows for each party to hide his own password. As before, we first recall their protocol and then show how to use it in order to instantiate a new UC-secure Oblivious Transfer. This OT scheme ends up being the most efficient to date, as we show by comparing to the recent [BC16] in Table 5.11.

Interestingly this protocol does not use Smooth Projective Hash Function, but Quasi Adaptive Non-Interactive Zero Knowledge proofs [JR13]. It is presented in Figure 5.9.

A Quasi Adaptive Non-Interactive Zero Knowledge Argument is a NIZK (2.6) argument where the soundness property is computational but adaptive. Let us describe formally what is a Quasi Adaptive Non-Interactive Zero Knowledge Argument.

Definition 30 ((Labelled) Quasi Adaptive Non-Interactive Zero Knowledge Argument). *Assuming a randomness distribution $\mathcal{D}_{\text{param}}$ (for a public set of parameters param) and a set of languages $\{\mathcal{L}_\rho\}_\rho$ parameterized by a randomness $\rho \leftarrow \mathcal{D}_{\text{param}}$ and associated with a relation $\mathcal{R}_\rho$ (meaning that a word x belongs to $\mathcal{L}_\rho$ if and only if there exists a witness w such that $\mathcal{R}_\rho(x, w)$ holds), a QA-NIZK argument Π for this set of languages $\{\mathcal{L}_\rho\}_\rho$ consists of five probabilistic polynomial time (PPT) algorithms $\Pi = (\text{Gen}_{\text{param}}, \text{Gen}_{\text{crs}}, \text{Prove}, \text{Sim}, \text{Ver})$ defined as follows:*

- $\text{Gen}_{\text{param}}(\mathfrak{K})$ returns the public parameters param .
- $\text{Gen}_{\text{crs}}(\text{param}, \rho)$ returns a common reference string crs and a trapdoor tk . We assume that crs contain the parameters param , a description of the language $\mathcal{L}_\rho$ and a description of a label space $\mathcal{L}$.
- Prove takes as input crs , a label $\ell \in \mathcal{L}$, a word x of the language $\mathcal{L}_\rho$ and a witness w corresponding to this word. If $(x, w) \notin \mathcal{R}_\rho$, it outputs failure. Otherwise, it outputs a proof π .
- Ver takes as input crs , a label ℓ , a word x and a proof π . It outputs a bit b (either 1 if the proof is correct, or 0 otherwise).
- Sim takes as input crs , a trapdoor tk , a label $\ell \in \mathcal{L}$ and a word x . It outputs a proof π for x (not necessarily in $\mathcal{L}_\rho$) with respect to the label ℓ .

These algorithms must satisfy the following properties

Perfect Completeness: *for all PPT adversary $\mathcal{A}$, all security parameter $\mathfrak{K}$, all public parameters $\text{param} \leftarrow \text{Gen}_{\text{param}}(\mathfrak{K})$, all randomness $\rho \leftarrow \mathcal{D}_{\text{param}}$, all label $\ell \in \mathcal{L}$ and all $(x, w) \in \mathcal{R}_\rho$, the following holds:*

$$\Pr[(\text{crs}, \text{tk}) \leftarrow \text{Gen}_{\text{crs}}(\text{param}, \rho); \pi \leftarrow \text{Prove}(\text{crs}, \ell, x, w) : \text{Ver}(\text{crs}, \ell, x, \pi) = 1] = 1$$

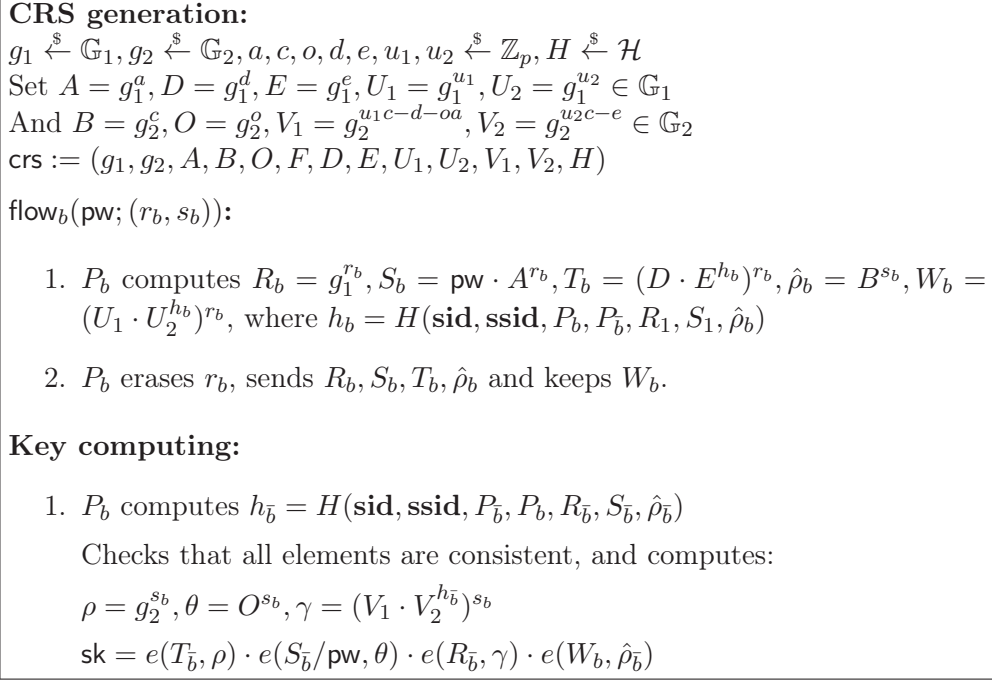


FIGURE 5.9: UC SECURE PAKE FROM [JR15]

Computational Adaptive Soundness: For a given security parameter $\mathfrak{K}$, a scheme achieves computational adaptive soundness if for all PPT adversary $\mathcal{A}$, the probability

$$\Pr[\text{param} \leftarrow \text{Gen}_{\text{param}}(\mathfrak{K}); \rho \leftarrow \mathcal{D}_{\text{param}}; (\text{crs}, \text{tk}) \leftarrow \text{Gen}_{\text{crs}}(\text{param}, \rho); \\ (\ell, x, \pi) \leftarrow \mathcal{A}(\text{param}, \text{crs}, \rho) : x \notin \mathcal{L}_\rho \wedge \text{Ver}(\text{crs}, \ell, x, \pi) = 1]$$

is negligible.

Perfect Zero-Knowledge: for all $\mathfrak{K}$, all $\text{param} \leftarrow \text{Gen}_{\text{param}}(\mathfrak{K})$, all $\rho \leftarrow \mathcal{D}_{\text{param}}$, all $(\text{crs}, \text{tk}) \leftarrow \text{Gen}_{\text{crs}}(\text{param}, \rho)$, all label $\ell \in \mathcal{L}$ and all $(x, w) \in \mathcal{R}_\rho$, the distributions $\text{Prove}(\text{crs}, \ell, x, w)$ and $\text{Sim}(\text{crs}, \text{tk}, \ell, x)$ are the same.

QA-NIZK-based PAKE

As explained in the original paper [JR15], the scheme is loosely-based on Quasi-Adaptive NIZK, in the sense that the flows use computation very close to what would be expected. In the figure 5.9, $\mathcal{H}$ is a family of hash functions.

5.2.5 Applying the Framework to Obtain a UC-Secure OT Scheme

Using the instantiation from [JR15] described in Figure 5.9, we apply our framework and directly obtain the protocol described in Figure 5.10.

The function G is assumed to map line numbers to group elements, and it should be collision-resistant and efficiently computable. One can use for example, the G function from Lindell in [Lin11].

CRS generation:

$g_1 \xleftarrow{\$} \mathbb{G}_1, g_2 \xleftarrow{\$} \mathbb{G}_2, a, c, o, d, e, u_1, u_2 \xleftarrow{\$} \mathbb{Z}_p, H \xleftarrow{\$} \mathcal{H}$
 Sets $A = g_1^a, D = g_1^d, E = g_1^e, U_1 = g_1^{u_1}, U_2 = g_1^{u_2} \in \mathbb{G}_1$
 And $B = g_2^c, O = g_2^o, V_1 = g_2^{u_1 c - d - o a}, V_2 = g_2^{u_2 c - e} \in \mathbb{G}_2$
 $\text{crs} := (g_1, g_2, A, B, O, F, D, E, U_1, U_2, V_1, V_2, H)$

Pre-Flow:

1. $\mathcal{S}$ generates $(\text{pk} = f_1 = g_1^\alpha, \text{sk} = \alpha \xleftarrow{\$} \mathbb{Z}_p)$
2. $\mathcal{S}$ erases random coins, keeps sk and sends pk .

Index query on i :

1. $\mathcal{R}$ picks $J \xleftarrow{\$} \mathbb{G}_1, s \xleftarrow{\$} \mathbb{Z}_p$, sets $\text{mask} \leftarrow F(J)$ and $c = (f_1^s J, g_1^s)$
2. $\mathcal{R}$ picks $r \xleftarrow{\$} \mathbb{Z}_p$ computes $R = g_1^r, S = G(i) \cdot A^r, T = (D \cdot E^{h_{\mathcal{R}, S}})^r, W = (U_1 \cdot U_2^{h_{\mathcal{R}, S}})^r$, where $h_{\mathcal{R}, S} = H(\text{sid}, \text{ssid}, \mathcal{R}, S, R, S, c)$
3. $\mathcal{R}$ erases r, s, J , sends $c, C = (R, S, T)$ and keeps W, mask .

Database answer:

1. $\mathcal{S}$ sets $J = c_1/c_2^\alpha$, $\text{mask} = F(J)$ and $h_{\mathcal{R}, S} = H(\text{sid}, \text{ssid}, \mathcal{R}, S, R, S, c)$
2. For $k = 1, \dots, n$:
 - (a) Picks $s_k \xleftarrow{\$} \mathbb{Z}_p$
 - (b) Sets $\rho_k = B^{s_k}, \rho'_k = g_2^{s_k}, \theta_k = O^{s_k}, \gamma_k = (V_1 \cdot V_2^{h_{\mathcal{R}, S}})^{s_k}$
 - (c) Computes $K_{\mathcal{R}, S, k} = e(T, \rho'_k) \cdot e(S/G(k), \theta_k) \cdot e(R, \gamma_k)$
 - (d) Sets $Q_k = L_k \oplus \mathcal{H}(K_{\mathcal{R}, S, k} \oplus \text{mask})$
3. $\mathcal{S}$ sends all $(Q_k, \rho_k)_{k \in [1, n]}$ and erases everything else.

Data recovery:

$\mathcal{R}$ computes $K_{\mathcal{R}, S, i} = e(W, \rho_i)$, and recovers $L_i = Q_i \oplus \mathcal{H}(K_{\mathcal{R}, S, i} \oplus \text{mask})$

FIGURE 5.10: UC SECURE OT FROM QA-NIZK

Theorem 14. *As the PAKE was UC-Secure in presence of adaptive corruption under SXDH, and ElGamal is IND-CPA under SXDH too, the Oblivious Transfer scheme presented in Figure 5.10 is a secure 1-out-of- n Oblivious Transfer, with the same handling of corruptions as the PAKE protocol under SXDH.*

The PAKE scheme directly fits in the framework, hence the security is inherited from the initial PAKE proven secure in [JR15].

The same basic arguments apply: We first start by switching the crs so as to be able to simulate password openings, which will allow an honest receiver to commit to a dummy line value, and retroactively compute the opening value for any possible line s . On the other hand, an honest server will extract the committed line number, and provide dummy answers for all the other ones. If the receiver is honest, then the server will also send a dummy answer for the valid line, since the encrypted value S provides the extra degree of freedom to open to any line value L_k provided belatedly by the environment.

Efficiency Comparison We compare our efficient UC secure Oblivious Transfer with recent ones. This is depicted in figure 5.11. Note that in the case where there are many queries for the same database the adaptive OT from Section 4.3 will be more efficient.

Communication cost comparisons of various Elliptic Curve based OT schemes

Paper	Assumption	# Group elements	# Rounds
Adaptive Security (1-out-of-2)			
[ABB ⁺ 13]	SXDH	$12 \mathbb{G}_1 + 1 \mathbb{G}_2 + 2\mathbb{Z}_p$	3
[BC15] ¹⁵	DDH	$15 \mathbb{G} + 2 \mathbb{Z}_p$	3
[BC16]	SXDH	$12 \mathbb{G}_1 + 4 \mathbb{G}_2 + 2 \mathbb{Z}_p$	3
This paper	SXDH	$6 \mathbb{G}_1 + 2 \mathbb{G}_2 + 2 \mathbb{Z}_p$	3
Adaptive Security (1-out-of- n)			
[ABB ⁺ 13]	SXDH	$\log n \mathbb{G}_1 + (n + 8 \log n) \mathbb{G}_2 + n \mathbb{Z}_p$	3
[BC15] ¹⁵	DDH	$(n + 9 \log n + 4) \mathbb{G} + 2n \mathbb{Z}_p$	3
[BC16]	SXDH	$4 \mathbb{G}_1 + (4n + 4) \mathbb{G}_2 + n \mathbb{Z}_p$	3
This paper	SXDH	$4 \mathbb{G}_1 + (n + 2) \mathbb{G}_2 + n \mathbb{Z}_p$	3

FIGURE 5.11: COMPARISON TO EXISTING OBLIVIOUS TRANSFER

¹⁵It should be noted that the [BC15] OT does not rely on pairings. Classical implementations suggest that the elements in one of the groups in our scheme will be at least twice as big as those from the non-pairing curve, which would give roughly the same efficiency between both schemes in the 1-out-of-2 case, with an edge for the [BC15] construction in term of computational requirements. However, the scaling in their paper is then worse when increasing the number of lines (see the 1-out-of- n case).

Chapter 6

Conclusion

Conclusion In this thesis we explored a lot of possible applications to Identity-based Cryptography, creating new Identity-based primitives each time we needed them. Sometimes new kind of primitives like DIBE and sometimes new instantiation/transformation like our transformation from IBE to BIBE. Also we focused on Oblivious Transfer itself with and without Smooth Projective Hash Functions. They were useful to find an instantiation of OLBE and secure our adaptive version of the OT but too slow to create a very fast OT.

Open Problems and future Work There are different axes of research in this thesis which leads to different future works. I think that since a lot of this thesis is based on IBE any new IBE with better efficiency could impact most of it. Indeed better building blocks allow better transformations. For example if we could find a tightly-secure DIBE, this will increase the impact of our work and give a more practical scheme. In my opinion our transformation from DIBE to ABE could be a starting point for future works. First the DIBE scheme itself is new and can maybe find different uses. We already describe how it can instantiate some IBE's variant, it would be interesting to check every applications of these primitives and see if our efficient instantiation can solve practicality problems. Secondly the fact that the access structure of the ABE has to be a DNF formula is a bit restrictive, it would be great to generalize to any kind of access structure without increasing the communication cost that much e.g. staying linear in the number of attributes.

Bibliography

- [ABB10] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 553–572. Springer, Heidelberg, May 2010.
- [ABB⁺13] Michel Abdalla, Fabrice Benhamouda, Olivier Blazy, Céline Chevalier, and David Pointcheval. SPHF-friendly non-interactive commitments. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part I*, volume 8269 of *LNCS*, pages 214–234. Springer, Heidelberg, December 2013.
- [ACD⁺06] Michel Abdalla, Dario Catalano, Alex Dent, John Malone-Lee, Gregory Neven, and Nigel Smart. Identity-based encryption gone wild. In Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener, editors, *ICALP 2006, Part II*, volume 4052 of *LNCS*, pages 300–311. Springer, Heidelberg, July 2006.
- [ACP09] Michel Abdalla, Céline Chevalier, and David Pointcheval. Smooth projective hashing for conditionally extractable commitments. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 671–689. Springer, Heidelberg, August 2009.
- [AFG⁺10] Masayuki Abe, Georg Fuchsbauer, Jens Groth, Kristiyan Haralambiev, and Miyako Ohkubo. Structure-preserving signatures and commitments to group elements. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 209–236. Springer, Heidelberg, August 2010.
- [AKN07] Michel Abdalla, Eike Kiltz, and Gregory Neven. Generalized key delegation for hierarchical identity-based encryption. In Joachim Biskup and Javier López, editors, *ESORICS 2007*, volume 4734 of *LNCS*, pages 139–154. Springer, Heidelberg, September 2007.
- [Att16] Nuttapong Attrapadung. Dual system encryption framework in prime-order groups via computational pair encodings. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 591–623. Springer, Heidelberg, December 2016.

- [BB04] Dan Boneh and Xavier Boyen. Efficient selective-ID secure identity based encryption without random oracles. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 223–238. Springer, Heidelberg, May 2004.
- [BBC⁺13a] Fabrice Ben Hamouda, Olivier Blazy, Céline Chevalier, David Pointcheval, and Damien Vergnaud. Efficient UC-secure authenticated key-exchange for algebraic languages. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 272–291. Springer, Heidelberg, February / March 2013.
- [BBC⁺13b] Fabrice Benhamouda, Olivier Blazy, Céline Chevalier, David Pointcheval, and Damien Vergnaud. New techniques for SPHF and efficient one-round PAKE protocols. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 449–475. Springer, Heidelberg, August 2013.
- [BBG05] Dan Boneh, Xavier Boyen, and Eu-Jin Goh. Hierarchical identity based encryption with constant size ciphertext. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 440–456. Springer, Heidelberg, May 2005.
- [BBS04] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 41–55. Springer, Heidelberg, August 2004.
- [BC15] Olivier Blazy and Céline Chevalier. Generic construction of UC-secure oblivious transfer. In Tal Malkin, Vladimir Kolesnikov, Allison Bishop Lewko, and Michalis Polychronakis, editors, *ACNS 15*, volume 9092 of *LNCS*, pages 65–86. Springer, Heidelberg, June 2015.
- [BC16] Olivier Blazy and Céline Chevalier. Structure-preserving smooth projective hashing. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 339–369. Springer, Heidelberg, December 2016.
- [BCG16] Olivier Blazy, Céline Chevalier, and Paul Germouty. Adaptive oblivious transfer and generalization. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 217–247. Springer, Heidelberg, December 2016.
- [BCG17] Olivier Blazy, Céline Chevalier, and Paul Germouty. Almost optimal oblivious transfer from QA-NIZK. In Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, editors, *ACNS 17*, volume 10355 of *LNCS*, pages 579–598. Springer, Heidelberg, July 2017.
- [BCGJ17] Olivier Blazy, Emmanuel Conchon, Paul Germouty, and Amandine Jambert. Efficient id-based designated verifier signature. *Internation-*

tional Conference on Availability, Reliability and Security, 12(44), 2017.

- [BDNS07] James Birkett, Alexander W. Dent, Gregory Neven, and Jacob C. N. Schuldt. Efficient chosen-ciphertext secure identity-based encryption with wildcards. In Josef Pieprzyk, Hossein Ghodosi, and Ed Dawson, editors, *ACISP 07*, volume 4586 of *LNCS*, pages 274–292. Springer, Heidelberg, July 2007.
- [BF01] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the Weil pairing. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 213–229. Springer, Heidelberg, August 2001.
- [BFT⁺10] Olivier Blazy, Georg Fuchsbauer, Malika Izabachène, Amandine Jambert, Hervé Sibert, and Damien Vergnaud. Batch Groth-Sahai. In Jianying Zhou and Moti Yung, editors, *ACNS 10*, volume 6123 of *LNCS*, pages 218–235. Springer, Heidelberg, June 2010.
- [BFPV11] Olivier Blazy, Georg Fuchsbauer, David Pointcheval, and Damien Vergnaud. Signatures on randomizable ciphertexts. In Dario Catalano, Nelly Fazio, Rosario Gennaro, and Antonio Nicolosi, editors, *PKC 2011*, volume 6571 of *LNCS*, pages 403–422. Springer, Heidelberg, March 2011.
- [BKP14] Olivier Blazy, Eike Kiltz, and Jiaxin Pan. (Hierarchical) identity-based encryption from affine message authentication. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 408–425. Springer, Heidelberg, August 2014.
- [BPR00] Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 139–155. Springer, Heidelberg, May 2000.
- [BPV12] Olivier Blazy, David Pointcheval, and Damien Vergnaud. Round-optimal privacy-preserving protocols with smooth projective hash functions. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 94–111. Springer, Heidelberg, March 2012.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- [CCG⁺94] Richard Chang, Benny Chor, Oded Goldreich, Juris Hartmanis, Johan Hastad, Desh Ranjan, and Pankaj Rohatgi. The random oracle is false. *Journal of Computer and System Sciences*, 49(1):24–39, 1994.

- [CCL15] Ran Canetti, Asaf Cohen, and Yehuda Lindell. A simpler variant of universally composable security for standard multiparty computation. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 3–22. Springer, Heidelberg, August 2015.
- [CDN09] Jan Camenisch, Maria Dubovitskaya, and Gregory Neven. Oblivious transfer with access control. In Ehab Al-Shaer, Somesh Jha, and Angelos D. Keromytis, editors, *ACM CCS 09*, pages 131–140. ACM Press, November 2009.
- [CDVW12] Ran Canetti, Dana Dachman-Soled, Vinod Vaikuntanathan, and Hoeteck Wee. Efficient password authenticated key exchange via oblivious transfer. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012*, volume 7293 of *LNCS*, pages 449–466. Springer, Heidelberg, May 2012.
- [CFH⁺07] Yang Cui, Eiichiro Fujisaki, Goichiro Hanaoka, Hideki Imai, and Rui Zhang. Formal security treatments for signatures from identity-based encryption. In Willy Susilo, Joseph K. Liu, and Yi Mu, editors, *ProvSec 2007*, volume 4784 of *LNCS*, pages 218–227. Springer, Heidelberg, November 2007.
- [CGW15] Jie Chen, Romain Gay, and Hoeteck Wee. Improved dual system ABE in prime-order groups via predicate encodings. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 595–624. Springer, Heidelberg, April 2015.
- [CHK⁺05] Ran Canetti, Shai Halevi, Jonathan Katz, Yehuda Lindell, and Philip D. MacKenzie. Universally composable password-based key exchange. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 404–421. Springer, Heidelberg, May 2005.
- [CHKP10] David Cash, Dennis Hofheinz, Eike Kiltz, and Chris Peikert. Bonsai trees, or how to delegate a lattice basis. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 523–552. Springer, Heidelberg, May 2010.
- [CK02] Ran Canetti and Hugo Krawczyk. Universally composable notions of key exchange and secure channels. In Lars R. Knudsen, editor, *EUROCRYPT 2002*, volume 2332 of *LNCS*, pages 337–351. Springer, Heidelberg, April / May 2002.
- [CKWZ13] Seung Geol Choi, Jonathan Katz, Hoeteck Wee, and Hong-Sheng Zhou. Efficient, adaptively secure, and composable oblivious transfer with a single, global CRS. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 73–88. Springer, Heidelberg, February / March 2013.

- [Coc01] Clifford Cocks. An identity based encryption scheme based on quadratic residues. In Bahram Honary, editor, *8th IMA International Conference on Cryptography and Coding*, volume 2260 of *LNCS*, pages 360–363. Springer, Heidelberg, December 2001.
- [CS98] Ronald Cramer and Victor Shoup. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In Hugo Krawczyk, editor, *CRYPTO'98*, volume 1462 of *LNCS*, pages 13–25. Springer, Heidelberg, August 1998.
- [CS02] Ronald Cramer and Victor Shoup. Universal hash proofs and a paradigm for adaptive chosen ciphertext secure public-key encryption. In Lars R. Knudsen, editor, *EUROCRYPT 2002*, volume 2332 of *LNCS*, pages 45–64. Springer, Heidelberg, April / May 2002.
- [DG17] Nico Döttling and Sanjam Garg. Identity-based encryption from the Diffie-Hellman assumption. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 537–569. Springer, Heidelberg, August 2017.
- [DH76] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- [EHK⁺13] Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Villar. An algebraic framework for Diffie-Hellman assumptions. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 129–147. Springer, Heidelberg, August 2013.
- [ElG84] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. In G. R. Blakley and David Chaum, editors, *CRYPTO'84*, volume 196 of *LNCS*, pages 10–18. Springer, Heidelberg, August 1984.
- [EMJ17] Nadia El Mrabet and Marc Joye. *Guide to pairing-based cryptography*. Chapman and Hall/CRC, 2017.
- [Fis06] Marc Fischlin. Round-optimal composable blind signatures in the common reference string model. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 60–77. Springer, Heidelberg, August 2006.
- [GH07] Matthew Green and Susan Hohenberger. Blind identity-based encryption and simulatable oblivious transfer. In Kaoru Kurosawa, editor, *ASIACRYPT 2007*, volume 4833 of *LNCS*, pages 265–282. Springer, Heidelberg, December 2007.

- [GH08] Matthew Green and Susan Hohenberger. Universally composable adaptive oblivious transfer. In Josef Pieprzyk, editor, *ASIACRYPT 2008*, volume 5350 of *LNCS*, pages 179–197. Springer, Heidelberg, December 2008.
- [GHPT17] Philippe Gaborit, Adrien Hauteville, Duong Hieu Phan, and Jean-Pierre Tillich. Identity-based encryption from codes with rank metric. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part III*, volume 10403 of *LNCS*, pages 194–224. Springer, Heidelberg, August 2017.
- [GL03] Rosario Gennaro and Yehuda Lindell. A framework for password-based authenticated key exchange. In Eli Biham, editor, *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 524–543. Springer, Heidelberg, May 2003. <http://eprint.iacr.org/2003/032.ps.gz>.
- [GMR88] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, April 1988.
- [GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008.
- [GS02] Craig Gentry and Alice Silverberg. Hierarchical ID-based cryptography. In Yuliang Zheng, editor, *ASIACRYPT 2002*, volume 2501 of *LNCS*, pages 548–566. Springer, Heidelberg, December 2002.
- [GS08] Jens Groth and Amit Sahai. Efficient non-interactive proof systems for bilinear groups. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 415–432. Springer, Heidelberg, April 2008.
- [Gui13] Aurore Guillevic. Comparing the pairing efficiency over composite-order and prime-order elliptic curves. In Michael J. Jacobson Jr., Michael E. Locasto, Payman Mohassel, and Reihaneh Safavi-Naini, editors, *ACNS 13*, volume 7954 of *LNCS*, pages 357–372. Springer, Heidelberg, June 2013.
- [HILL99] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999.

- [JR13] Charanjit S. Jutla and Arnab Roy. Shorter quasi-adaptive NIZK proofs for linear subspaces. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part I*, volume 8269 of *LNCS*, pages 1–20. Springer, Heidelberg, December 2013.
- [JR15] Charanjit S. Jutla and Arnab Roy. Dual-system simulation-soundness with applications to UC-PAKE and more. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015, Part I*, volume 9452 of *LNCS*, pages 630–655. Springer, Heidelberg, November / December 2015.
- [Kal05] Yael Tauman Kalai. Smooth projective hashing and two-message oblivious transfer. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 78–95. Springer, Heidelberg, May 2005.
- [KV09] Jonathan Katz and Vinod Vaikuntanathan. Smooth projective hashing and password-based authenticated key exchange from lattices. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 636–652. Springer, Heidelberg, December 2009.
- [LDB03] Ninghui Li, Wenliang Du, and Dan Boneh. Oblivious signature-based envelope. In Elizabeth Borowsky and Sergio Rajsbaum, editors, *22nd ACM PODC*, pages 182–189. ACM, July 2003.
- [Lin11] Yehuda Lindell. Highly-efficient universally-composable commitments based on the DDH assumption. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 446–466. Springer, Heidelberg, May 2011.
- [LW12] Allison B. Lewko and Brent Waters. New proof methods for attribute-based encryption: Achieving full security through selective techniques. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 180–198. Springer, Heidelberg, August 2012.
- [NY90] Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *22nd ACM STOC*, pages 427–437. ACM Press, May 1990.
- [OT10] Tatsuaki Okamoto and Katsuyuki Takashima. Fully secure functional encryption with general relations from the decisional linear assumption. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 191–208. Springer, Heidelberg, August 2010.
- [Ped92] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *CRYPTO’91*, volume 576 of *LNCS*, pages 129–140. Springer, Heidelberg, August 1992.

- [RS92] Charles Rackoff and Daniel R. Simon. Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In Joan Feigenbaum, editor, *CRYPTO'91*, volume 576 of *LNCS*, pages 433–444. Springer, Heidelberg, August 1992.
- [RSA78] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signature and public-key cryptosystems. *Communications of the Association for Computing Machinery*, 21(2):120–126, 1978.
- [SBWP03] Ron Steinfeld, Laurence Bull, Huaxiong Wang, and Josef Pieprzyk. Universal designated-verifier signatures. In Chi-Sung Lai, editor, *ASIACRYPT 2003*, volume 2894 of *LNCS*, pages 523–542. Springer, Heidelberg, November / December 2003.
- [Sha84] Adi Shamir. Identity-based cryptosystems and signature schemes. In G. R. Blakley and David Chaum, editors, *CRYPTO'84*, volume 196 of *LNCS*, pages 47–53. Springer, Heidelberg, August 1984.
- [SW05] Amit Sahai and Brent R. Waters. Fuzzy identity-based encryption. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 457–473. Springer, Heidelberg, May 2005.
- [Wat05] Brent R. Waters. Efficient identity-based encryption without random oracles. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 114–127. Springer, Heidelberg, May 2005.
- [Wat09] Brent Waters. Dual system encryption: Realizing fully secure IBE and HIBE under simple assumptions. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 619–636. Springer, Heidelberg, August 2009.
- [Zhe97] Yuliang Zheng. Digital signcryption or how to achieve $\text{cost}(\text{signature} \ \& \ \text{encryption}) \ll \text{cost}(\text{signature}) + \text{cost}(\text{encryption})$. In Burton S. Kaliski Jr., editor, *CRYPTO'97*, volume 1294 of *LNCS*, pages 165–179. Springer, Heidelberg, August 1997.

Abstract

We investigate the possibilities that **Identity-based Encryption** offers when used out of their original purpose. We manage to generalize a whole class of different identity-based encryption schemes into Downgradable Identity-based Encryptions. We found a generic way to construct Blind Identity-based Encryptions. These two works lead both to applications that are not a priori linked with IBE: Attribute-based Encryption from Downgradable IBE and Oblivious Transfer from Blind IBE. In the case of Affine IBE we manage to reduce the communication cost from linear to logarithmic. As application we also find a way to use Hierarchical IBE to construct a special type of signature called Identity-based Designated Verifier Signature. We continue the research out of the context of IBE's application with Oblivious Transfer. We manage to generalize the concept of Oblivious Transfer into a new protocol called Oblivious Language-based Envelope encompassing many kind of protocols. Finally, we construct Oblivious Transfer with a very different primitive called Password Authenticated Key Exchange. Surprisingly, with some optimizations this last transformation leads to a very efficient Oblivious Transfer Protocol. The Identity-based Encryption is our main basis of work, thus efficient instantiations of this primitive were the key of our own efficiency, thus we used the instantiation from Blazy et als at CRYPTO'14 which is efficient, tight secure and affine.

Résumé

Dans cette thèse nous étudions les possibilités que les **chiffrements basés sur l'identité** offrent quand ils sont utilisés dans un but différent qu'un simple chiffrement. Nous avons pu généraliser différents types de chiffrement basés sur l'identité en une nouvelle primitive nommé Downgradable Identity-based Encryption (DIBE). Nous avons trouvé un moyen générique de transformer de simple IBE en des IBE en blanc, dans le cas où l'IBE est affine nous rendons le coût de communication très faible (de linéaire à logarithmique). Ces deux primitives ont donné lieux à différentes applications : les chiffrements basés sur les attributs pour la première et le transfère inconscient pour la deuxième. Une autre application est l'utilisation d'IBE hiérarchiques pour créer des signatures à vérifieur désigné basées sur l'identité. Ensuite nous nous avons regardé le transfère inconscient seul et avons réussi à le généraliser en un nouveau protocole nommé Oblivious Language-based Envelope. Finalement, nous avons construit une transformation d'un protocole à un autre, d'un échange authentifié de clés par mot de passe nous avons construit un transfère inconscient. En prenant une instantiation particulière nous obtenons un protocole plus efficace que tous les précédents pour le même niveau de sécurité. La primitive chiffrement basé sur l'identité est notre outil principal pour réaliser nos constructions. Nous avons donc besoin d'une instantiation efficace de cette primitive. Nous avons utilisé celle de Blazy Kiltz et Pan à CRYPTO'14 qui est très efficace mais possède aussi une structure particulière dite affine.